Data Analysis & Visualisation

A reader for MADS-DAV

Raoul Grouls

November 3, 2026

Contents

1 Introduction **6**

2 Lesson 4: Distributions **7**
2.1 What is a distribution? 7
2.1.1 Two kinds of not knowing 7
2.1.2 Three rules make something a distribution 8
2.1.3 Continuous distributions: there is 0 chance of exactly 21.0 °C 8
2.1.4 Same rolls, two plots: only one is a distribution 9
2.1.5 Start from what you know about the process 9
2.1.6 Six shapes, six ways numbers come about 10
2.1.7 Adding makes a bell; multiplying makes a long tail 10
2.2 Which distribution fits my data? 11
2.2.1 Messages per day: the rate isn't steady 12
2.2.2 A strange best fit can mean several groups 12
2.2.3 A qq-plot shows where model and data disagree 13
2.3 What is an outlier? 14
2.3.1 What does an outlier mean? 14
2.3.2 The bell has no room for expensive rides 15
2.3.3 Outliers, or the wrong model? 15
2.3.4 The whisker rule makes four assumptions 16
2.3.5 What not to do: judge data without the process 16
2.3.6 One $150 ride, three verdicts 17
2.3.7 Ordinary among all penguins, rare among her own 18
2.4 Did something change, or is it chance? 19
2.4.1 Did this person's texting change? 20
2.4.2 Shuffle the labels to see what chance does 20
2.4.3 Bayesian Methods for Hackers: a confident switch 21
2.4.4 Are those assumptions right? 21
2.4.5 Searching for the day costs you the evidence 22
2.4.6 Same lift, two verdicts 23
2.5 Is my model done? 24
2.5.1 Before vaccination, the residual looks like chance 25
2.5.2 The residual reveals what the model missed 25

2.6 The five questions, answered . . . 26
2.7 Feedback on your distribution plots . . . 27

3 Lesson 5: Modelling relationships 28
3.1 What is a model? . . . 28
3.1.1 Leonardo's earth: not true, still useful . . . 29
3.1.2 Three lines through the same cars . . . 29
3.2 Why keep a model simple? . . . 30
3.2.1 Four shapes worth knowing by name . . . 31
3.2.2 Nothing in nature grows exponentially for long . . . 32
3.2.3 A line plus a sine: four numbers, 93% explained . . . 32
3.2.4 A better fit is good, until you fit the noise . . . 33
3.2.5 A model that can fit anything tells you nothing . . . 33
3.2.6 Too simple, about right, too flexible . . . 34
3.2.7 If you can't explain a number, don't report it . . . 35
3.3 How do you build a model? . . . 36
3.3.1 Step 0 · A running total. How many died per day? . . . 36
3.3.2 Preattentive processing: what do you see? . . . 37
3.3.3 Step 1 · Too much noise: average over a week . . . 38
3.3.4 Step 2 · Deaths come about two weeks after a positive test . . . 38
3.3.5 Step 3 · Tens against thousands: compare with z-scores . . . 39
3.3.6 Step 4 · Add what you know: vaccination started on 6 January . . . 40
3.3.7 Add b: the deaths the tests don't explain . . . 41
3.3.8 Step 5 · Say how wrong the model is allowed to be . . . 42
3.3.9 Every step of reasoning becomes a line of code . . . 42
3.3.10 Previous lesson: the residual shows what the line misses . . . 43
3.3.11 How do you model a vaccine? Look at the ratio . . . 43
3.3.12 A logistic switch has two knobs: when and how fast . . . 44
3.3.13 The switch, fitted: the date falls out of the fit . . . 45
3.3.14 The residual looks like chance again . . . 46
3.3.15 The whole loop as a map, drawn by atlas . . . 47
3.4 When should you believe a pattern? . . . 48
3.4.1 Fifteen comparisons, one below the line . . . 48
3.4.2 How often do you find something when there is nothing? . . . 49
3.4.3 A real pattern predicts people it hasn't seen . . . 49
3.4.4 Three legs make a finding believable . . . 50
3.4.5 The comparison you choose is part of the claim . . . 51
3.4.6 Test at the level of your claim . . . 52
3.5 What can a correlation hide? . . . 53
3.5.1 The same penguins, twice: the line turns around . . . 53
3.5.2 Seven of fifteen correlations change sign . . . 54
3.5.3 A penalty on the weights: what L1 and L2 do . . . 54
3.5.4 A penalty on the weights finds the useful features . . . 55
3.6 Can you recognise someone by how they type? . . . 56
3.6.1 47% right, but why? . . . 56
3.6.2 Three ways to write a smiley, and nobody mixes them . . . 57
3.6.3 Fifty messages at a time: ten habits do what 17,918 words do . . . 58
3.6.4 The same eight people, four years later . . . 58

3.7 The six questions, answered . . . 60

4 Lesson 6: Many dimensions 61
4.1 What does PCA show you? . . . 61
4.1.1 The line with the most spread loses the least . . . 62
4.1.2 PCA finds new axes, and you choose how many to keep . . . 62
4.1.3 Change the unit, and the first component changes . . . 63
4.1.4 Standardise first, or PCA measures your units . . . 64
4.2 When is a flat picture not enough? . . . 65
4.2.1 PCA casts a shadow; t-SNE unrolls the sheet . . . 65
4.2.2 The manifold hypothesis: many columns, few real dimensions . . . 66
4.2.3 On 784 pixels the shadow is a smear; t-SNE finds ten groups . . . 66
4.2.4 t-SNE: the map must pick the same neighbours . . . 67
4.2.5 Which points group together survives; where the groups sit does not 68
4.2.6 Preattentive processing: what do you see? . . . 69
4.2.7 PCA or t-SNE? Two tools that answer different questions . . . 69
4.3 How does text become a vector? . . . 70
4.3.1 A trigram is any run of three characters . . . 70
4.3.2 “QAnon is two different people” . . . 71
4.3.3 What separates the 4chan months: questions, and links . . . 71
4.3.4 The number that did not move answered another question . . . 72
4.3.5 Every stretch of time resembles itself . . . 73
4.3.6 This picture is not evidence for two authors . . . 73
4.4 Why use a model somebody else trained? . . . 74
4.4.1 The encoder runs once; the head trains on what it stored . . . 74
4.4.2 Without a cache, the encoder does nearly all the work, every epoch 74
4.4.3 An organised space, before anything is trained . . . 75
4.4.4 Cosine similarity compares the direction, not the length . . . 76
4.4.5 Nearest neighbour: copy the label of the most similar photo . . . 76
4.5 What is one row, when a row is “haha”? . . . 78
4.5.1 Who wrote these? . . . 78
4.5.2 One message per row: no authors in the picture . . . 78
4.5.3 Three ways to merge messages into one row . . . 79
4.5.4 Does the second half of a unit belong with its first half? . . . 79
4.5.5 Near in meaning, not in habit . . . 80
4.5.6 A race: which vector knows who is writing? . . . 81
4.5.7 Check a model on messages where you know the answer . . . 82
4.6 Why did k-means find three clusters? . . . 83
4.6.1 k-means finds three groups in anything . . . 83
4.6.2 The three groups are short, medium and long messages . . . 84
4.6.3 A picture of 7% is a shadow of 7% . . . 84
4.6.4 If a tool always finds something, finding something proves nothing . 85
4.7 Up, and back down: what to do with your plot . . . 85
4.7.1 Map data up to many dimensions, then back down to two to look . 85
4.7.2 A picture tells a story when two routes agree . . . 86
4.8 The six questions, answered . . . 87
4.9 Feedback on your high-dimensional plots . . . 88

5 Lesson 7: Networks 89

5.1 What is a graph? . . . 89
5.1.1 A graph is two sets: nodes and edges . . . 89
5.1.2 Nodes and edges can be anything . . . 90
5.1.3 Edges can have a weight, and nodes a kind . . . 91
5.2 What are the nodes and edges in your chat? . . . 92
5.2.1 Two metrics to define a relationship . . . 92
5.2.2 The window is a metric too . . . 93
5.2.3 Topics: what is mentioned together . . . 94
5.2.4 Different lenses on the same dataset . . . 95
5.3 What does a graph measure add? . . . 95
5.3.1 The Medici were the broker . . . 96
5.3.2 Are your friends friends with each other? . . . 97
5.3.3 A small world: closed circles, and a few shortcuts . . . 98
5.3.4 An open friendship circle, and thoughts of suicide . . . 99
5.3.5 How you construct an edge has impact downstream . . . 100
5.3.6 Is the line carried by the loudest few? . . . 101
5.3.7 Which degree adds something? . . . 102
5.3.8 Does the window decide who is central? . . . 103
5.4 Graph or heatmap? . . . 105
5.4.1 One graph, two layouts, identical degrees . . . 105
5.4.2 The same network as a matrix . . . 106
5.4.3 Sometimes the drawing wins . . . 106
5.4.4 Sometimes the matrix wins . . . 107
5.4.5 A dashboard: ask your agent for one . . . 108
5.5 Can an algorithm find the groups? . . . 109
5.5.1 A karate club that split in two . . . 110
5.5.2 One eigenvector predicts the split . . . 111
5.5.3 Louvain maximises modularity . . . 112
5.5.4 Louvain on the words: five topics . . . 112
5.6 Are the groups real? . . . 113
5.6.1 Louvain colours every network into tidy clumps . . . 114
5.6.2 Is Q more than chance? Rewire and compare . . . 114
5.6.3 Finding hubs typically needs support from additional data . . . 115
5.6.4 Do the groups come back? . . . 116
5.7 A network analysis in four steps . . . 117
5.8 The six questions, answered . . . 118
5.9 Feedback on your network plots . . . 119

6 Reading code with atlas **121**
6.1 The problem with reading a lot of code . . . 121
6.1.1 Bad code is a shape . . . 123
6.2 Why a picture works better . . . 123
6.3 What atlas draws . . . 124
6.3.1 The same figure from two very different programs . . . 125
6.3.2 A red dot is a question . . . 125
6.4 The mathematics behind the picture . . . 126
6.4.1 A category: functions are arrows . . . 126
6.4.2 Effects belong to the arrow . . . 126

6.4.3 Signatures say the arrow 127
6.4.4 A quotient: zooming out is gluing 127
6.4.5 Reach: what a step does, and what a step inside it does 128
6.4.6 One direction 128
6.4.7 Eleven questions atlas asks 129
6.5 Working with your agent on the same picture 130
6.6 A new level of abstraction 130
6.7 What the picture of your project should look like 131
6.8 Atlas in your final assignment 131

1 Introduction

This reader accompanies the course Data Analysis & Visualisation. The slides of each lesson are built to be shown, not read: they carry a plot, a sentence and a question, and the reasoning happens in the room. This reader is where that reasoning is written down, so you can go back to it when you are working on your own data and the lesson is a few weeks behind you.

The course is not a statistics course, and nobody has to become a statistician. What I want you to learn is to *reason* about data: to ask how the numbers in front of you came about, which model of that process makes sense, and what you are allowed to conclude from it. Most mistakes I see in data analysis are not mistakes in the code or in the formulas. They are mistakes in thinking: a model that was chosen without noticing it was a choice, a pattern that was found by looking for it, a title that claims more than the data can carry. A good visualisation is the end of that reasoning, not a replacement for it.

The course runs from visualisation principles and comparing categories, through time, distributions and modelling, to high-dimensional data and dashboards. Each lesson gets its own section in this reader, and a topic that runs through the whole course, such as reading your own code with atlas, gets a section of its own. Each lesson's section follows the deck of that lesson: a subsection per question on the agenda, and a subsubsection per slide with real data, under the title of that slide. You can read the two side by side. Each question ends with the same recap as the deck, "again, concretely". The sections are written in the order I need them, so not every lesson is in here yet.

2 Lesson 4: Distributions

You probably already know the bell curve, the standard deviation and the boxplot. This lesson is about everything else. The lesson is organised around five questions, and I would like you to be able to answer all five at the end of it:

1. **What** is a distribution? Chance has rules. A distribution describes them with parameters.
2. **Which** distribution fits my data? Think about the underlying drivers before you fit.
3. What is an **outlier**? This depends on the distribution you choose. Choose the wrong one and ordinary values look like outliers.
4. Did something **change**, or is it chance? Random data always fluctuates. Compare before and after an event, and ask how big a difference chance alone makes.
5. Is my **model** done? What your model can't explain might tell you whether you missed something.

The first two are about describing data. The last three are about what you can do once you have a description: decide what is unusual, decide whether something really changed, and decide whether your model has captured what is going on.

2.1 What is a distribution?

You cannot predict a single roll of a die. You can, however, say a lot about a thousand rolls: every face will come up roughly a sixth of the time. Chance is not chaos; chance has rules, and a distribution is a description of those rules: every possible outcome, and how often it happens. Often a few numbers are enough. The bell curve needs a mean and a standard deviation; a count at a steady rate needs only its average. Those numbers are the *parameters* of the distribution.

A distribution is good for three things, and they are the rest of this lesson. It lets you *describe* a random process with just a few parameters. It lets you *judge* what is normal and what is unusual. And it lets you *compare*: two distributions, before and after something happened, tell you whether the mechanism behind a random process has changed.

2.1.1 Two kinds of not knowing

It helps to separate two kinds of not knowing. The first is *aleatoric* uncertainty, randomness that is part of the process itself. The word comes from the Latin *alea*, a die[1]. Even if you know everything there is to know about a fair die, you only know how often each face comes up, never which face comes up next. The second is *epistemic* uncertainty: things you could know, but don't (yet). Your travel time is uncertain partly because you haven't checked the traffic. Check it, and that part of the uncertainty shrinks.

Real data always mixes the two. A distribution describes whatever uncertainty is left after you have used what you know, which is why a better model, with more information, gives

[1]The oldest dice were astragali, the ankle bones of sheep and goats, which land on one of four sides. The ones on the slide are Greek glass copies from the 3rd–2nd century BC, in the Metropolitan Museum of Art.

a narrower distribution. Keep this in mind for the last question of this lesson: a model that looks finished might still be missing knowledge.

2.1.2 Three rules make something a distribution

Definition 2.1. A *probability distribution* assigns a probability to the outcomes of a random process, such that

1. no outcome has a negative probability;
2. the probabilities over all possible outcomes sum to 1;
3. the probability of "this or that", for outcomes that cannot happen together, is the sum of their probabilities: $P(1 \text{ or } 2) = \frac{1}{6} + \frac{1}{6}$.

That is all. A die is *discrete*: it has a finite number of outcomes, and every bar is a probability. For a measurement, such as the flipper length of a penguin, the sum in the third rule becomes an area: the probability that a flipper is between 190 and 200 mm is the area under the curve between those values, 0.43 for Adelie penguins.

2.1.3 Continuous distributions: there is 0 chance of exactly 21.0 °C

A measurement such as a temperature is *continuous*, and then something strange happens. Suppose the temperature in a room over the day follows a bell curve with mean 21 °C and standard deviation 0.3. What is the probability that it is exactly 21.0 °C? The honest answer is zero. The temperature can be 21.0, 21.03, 21.0317, and so on: there are infinitely many values, and a single value is a line under the curve, which has no area.

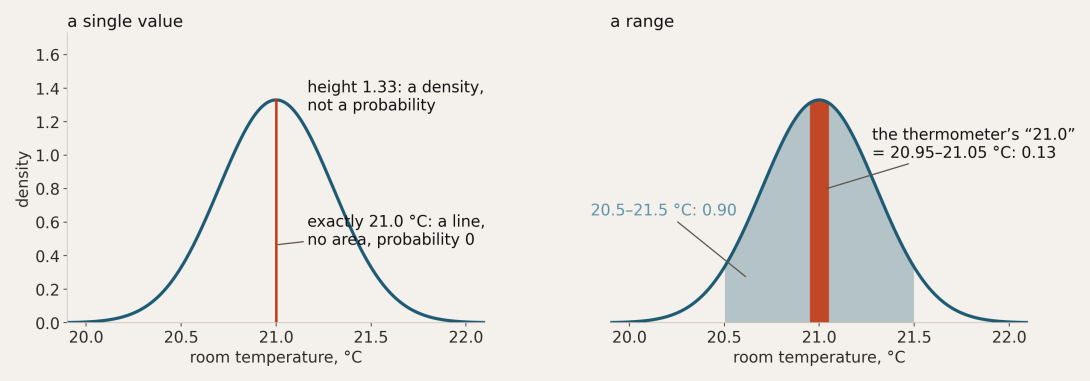


Figure 1: Room temperature, mean 21 °C, sd 0.3. Only an area under the curve is a probability.

What you can ask about is a range. The probability that the temperature lies between 20.5 and 21.5 is the area under the curve between those two values, and here that area is 0.90. Even the thermometer that shows "21.0" is giving you a range: it rounds, so "21.0" means somewhere between 20.95 and 21.05, which has probability 0.13.

The height of the curve is therefore not a probability. It is a *density*, and only an area under a density is a probability. The quickest way to convince yourself: at its peak, the curve in this example has height 1.33, and no probability can be larger than one. This is why the y-axis of a continuous distribution should say *density*. For a count, like a die or messages per day, single values do have a probability: those are bars.

2.1.4 Same rolls, two plots: only one is a distribution

Notice what is *not* in the definition: whether you draw bars or a line. A plot is a distribution because it follows the three rules, not because of how it looks. The most common plot I get handed as "the distribution" is the result of `value_counts` followed by a bar chart. Figure 2 shows why that is not the same thing. Ten rolls of a fair die happened to produce no 5 and no 6. The left plot silently drops those faces from the axis, and its counts only mean something if you know there were ten rolls. It describes ten rolls, not the die. The right plot puts every outcome the process *can* produce on the x-axis, including the ones that didn't happen, and its y-axis holds shares that sum to one. Only that plot can be compared with a model (here 1/6 per face) or with a second sample of a different size. Sometimes you really do want counts; then say so on the axis and don't call it a distribution.

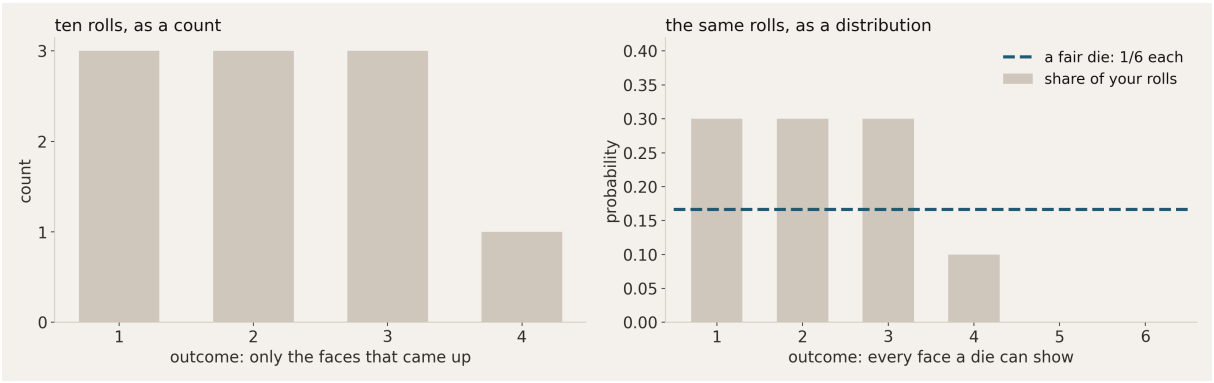


Figure 2: The same ten rolls of a fair die, twice. Only the right plot is a distribution.

2.1.5 Start from what you know about the process

The skill I want you to practise is choosing a distribution by thinking about how your numbers came about, before you look at a histogram. Stand under a tree and count how many leaves fall in each minute. You cannot predict which leaf falls when, but you do know a few things about the process, and each of them narrows down the choice:

- **What kind of number is it?** A count: $0, 1, 2, 3, \ldots$ Never -1, never 2.5. So we need a discrete family.
- **Are the events independent?** Yes: one leaf doesn't pull another one down.
- **Is the rate steady?** On a calm day, roughly: about the same number of leaves every minute, give or take chance.

Those three facts are exactly the assumptions of the *Poisson* distribution, and then a single number, the average per minute, describes the whole distribution. The interesting part is what happens when an assumption breaks. When the wind picks up, the rate is no longer steady: gusts make it wander, and the counts spread out far more than a Poisson allows. Then you need another family. We will meet exactly this in chat data under question two.

2.1.6 Six shapes, six ways numbers come about

You don't need to memorise formulas for the families of distributions. What you need is the one-liner that tells you which process produces which shape. Table 1 has the six we use in this course.

Family	The process that produces it	Example
Normal	Many small, independent parts *added* up	Body height
Lognormal	Many factors *multiplied* together	Duration of a project
Exponential	The wait for the next event, at a steady rate	Time between messages
Poisson	The number of events in a window, at a steady rate	Leaves per minute
Weibull	Time until something stops, when the risk changes with age	Lifetime of a part
Pareto	The rich get richer	Followers on social media

Table 1: Six families and the processes behind them.

The assumption people forget is the steady rate. Poisson and exponential both need one. Put the arrivals at a station during rush hour and during a quiet afternoon in one column, and neither fits. Poisson is the only count in the table, so its bars are probabilities; the others are curves where an area is a probability.

2.1.7 Adding makes a bell; multiplying makes a long tail

The difference between the first two families is one of the most useful things you can know about data. Roll thirty dice and add them up. Each die on its own is flat: every face is equally likely. But the sum is a bell. Two dice already make a triangle, five look like a bell, thirty are a bell. Human height works the same way: it is the sum of the contributions of more than a hundred genes that control the length of your femur, your spine, your neck. Many small things added together: that is where the normal distribution comes from[2].

Now multiply instead of add. Every factor makes the result more lopsided, with a long tail to the right. Projects work like this: a delay doesn't add a fixed week, it stretches everything that depends on it. If task a takes 1.5 times as long as planned and it has to be done ten times, the delay compounds. One trick is worth remembering here: the logarithm of a product is a sum of logarithms,

$$\log(x_1 \cdot x_2 \cdots x_n) = \log x_1 + \log x_2 + \cdots + \log x_n.$$

So if something is made by multiplying, its *logarithm* is made by adding, and the logarithm looks like a bell. That is exactly what a lognormal distribution is.

Project planning: delays multiply. Figure 3 shows this in real data: 10,100 tasks from ten years of commercial software development, each with the estimated and the actual number of hours[3]. The median task is exactly on time, so the estimates are not simply

[2]This is the central limit theorem, without the formula.

[3]The SiP dataset: Jones, D. & Cullum, S. (2019), `https://github.com/Derek-Jones/SiP_dataset`, CC BY-NC-ND 4.0. Erik Bernhardsson analysed the same data in "Why software projects take longer than you think" (erikbern.com, 2019).

optimistic. But the tail runs one way. The average task takes 1.37 times its estimate, one in ten takes more than twice as long, one in a hundred more than seven times, and the worst one 281 times. A bell with this mean and standard deviation would put 39% of the tasks below zero hours, which is impossible. On a log axis, though, the ratios form a rough bell around 1: finishing in half the time (13%) is about as common as taking twice as long (10%). Symmetric in factors, not in hours. That is the signature of multiplying. (The spike in the middle is not the work but the people: 34% of the tasks were booked at exactly their estimate.)

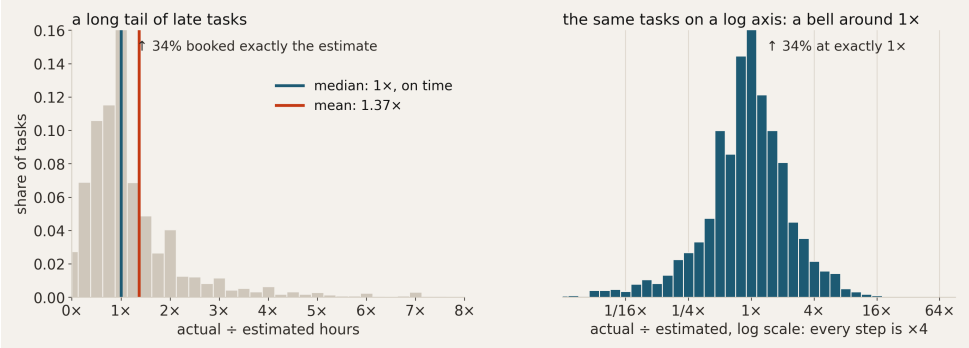


Figure 3: Actual divided by estimated hours for 10,100 software tasks. On a log axis the ratios form a rough bell around 1×.

Again, concretely. Dice add up to a bell. Project delays multiply into a long tail. Leaves on a calm day are a Poisson count. In each case the family followed from what we knew about how the numbers came about, not from a histogram. A distribution describes a random process: every possible outcome, with shares that sum to one. What you know about the process narrows down the family: a count or a measurement, added or multiplied, a steady rate or not. Then the data gets to check that choice. That is question two.

2.2 Which distribution fits my data?

A computer will fit any family to any data, and it will always return a winner. Choosing the family is your job. Fitting a distribution consists of three steps, and the order matters.

1. **Think.** Which families make sense for how these numbers came about? Is it a count or a measurement? Can it be negative? Is it a sum or a product, does it have a steady rate, does it have a snowball effect? Often several families fit about equally well; then the process decides.

2. **Fit.** For each family that makes sense, let the computer find the parameters under which your data is most probable: the mean and standard deviation of a bell, the rate of a Poisson. This is called *maximum likelihood*, and it is the part computers are good at.

3. **Check.** Do model and data agree, up to the differences chance makes anyway? Look where they differ, in the tails too. A qq-plot shows you.

The worst way to start is to hand a column to a fitter and report whatever wins. A winner that makes no sense for the process is noise, not an answer. The distribution fitter in goad,

the course toolkit, also gives you test scores for every family. With thousands of rows every family gets rejected by a formal test, so use the scores to rank, and use your eyes and your knowledge of the process for the rest. A count is fitted with `discrete=True`, because a count has a probability per whole number.

2.2.1 Messages per day: the rate isn't steady

Messages per day on a chat channel are a count, so the first candidate is a Poisson. Before fitting, check its assumption: is the rate steady? On a chat channel that is unlikely, and figure 4 shows how unlikely. Over 250 ordinary Thursdays the Italian Ubuntu channel had between 10 and almost 1,800 messages a day, 564 on average. A Poisson with that mean allows about 24 messages either way and squeezes 164 of the 250 days into a single bar. Clearly wrong.

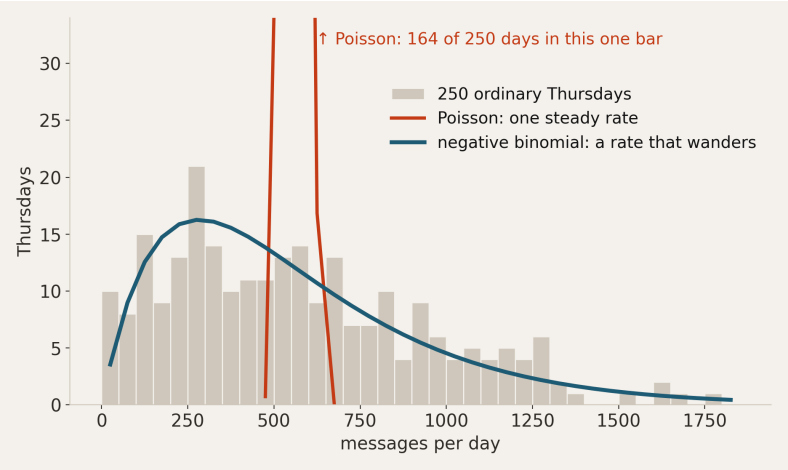


Figure 4: Messages per ordinary Thursday in #ubuntu-it. Poisson (red) against negative binomial (blue).

There is a quick way to see this without a plot. For a Poisson distribution, the variance equals the mean:

$$\mathrm{Var}(X) = \mathbb{E}[X].$$

If the variance of your counts is much larger than their mean, the Poisson is the wrong model. The *negative binomial* is a Poisson whose rate is itself drawn anew every day, and it fits these data well: mean 564, standard deviation 399. Who is around, what broke, the time of year: they all move the rate. That is the wind from the leaves example.

Notice that the failed Poisson is informative. It tells you the rate moves. The honest summary of an ordinary Thursday is not "about 560 messages" but "a rate of about 560 that routinely halves or doubles".

2.2.2 A strange best fit can mean several groups

The penguin dataset[4] contains the flipper length of 333 birds. Put that column into a fitter and the best single curve is a *beta* distribution. A beta describes a percentage you are

[4]Palmer Penguins: Horst, A.M., Hill, A.P. & Gorman, K.B. (2020), `https://allisonhorst.github.io/palmerpenguins/`.

unsure about: flip a coin ten times, get seven heads, and the beta tells you which values of "the chance of heads" are plausible. It lives between zero and one. The fitting library (scipy) lets every family shift and stretch, so it can lay a beta over 170 to 235 millimetres. The fit is real; the meaning is nil. Nothing about a flipper length is a percentage.

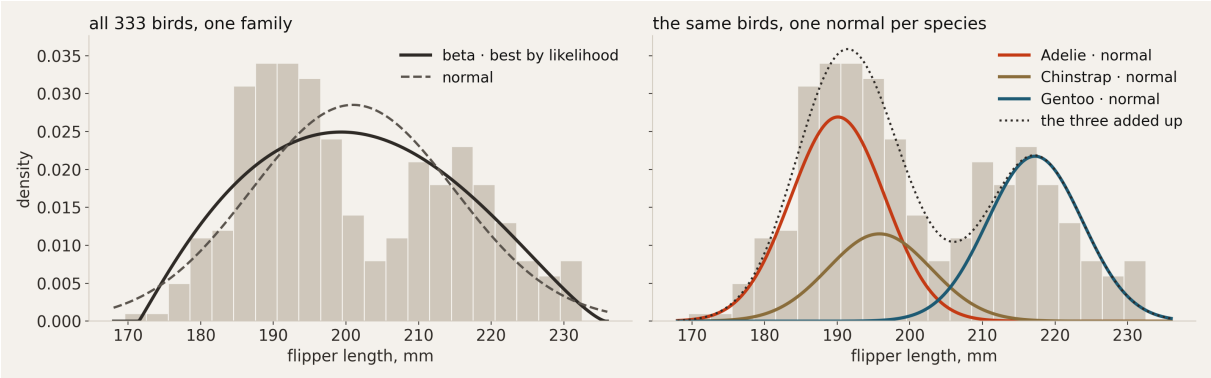


Figure 5: Left: one curve for all 333 penguins. Right: one normal distribution per species.

A winner like that is a warning. The histogram has two humps, and two humps usually mean two or more groups. Here we know the groups: three species. Fit one normal distribution per species and each one fits fine (figure 5); Adelie and Chinstrap overlap, Gentoo sits twenty millimetres further. The column was never one population. A species has a flipper length; "a penguin" doesn't. What told us to split was knowing the data, not the fit table. With some experience you can also see it in the plot: the gap between the humps is about two standard deviations, which is what two bells lying next to each other look like.

2.2.3 A qq-plot shows where model and data disagree

You know the boxplot. Its box marks three quantiles: the lengths below which a quarter, a half and three quarters of the birds fall. A *qq-plot* does that for every single observation. Sort the data; for each rank, ask the fitted model which value it expects at that rank; plot the real value against the expected value. If the model is right, the dots lie on the diagonal. Where the dots leave the line, the model is wrong, and the way they leave it usually tells you why.

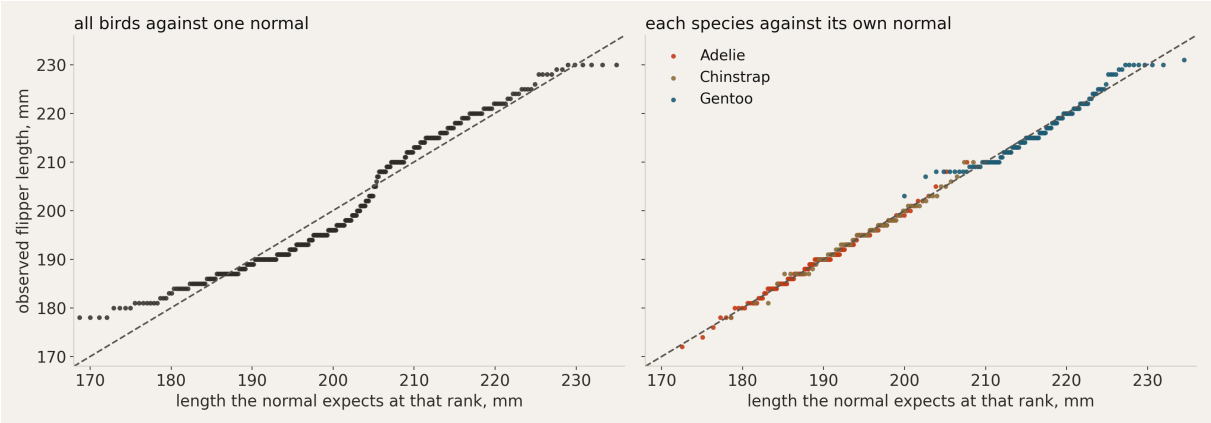


Figure 6: qq-plots against the normal. Left: all birds against one bell. Right: each species against its own.

For all penguins against one bell, the dots snake around the line in an S-shape: too few birds in the middle, too many at both ends, because the pooled data has a dip in the middle where a bell has its peak. Each species against its own bell follows the line, apart from small steps that come from measuring to the whole millimetre. The qq-plot is most useful in the tails, where a histogram has too few points to show you anything. We use it on taxi fares under question three.

Again, concretely. The fitter crowned a stretched beta. Split by species, and three plain bells appear. The fitter always returns a winner; whether that winner makes sense is your call. Before trying a stranger family, ask whether your column holds several groups. Then check with a qq-plot: where the dots leave the line, the model is wrong. Messages per day failed the Poisson because the rate moves; the penguins failed one bell because there were three populations. Both failures told us something about the data.

2.3 What is an outlier?

Here is the central idea of this question: *an outlier is not a property of the data, it is a property of your model.* An outlier is a value your model didn't expect. Pick the wrong model, and ordinary values look strange.

2.3.1 What does an outlier mean?

When you find an outlier, there are three possible explanations, and I list them in the order in which you should suspect them.

1. **A wrong model.** This is by far the most common reason, certainly among students. The value is ordinary; your distribution doesn't fit the data, or it mixes groups. A value that looks like an outlier may simply belong in a different distribution: another species of penguin, another phase of a process. Fix the model, not the data.
2. **A rare, real value.** Unusual but possible: sometimes there really is an extremely long taxi ride or an exceptionally busy day. Keep it; it is often the most interesting row in your dataset.

3. **An error.** A value the process cannot produce: a negative duration, a date in 1970, an amount of 70 million among amounts of 4,000. Remove it, and say that you did. But consider this last, not first.

Statistics can point at the rows to look at. Which of the three it is takes knowledge of the process. And the two rules of thumb everyone knows, the boxplot whiskers and "more than three standard deviations from the mean", both assume a bell. On skewed data, they flag ordinary values.

2.3.2 The bell has no room for expensive rides

Taxi fares, 6,433 New York rides. The mean is $13, the median $9.50: the tail pulls the mean up. Read the qq-plot in figure 7 the same way as the penguins: each dot is one ride, its real fare against the fare the model expects at that rank. Against a fitted bell the dots bend away from the line. At the bottom the bell expects negative fares; at the top it expects about $55 where the real top is $150. A right-skewed family, the lognormal, follows much further.

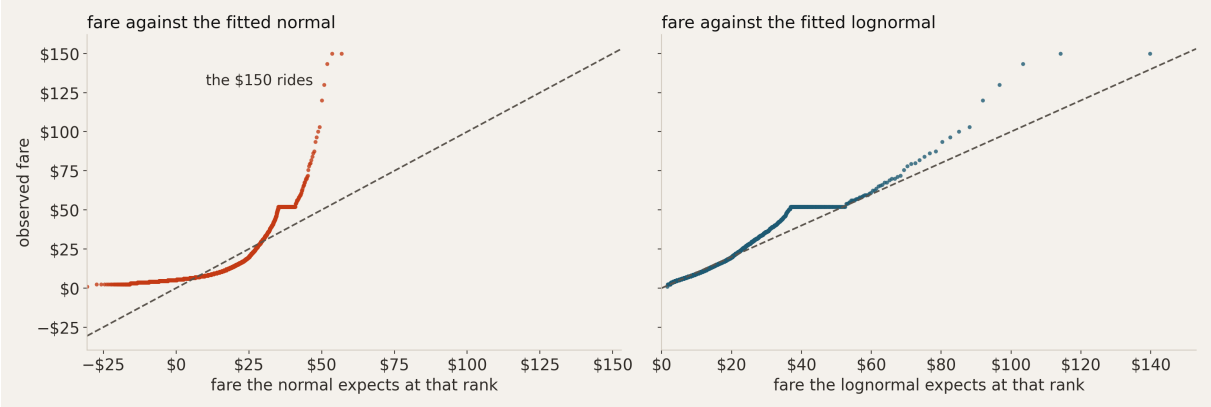


Figure 7: Taxi fares against a bell (left) and a lognormal (right).

Why are the fares so skewed? Don't read a deep mechanism into it. Fares are a base charge plus a meter, so mostly a sum; the skew comes from the trips. Most rides are short, and for long distances people choose a train or a car, because a taxi becomes too expensive. The tail is cut off by people's choices, not by chance. We just need a family with room on the right. And the flat run at $52 is the fixed airport fare: a second process in the same column.

2.3.3 Outliers, or the wrong model?

Most people use an outlier definition without knowing they chose one. The dots beyond the whiskers of a boxplot come from Tukey's rule: the whiskers reach 1.5 times the interquartile range (IQR) past the box, and everything beyond is drawn as an outlier. Tukey designed that rule for roughly bell-shaped data; on a normal distribution it flags 0.7% of the observations.

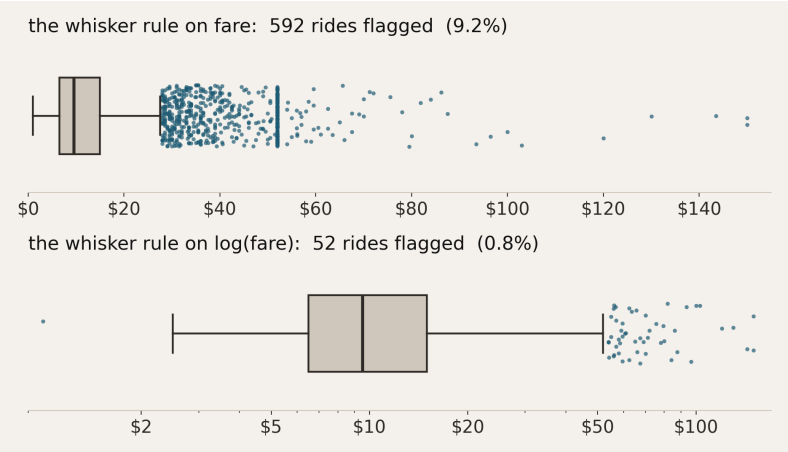


Figure 8: Tukey's whisker rule on the taxi fares, and on the logarithm of the same fares.

On the taxi fares, Tukey's rule flags 592 rides as outliers: 9.2%, including every airport ride (figure 8). Not 592 strange rides: nothing is wrong with those rides, the model behind the rule is. Apply the same rule to the *logarithm* of the fare, whose shape is close to a bell, and it flags 52 rides, 0.8%, which is about what the rule expects. The rule isn't wrong. It assumes a bell.

2.3.4 The whisker rule makes four assumptions

Tukey's rule quietly makes four assumptions:

1. **Shape: roughly symmetric.** A bell, not a long tail. On taxi fares the rule flags 592 ordinary rides.
2. **Population: one group.** Mix groups, and an ordinary member of another group looks like an outlier.
3. **Numbers: continuous.** Measurements. Counts, categories and rounded values pile up on the same numbers, and the quartiles and whiskers land on awkward ties.
4. **Order: independent.** Every value stands on its own. In a time series, where every value builds on the previous one, the rule is out of scope.

Break one of these, and a dot past the whiskers no longer means "unusual". The same four assumptions hide behind "more than three standard deviations". Next: an analysis that breaks three of them.

2.3.5 What not to do: judge data without the process

Here is a real student analysis from an earlier year, shown as an example of what not to do. The data is the axle load of a garbage truck over a morning. The student saw gaps and an uneven number of measurements per day, wrote that this suggested "inconsistent data collection", and concluded that about 30% of the data was unreliable.



Figure 9: The axle load of a garbage truck through one morning.

Now think about the process instead. The load grows while the truck collects along its route. It drops back to roughly the weight of the empty truck when the truck is emptied, and then the next round starts. The flat stretches and the gaps fit driving to the dump, waiting and breaks. A sudden drop of several thousand kilos is not someone throwing out a very heavy bag; it is the truck being unloaded. And a day with fewer measurements might simply be a shorter round.

Check the four assumptions, and three of them are broken. The measurements are not independent: if the truck weighs 20,000 kg and two bags are added, the next measurement is 20,050 kg, strongly correlated with the previous one. There is not one population: a full truck and an empty truck are different situations. And so there is no single distribution over the day, and no whisker rule or threshold can say anything about reliability. None of this data was shown to be unreliable; the analysis never asked how it was made.

What would work? Two things, both suggested by students in class: split the data into the two processes (collecting and unloading), and take the *difference* between consecutive measurements. The differences (10 kg, 50 kg, 100 kg) are close to independent, and they do have a distribution. Against that distribution, a jump of 2,000 kg during collection really is an outlier, and a very interesting one.

2.3.6 One $150 ride, three verdicts

The most expensive ride in the dataset costs $150. Is it an outlier? That depends entirely on the model you hold it against. Figure 10 shows, on a log scale, the share of rides at least this expensive: each step down is 1,000 times rarer. Black is the data.

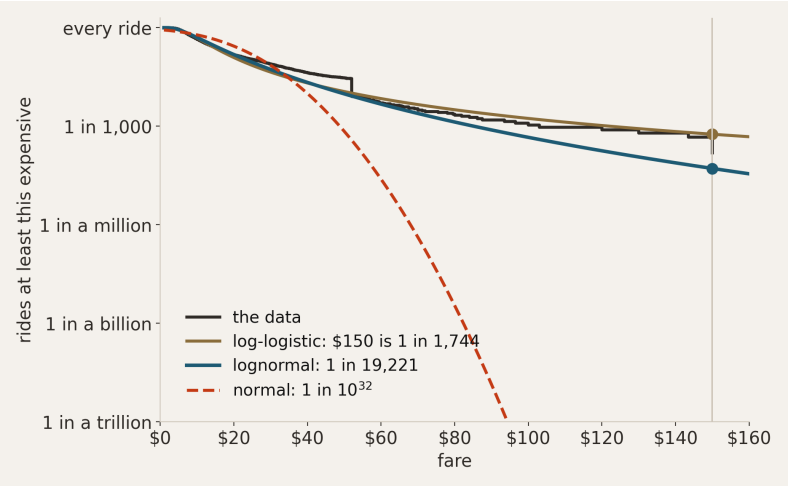


Figure 10: The share of rides at least this expensive, under three families.

- **1 in 10^{32}.** Under a fitted bell the ride is 11.9 standard deviations above the mean. To give you a feeling for that number: there are roughly 2×10^{16} ants on earth[5]. Pick a random ant; now I pick a random ant too, and it is the same one. Now do that twice in a row. That is the kind of rare we are talking about. Under this model, the ride cannot have happened.
- **1 in 19,000.** Under the lognormal, the best of goad's families.
- **1 in 1,700.** Under the log-logistic, a skewed family outside goad's list, which follows the real tail more closely. With 6,433 rides, a few rides like this are within expectation.

The family decides how rare a value is. A check in numbers: 7 rides cost $100 or more; the lognormal expects 3, the log-logistic 11. And a warning: the Burr family has a better likelihood than both, because it fits the bulk of the data, and it expects 32 rides of $150 or more where there are 2. The best fit overall is not the best fit in the tail, and the tail is where outlier questions live. Where data is thin, check the tail itself. Note that neither skewed family says "delete the ride"; only the bell says it is impossible.

2.3.7 Ordinary among all penguins, rare among her own

A wrong model doesn't only make you find outliers that aren't there; it also makes you miss the ones that are. One Adelie female in the penguin data has a flipper of 202 mm. Among all penguins she is completely ordinary: she stands in the middle of the crowd of another species, and no rule would flag her. Among the 73 Adelie females she is two and a half standard deviations long, about one in 95.

[5]Schultheiss, P. et al. (2022). The abundance, biomass, and distribution of ants on Earth. *PNAS*, 119(40).

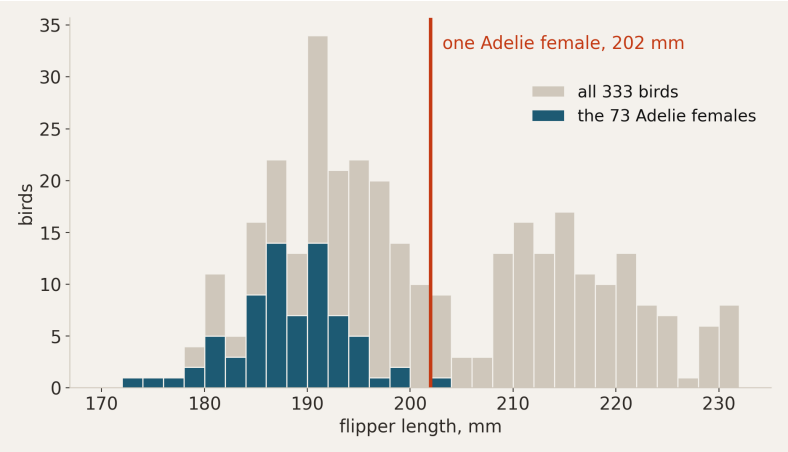


Figure 11: Grey: all 333 birds. Blue: the 73 Adelie females. Red: one Adelie female with a 202 mm flipper.

It runs the other way too: the longest flipper in the data, 231 mm, looks rare among all penguins (one in 146) and is an ordinary large Gentoo male (one in 22). "Unusual" always means unusual *for a population*, so pick a population that exists. What to do with her is something the numbers cannot settle. It could be a mistyped measurement, a bird whose sex or species was recorded wrongly, or a genuinely large female. Someone who handled the birds knows.

Why you want to find outliers. After all these warnings you might think outliers are a nuisance. They are not. Outliers are often the first sign of a problem: in a factory, an unusual measurement is often an early warning that something is about to break; in a sales process, it can announce a dissatisfied customer or rising costs. And outliers are a way into discovering *groups* you didn't know about. In a chat export, for example, you will often find a sharp peak in message length around eight characters. That is not a habit of the writers: it is the export writing a placeholder such as "image omitted" every time someone sent a picture. It is a different process in the same column, and it does not belong in the distribution of what people write. Often, when you go looking for an outlier, what you are really looking for is a different process. That is why it pays to have more than the normal distribution in your toolbox.

Again, concretely. A $150 taxi ride is impossible under a bell and merely rare under a skewed family. Same ride, different model. Before you call something an outlier, check that your distribution fits, in the tails too, and that it describes one population. If it fits and the value is still extreme: is it an error, or a rare real event? That takes knowledge of the process, not statistics. In your chat that might be an export glitch, a birthday, or the week someone joined. If you remove anything, report how many values, on what criterion, and whether your conclusion survives with them left in.

2.4 Did something change, or is it chance?

A distribution describes a process. When something changes the process, the distribution moves with it. Someone *falls in love* and sends more messages, at different hours, to one person. A *holiday* means fewer messages and a different rhythm of the day. A *new release*

fills a chat channel with questions. So you compare the distribution before and after the event.

The catch is that two samples always differ a little, even when they come from exactly the same process, and our eyes are very good at seeing change in noise. The question is therefore never "is there a difference?", but "is the difference bigger than the differences chance alone produces?"

2.4.1 Did this person's texting change?

The example comes from the first chapter of *Bayesian Methods for Hackers*[6]: the number of text messages one person sent per day, over 74 days. From around day 45 the average goes up from 17.8 to 22.8 messages a day. Did this person's texting habits change? Perhaps something happened: a new relationship, a new job. But look at the individual days: 2 messages one day, 72 another. With noise this large, a difference of 5 a day could easily be luck. How do you tell?

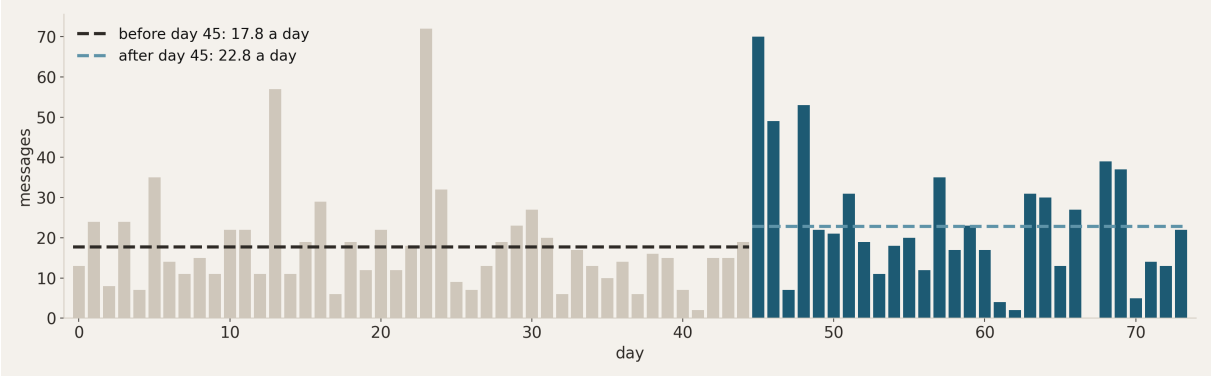


Figure 12: Messages per day for 74 days.

2.4.2 Shuffle the labels to see what chance does

Here is a way to find out that needs no formula and no distribution family at all. If nothing changed at day 45, then which days we call "before" and which "after" is arbitrary. So we can shuffle those labels.

Definition 2.2. A *shuffle test* (or permutation test) compares the difference between two groups with the differences you get when the group labels are randomly reassigned. Shuffle the labels, compute the difference again, and repeat many times (say 10,000). The share of shuffles that produce a difference at least as large as the real one is the *p-value*: how often chance alone does this well.

For the texting data we keep 45 days labelled "before" and 29 labelled "after", shuffle, and compute the difference in means, ten thousand times. That cloud is what chance alone makes (figure 13). The real difference sits in its upper tail, but 14% of the shuffles produce a difference at least as large, in either direction ($p = 0.14$). A change is possible; this alone doesn't show it. Roughly: the more noise in your data, the larger the chance that what you see is chance.

[6]Davidson-Pilon, C. (2015). *Bayesian Methods for Hackers: Probabilistic Programming and Bayesian Inference*. Addison-Wesley. The notebooks are at `https://github.com/CamDavidsonPilon/Probabilistic-Programming-and-Bayesian-Methods-for-Hackers`.

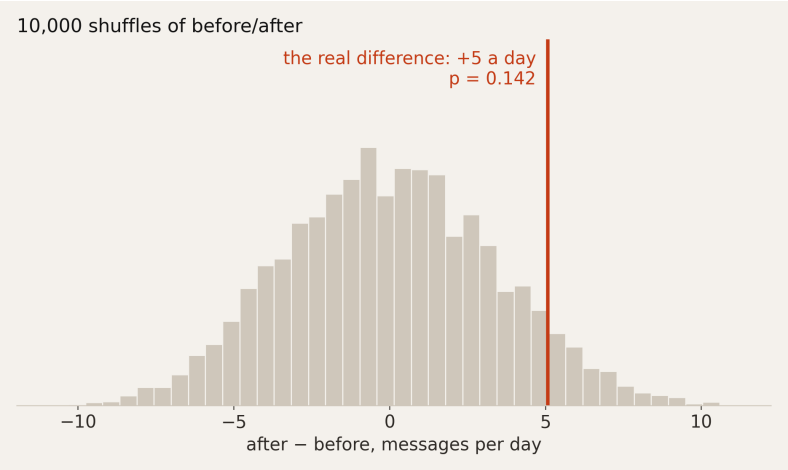


Figure 13: Grey: the differences chance produces in 10,000 shuffles. Red: the real difference, +5 a day.

2.4.3 Bayesian Methods for Hackers: a confident switch

The book answers the same question in a Bayesian way, and comes out much more confident. Its model says: the messages on each day are Poisson distributed, with rate λ_1 before an unknown day τ and rate λ_2 from τ on. The switch day τ gets a uniform prior: before we look at the data, every one of the 74 days is equally likely to be the switch. A sampling method, Markov chain Monte Carlo (MCMC), then finds which combinations of λ_1, λ_2 and τ explain the data well. The verdict: from 17.8 to 22.7 messages a day, from day 44 or 45 on, and the probability that the rate went up is

$$P(\lambda_2 > \lambda_1) = 0.998.$$

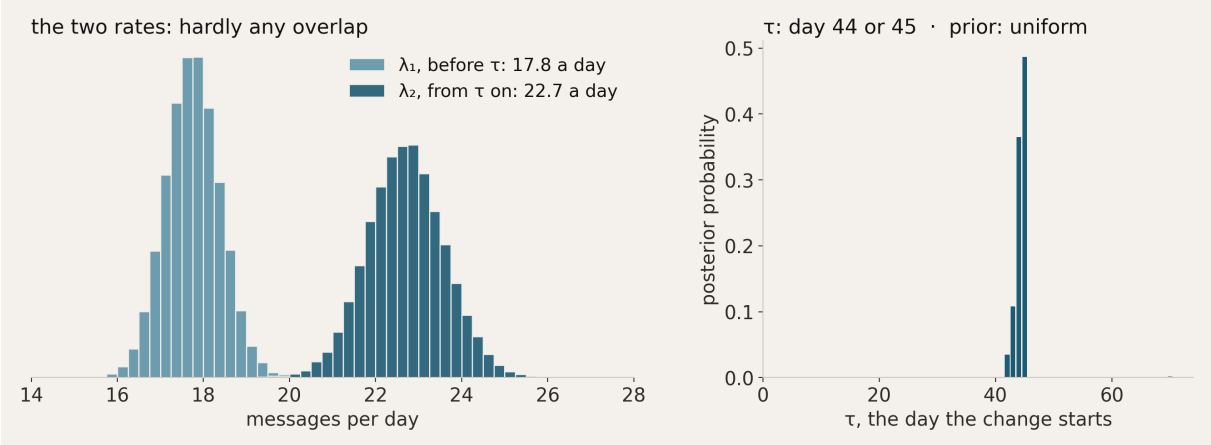


Figure 14: The book's model. Left: the two rates. Right: the switch day τ.

The shuffle said a difference this size turns up in one shuffle out of seven. Same data, very different confidence.

2.4.4 Are those assumptions right?

Where does the difference come from? When we discussed this in class, the right objection came from the room: look at the spread. The book's method is fine; the question is

whether its assumptions hold. A Poisson assumes $\mathrm{Var}(X) = \mathbb{E}[X]$, so a day with an average of 18 messages varies by about 4 either way. These days run from 0 to 72: the variance is around 155 to 250, against a mean of 18 to 23. The assumption is violated, and because the Poisson underestimates the day-to-day noise, it is far too sure about each rate. That is why the two rates hardly overlap in the book.

Keep everything else in the book's model the same and only let the days vary more, with negative binomial counts instead of Poisson counts. Then no day stands out as the switch any more, and the probability that the second period is busier drops from 0.998 to 0.59. If you knew in advance that something happened on day 45, it is 0.94: a likely increase, not a certain one.

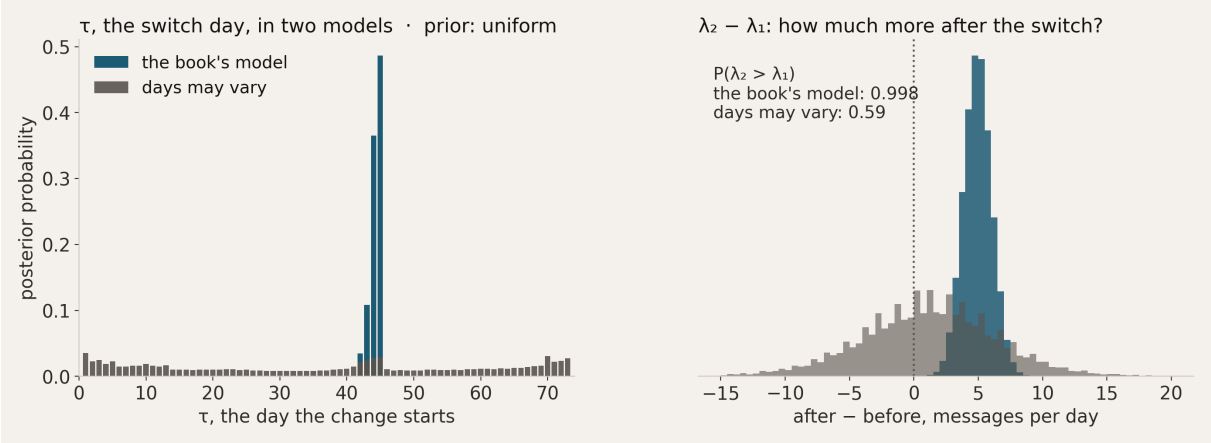


Figure 15: The same model with days that may vary.

The technique is elegant; the distribution was wrong. The careful conclusion of the shuffle test turns out to be the more reliable one. There is a broader lesson here: you can model the same phenomenon in many ways, and depending on how you look, you get a different answer. A good reason to reject a model is a violated assumption, not a preference for a different answer.

2.4.5 Searching for the day costs you the evidence

There is one more thing hiding in the question "did the texting change?", and it is perhaps the most important point of this lesson. There are two different questions.

1. *The day is known in advance.* Something happened on day 45, and I want to know whether the texting went up after it. That is one comparison. The shuffle test gives 7% (now counting only shuffles where "after" is busier), and the model with varying days gives 0.94.

2. *The day is searched in the data.* I don't know whether anything happened, so I try every split and keep the biggest difference. Now chance gets a try at every possible split; here 55 of them, with at least ten days on each side. The best of those 55 tries beats +5 in 45% of the shuffles. The model gives 0.59.

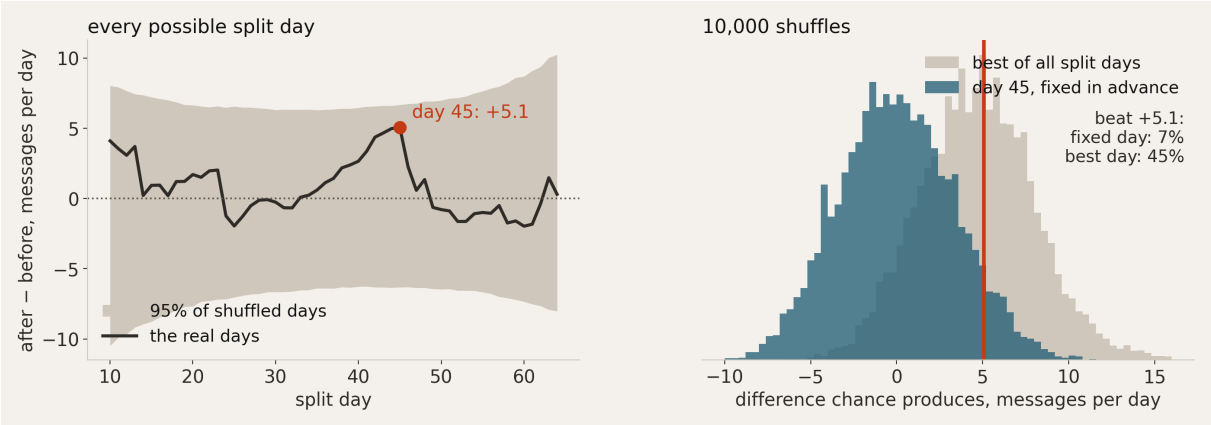


Figure 16: Left: the difference for every possible split day; the grey band is what shuffled days do. Right: the biggest difference chance finds when it may pick the day (grey), against the difference at a day fixed in advance (blue).

A difference of +5 is unusual for one fixed day and ordinary for the best of 55. Day 45 looks special because it was picked for looking special. An event you know beforehand turns 55 questions into one. The Bayesian model does the same bookkeeping automatically: with τ unknown, it has to spread its belief over all days, and with realistic day-to-day noise no day stands out. With Poisson counts the data look so decisive that this cost disappears, which is why the book could find the day and be sure of it at the same time.

I like to explain this with dice. Put 100,000 people in a hall and let each of them roll a die six times. If you afterwards find someone who rolled six sixes, that is not proof of special powers, only of a big hall: with that many people you should expect it. But if you point at someone *beforehand* and say "this person has special powers", and they roll six sixes, that is spectacular. And if they do it a second time, you call the army. Predicting the future afterwards is easy; the trick is doing it beforehand.

This has a practical consequence for your own analysis. If you know beforehand that a trip, a hospital stay or the weeks of morning sickness in a pregnancy define your labels, you can simply use those labels. A short period or a small number of observations is then not a reason to give up, because a shuffle test on labels that were fixed in advance can still come out very significant. Name the event before you look. Searching first and testing afterwards finds a switch in almost any series.

2.4.6 Same lift, two verdicts

The same test, on two Ubuntu chat channels: is a release day busier than an ordinary Thursday? The labels are shuffled 2,000 times, as in notebook 04.4. In both channels a release day is busier, by a factor of about one and a half. In the small Italian channel, #ubuntu-it, release days are 41% busier, but the real difference sits at the edge of the shuffle cloud: $p = 0.07$. Ten days of a rate that wanders this much can't settle it. In the big channel, #ubuntu, release days are 47% busier, and far outside anything the shuffles produce: $p < 0.001$.

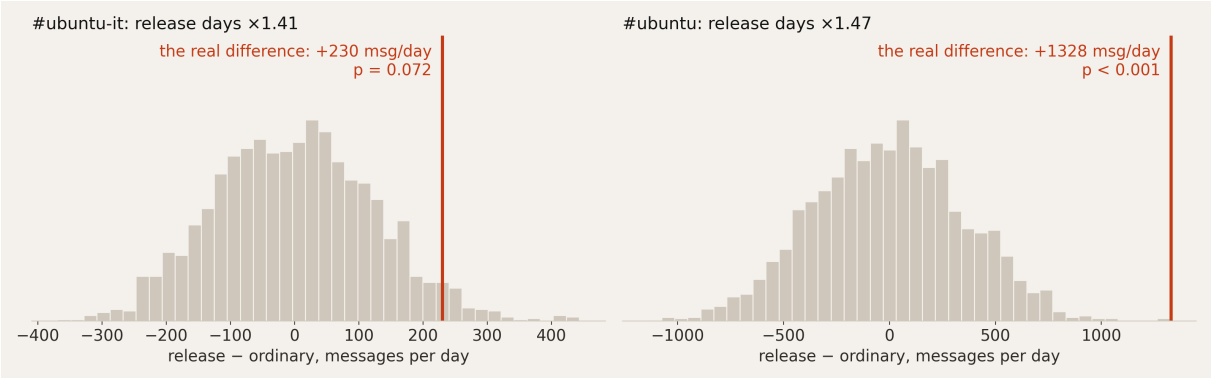


Figure 17: Release days against ordinary Thursdays in two channels, 2,000 shuffles.

Same effect, different evidence. The reason is the spread of the ordinary days: in #ubuntu-it they wander by two thirds of their mean, in #ubuntu by a third, because many people averaging out make days more alike. There is no sharp line between chance and change: the further outside the cloud, the more convincing.

Again, concretely. A 40% busier release day is borderline in a small channel and unmistakable in a big one. The noise decides. Compare before and after, then ask how big a difference chance alone makes; shuffling the labels answers that without assuming a family. The noisier the ordinary days, the more event days you need before you can see a change, and that spread is something you can see before any event happens.

2.5 Is my model done?

The last question. Fit a model, subtract it from the data, and look at what your model can't explain:

$$\text{observed} - \text{model} = \text{residual}.$$

The model doesn't need to be a statistical model; a simple formula or a straight line will do. What matters is what the residual looks like.

- **The residual looks like chance:** errors around zero, from a plausible family, with no pattern over time. Then you may be done. *May*: a cause you never measured also looks like chance, and adding it could still make the spread much smaller. That is the epistemic uncertainty from the start of this lesson.
- **The residual has a pattern:** a step, a drift, a bend. Then your model is missing a mechanism, and when the pattern starts is often the most interesting finding. That doesn't make the model useless; whether it is good enough depends on what you need.

Good enough is a goal, not a statistic. If you bet on something that is fifty-fifty, a mechanism that picks the right side 51% of the time makes you rich, as long as you can bet often enough in a row and the mechanism stays consistent. You don't need 99%. (Living through the 49% is the hard part, so write an algorithm for it.) What you do need is to communicate clearly what your model does and does not capture.

2.5.1 Before vaccination, the residual looks like chance

For this example I deliberately use data that is not chat data, because what transfers is the loop of modelling and checking, not the dataset. The data are the public Dutch COVID figures. A pipeline prepared them: daily deaths from the cumulative count, shifted fourteen days back next to the positive tests that led to them, and a seven-day average against the weekly reporting rhythm. The model is simple. On average, people who die of COVID die about two weeks after their infection, so deaths two weeks later are a fixed fraction of positive tests: a straight line. Fitted on the 85 days from 13 October 2020 to 5 January 2021, before vaccination started, the line is

$$\text{deaths}_{t+14} \approx 0.0065 \times \text{positive tests}_t + 20.$$

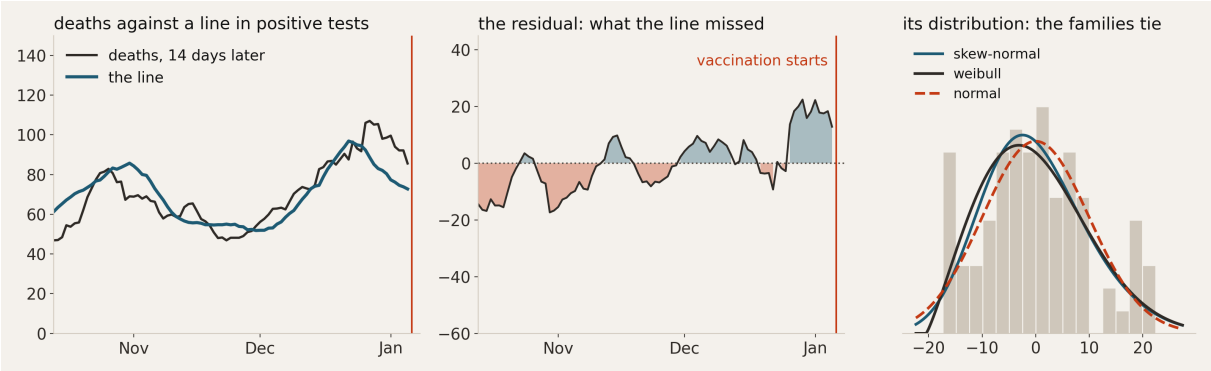


Figure 18: Before vaccination (85 days). Left: the model. Middle: what the line missed. Right: its distribution.

On that window the residual waves around zero with no drift (figure 18); the last week runs high at the winter peak. There is also a bump around Christmas, and in class the temptation was to read it as the start of an upward trend. Be careful which pattern you think you see: if that were a real trend, it would mean the virus became deadlier over time. Much more plausible is that the Christmas holiday and the lockdown rhythm run through the data. When we fit distributions to the residual, six families fit within a few points of each other and none is rejected. Errors the data can't tell apart: that is what chance looks like.

2.5.2 The residual reveals what the model missed

Now fit the same kind of line on all 232 days, from October 2020 to June 2021 (figure 19). Nothing in the model knows that a vaccine exists. From March it predicts more deaths than happen, month after month. The residual changes character: positive through the winter wave, and then from 27 February to the end a long negative run that never comes back. Every family is rejected; the best is the uniform, which is what a fitter says when the errors lie along a trend instead of around zero.

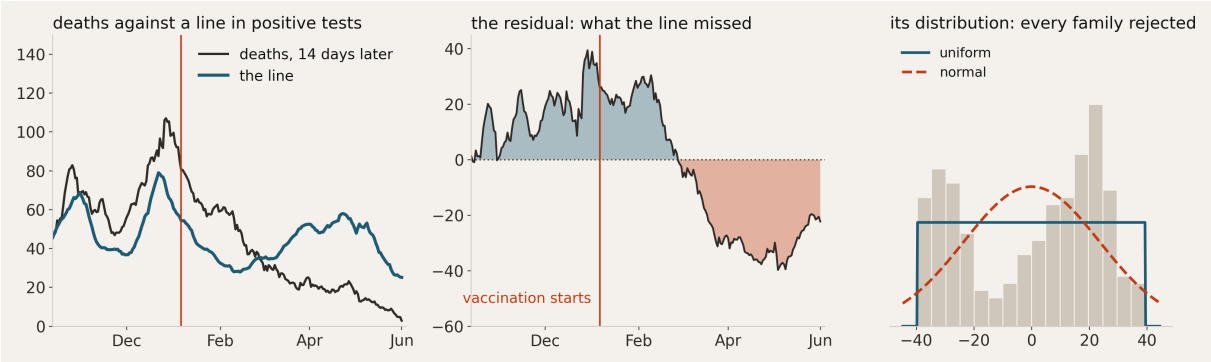


Figure 19: The same kind of model over 232 days, October 2020 to June 2021.

The residual is pointing at a process the model left out, and at roughly when it started: after vaccination, the same number of positive tests comes with fewer deaths. The correlation between tests and deaths over the whole period looks weak, but it is really two different random processes. Notice what the residual does and does not do. It dates the change; it does not prove the cause. Better treatment, or a change in who got tested, could leave a similar run.

Again, concretely. One straight line, two windows. Before vaccination, the line was a fine model: what it missed looked like chance, and for many purposes the line is good enough. Over the full window, the residual has a run with a start date: a mechanism is missing, here vaccination, and that pattern is the most informative thing the model produced. Next lesson, notebook 05.3 hands you this residual and asks you to write the missing mechanism into the model.

2.6 The five questions, answered

Question	Answer
What is a distribution?	A model of what you can predict about randomness: every possible outcome, with shares that sum to one.
Which distribution fits my data?	First reason about a model that fits the underlying process, then fit, then check. Never the other way around.
What is an outlier?	It shows what you modelled as “expected”.
Did something change, or is it chance?	Only if the difference is bigger than chance alone makes. Name the event before you look.
Is my model done?	Study the residual, and wonder which patterns your model did not pick up.

Notebooks 04.1 to 04.5 work through each of the questions, ending on your own chat.

2.7 Feedback on your distribution plots

In the second half of the lesson we discussed the distribution plots you made of your own data. The same points came back again and again, so I collect them here.

One message, in half a second. A good visualisation works pre-attentively: the reader picks up the message within about 500 milliseconds, before they start reading. That only works for simple messages: something is bigger, smaller, shifted, or happened. Two clearly different things, not more.

Reduce the clutter. Remove the grid. Bring the palette back to two meaningful colours, for example grey for "before" and one colour for "during". Six lines of legend where one label next to the line would do is noise. Labels like "axis is logarithmic" or "line is a fitted lognormal" can go if the axis already says so.

One visualisation, not two. People usually show two plots because they cannot choose. Two plots are only defensible if together they tell one story.

The title must match what the data can carry. A neutral title is a waste; you are allowed to bring a message. But the message has to be backed up by the data. "Our first holiday together made our group of friends closer", based on the observation that messages got longer, is too big a leap: long messages can also be long arguments about who still owes money for that dinner. Either choose a less ambitious title, or dig deeper into what those messages are about. Always ask yourself: do I have the data to back this up?

Show a real distribution. Follow the three rules of definition 2.1, and decide whether you are showing probabilities (discrete) or a density (continuous). Put *density* on the axis of a continuous plot. A y-axis that runs from 0 to 75% under a smooth fitted curve tells the reader you haven't made that choice.

Report the shuffle test. If you compare two distributions, the number your reader is waiting for is how likely the difference is under chance: "p-value after 10,000 shuffles". Not the standard deviation of your lognormal fit. Parameters are only interesting when you compare two processes, such as the two rates before and after day 45.

Be careful with log scales in a presentation. A log axis compresses, and makes two distributions look alike that are very different on a normal axis. Your audience cannot undo the logarithm in their head.

Know who you make it for. Transparent histograms behind fitted lines are fine for a trained reader and noise for a business audience. That is a choice, not a matter of right or wrong.

A null result needs tension. Sometimes you find that nothing changed. That is interesting only if there was a strong expectation that something would. If nobody expected an effect, finding none is not a story; dig deeper.

3 Lesson 5: Modelling relationships

The previous lessons each added a way of looking at data: comparing categories, following something over time, smoothing out a trend, describing randomness with a distribution. This lesson puts them together. A model of a relationship is a small formula with two sides: the number of people who die equals the number of people who test positive, times some factor. If you made your plots well, you should be able to write what they show as a formula like that. And every earlier lesson can be a building block in it: a category, a time effect, a smoothing step, a distribution for the part you cannot explain.

The lesson is organised around six questions:

1. **What** is a model? A simplification you choose. Never the whole truth, and never neutral.
2. **Why** keep a model simple? A computer can fit anything. People can only understand simple shapes.
3. **How** do you build a model? Look at the data, think about the world, add one thing.
4. **When** should you believe a pattern? If you look at enough things, you always find something.
5. What can a **correlation** hide? A variable you left out can reverse a correlation, or create one.
6. Can you **recognise** someone by how they type? A model that works can still be right for the wrong reason.

The first two are about models in general, and about two ideas that almost every student brings in without saying them: that a model is true, and that a complex model is better than a simple one. The third is the heart of the lesson: the COVID model from the end of lesson 4, built step by step. The last three are about believing what you find.

3.1 What is a model?

A model is like a map: useful because it leaves things out. If you want to navigate, you don't use an aerial photograph. It has all the detail, and that is exactly what makes it hard to use. A map with a few colours and lines is better, because someone decided what to leave out[7]. A map of Utrecht that showed every stone would be as big as Utrecht. Modelling is the art of leaving things out, and that has three consequences.

- **A model is not true.** It is always an approximation, and there is always some distance between the model and the world. What you want is that the distance doesn't matter for what you use it for. Newton's laws describe how thrown objects move, and they work perfectly for everything you will ever throw. Near something very heavy, or at speeds close to the speed of light, they break down. Usually nobody cares, but your phone's navigation does: it times signals from satellites, and if it ignored the bending of time and space that Einstein described, you would not arrive where you wanted to go. So there you need the better model. Is Einstein's model then true? No; it is a better approximation, and it too breaks down in special cases. That is fine. It is still useful.

[7] "The map is not the territory" is Alfred Korzybski's phrase.

- **A model is not objective.** This is the misuse of models that bothers me most: "the data says so". The data says nothing. You chose the shape, the variables and the slice of data, and someone else could choose differently with the same data. Leaving out the choices you made is lying by omission: nothing you said was false, but the other person no longer has a good map of what you did.
- **A model is useful.** A good simplification shows you something you could not see in the raw numbers. You may treat discrete data as continuous and fit a normal distribution, knowing that it is not strictly right, as long as you can defend why. A Poisson distribution is not "true" either. It is also an approximation.

3.1.1 Leonardo's earth: not true, still useful

In his notebook on water and the earth, written in the early 1500s, Leonardo da Vinci drew the earth as a circle and a triangle: the circles are the oceans, the triangle the mountains and land masses[8]. He knew very well that water and mountains look different. He drew it because a geometric shape lets you reason about something too big and too complicated to see whole.

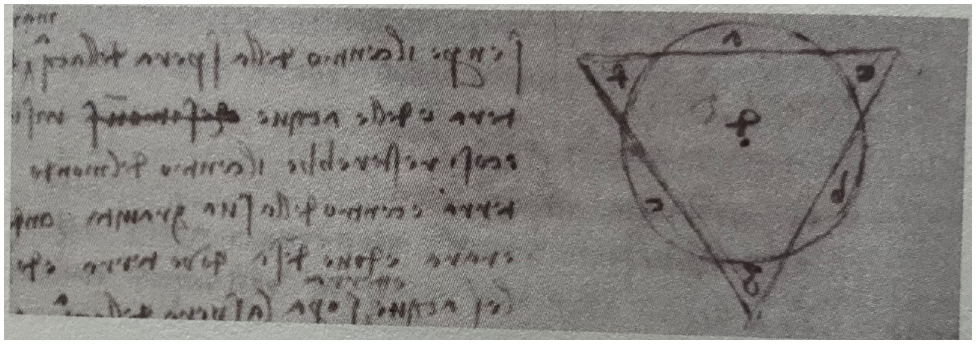


Figure 20: Leonardo's geometric model of the earth, from the Codex Leicester.

I find it a wonderful picture, even with everything we know now. It makes me see the earth as a drop of water floating through space, with the mountains as stones drifting in it. What is that useful for? I'm not sure; perhaps for thinking about the place of the earth in the universe. But it shows something new, even though its maker knew it wasn't true. Sometimes it is more interesting to ask *what does the model show me* than to strive for the absolute truth.

3.1.2 Three lines through the same cars

The same point with data. Figure 21 shows 398 cars, their weight against their fuel economy in miles per gallon[9]. Heavier cars use more fuel; the question is what kind of relationship this is, and each of the three lines is a different claim about that.

[8] Leonardo da Vinci, *Codex Leicester*, folio 35v, a detail.

[9] The Auto MPG dataset from the UCI Machine Learning Repository.

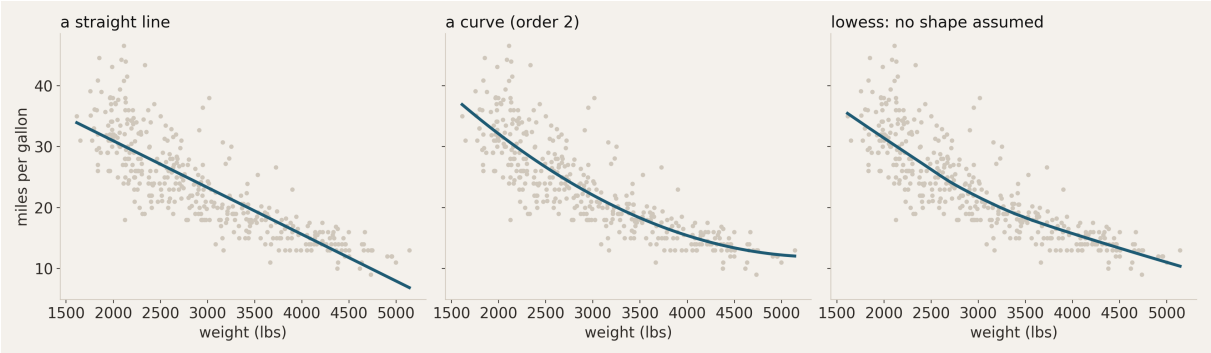


Figure 21: 398 cars, weight against fuel economy. A straight line, a line with a square in it, and lowess.

The straight line says every extra pound costs the same. It is not a bad model: $R^2 = 0.69$, and the p-value is around 10^{-103}. Both numbers are true, and neither tells you that the line has the wrong shape. The picture tells you immediately: the line sits below the lightest and the heaviest cars and above the ones in the middle. It is averaging a curve. Give the model one extra parameter, a term with a square in it, and it can follow the bend. Python then works out how much bend the data needs. On the right, *lowess* draws what the data shows without any formula: a kind of moving average that turns the points into a line. It finds the same bend without being told there is one, but it is not really a model: it has no parameters you can read, and it doesn't predict. Use lowess to look; once you know the shape, choose a formula.

Which of the three is the true one? None. They are three claims, and you pick the one you can defend. If you are choosing a car, "heavier cars use more fuel" is all the model you need in your head; nobody needs a quadratic to buy a car. Different goals, different models.

Again, concretely. The map is not the territory. A model is a simplification you choose: the shape, the variables, the slice of data. So it is not true and not objective, and its value often lies precisely in what it leaves out. Students often hand in a model as if the data produced it. It did not: you made the choices. That is not a weakness; it is the job. The weakness is not knowing that you chose. And if you choose anyway, why choose something simple? That is the next question.

3.2 Why keep a model simple?

The usual reflex is: why bother with simple models, when deep learning can learn anything from data? It is true that a computer with enough data can find very complex patterns, and for prediction that is often what you want. But people can only understand simple ones; there is a bandwidth to what we can follow. A deep learning model can have millions of parameters. It may predict well, but we can only understand what comes out of it, not what happens inside.

That matters more than it sounds. In my work for the province of Utrecht, imagine a brilliant system, even a flawless one, that says: build a roundabout there. Why? Trust me. That is not accepted, and it shouldn't be. People want to know why. A specialist who cannot explain everything can still give a metaphor or a story that makes it click,

and then people are willing to stand behind the decision. People take responsibility for something only when they can understand, follow or trust it, and a model cannot take responsibility. If the roundabout turns out to be a mistake and you ask who is responsible, the answer cannot be "nobody".

Simple models are useful for three reasons. They give *insight*: four simple shapes cover a surprising amount of the world, and each of them tells a story. They are *restricted*: a model that can fit everything tells you nothing, while a simple model can fail, so when it fits, that means something. And precisely because a simple model is limited, it is most interesting where it fails. Every time a simple model goes wrong, you learn something: you think about what is happening there, model it, and look at what is left. That leftover, the residual, is only visible because the model is not flexible enough to swallow it.

3.2.1 Four shapes worth knowing by name

The four horsemen of modelling are the line, the sine, the exponential and the logistic. You don't need to know their formulas by heart; you need to recognise them by sight and know when to use them, because each of them is a sentence.

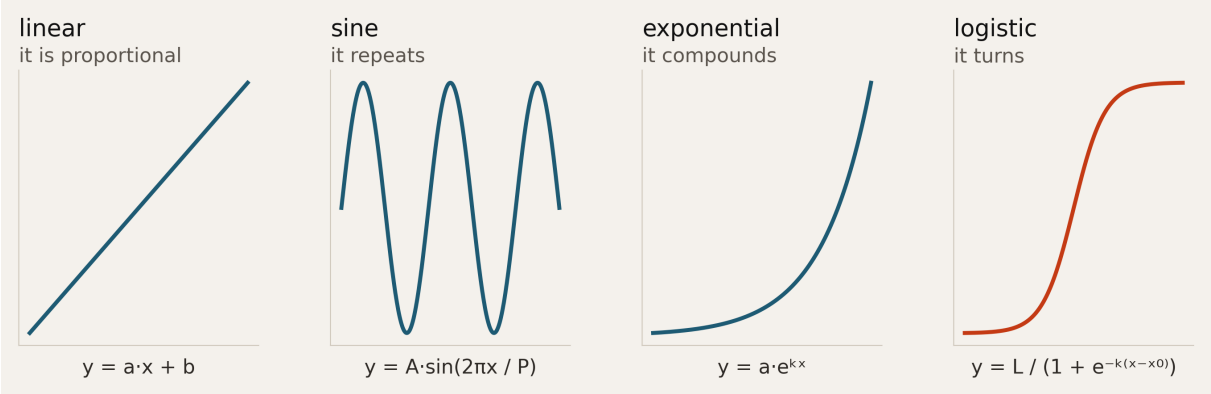


Figure 22: The four shapes: linear, sine, exponential and logistic.

- **Linear** says *proportional*: twice as much in, twice as much out. Price per kilo, or deaths per positive test: if a thousand people test positive and the factor is 0.01, ten of them die.
- **Sine** says *it repeats*: the same pattern on a fixed rhythm, every day, week or season. Usually a sine is about time.
- **Exponential** says *it compounds*: every step multiplies by the same factor, so it grows faster and faster. Interest, or the start of an epidemic.
- **Logistic** says *it turns*: it grows like an exponential at first, then slows down and levels off at a ceiling. Turned around, it is a switch from one level to another.

My estimate is that these four, and sums and products of them, cover about 80% of what you meet in data when you model it simply. The COVID model under question three uses three of them.

3.2.2 Nothing in nature grows exponentially for long

The old riddle: duckweed on a pond doubles every day and covers the whole pond on day 30. On which day was it half covered? Day 29, because the last doubling covers the second half. That is why exponential growth surprises people: for a long time almost nothing seems to happen, and then everything happens at once.

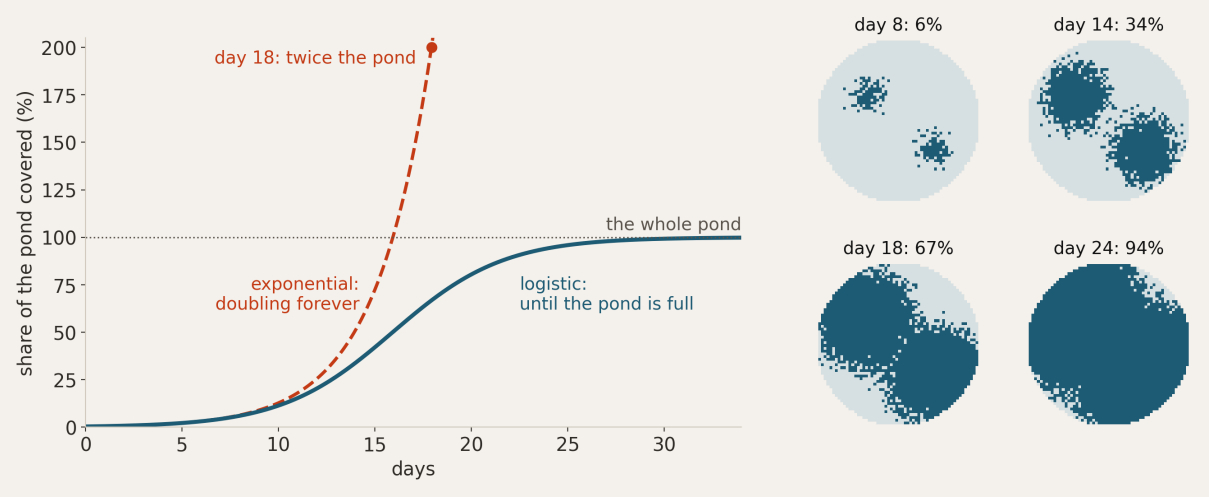


Figure 23: Duckweed that doubles every two days, as an exponential and as a logistic. An illustration, not measured data.

A real pond adds what the riddle leaves out: the pond is only so big. In figure 23 the duckweed starts on 0.4% of the pond and doubles every two days while there is room. The exponential keeps doubling and covers twice the pond on day 18, which cannot happen. The logistic has the same start and the same doubling, and slows down as the free water runs out: half covered on day 16, nearly full by day 25. Natural systems never grow exponentially for long, because something always runs out: room, food, light, or for an epidemic, the people it can still infect. Markets don't keep growing either, however much shareholders want them to. The dangerous part is that in the first stretch of the curve you cannot see the difference between the two. If you extrapolate an exponential, think about where the edge of the pond is.

3.2.3 A line plus a sine: four numbers, 93% explained

Monthly airline passengers from 1949 to 1960 combine two of the shapes: a line for the trend, and a sine for the summer holidays. A straight line in time, about 2,650 more passengers every month, explains 85% of the variation. Add a sine that repeats every twelve months, which costs two extra numbers (its amplitude and its phase), and that rises to 93%.

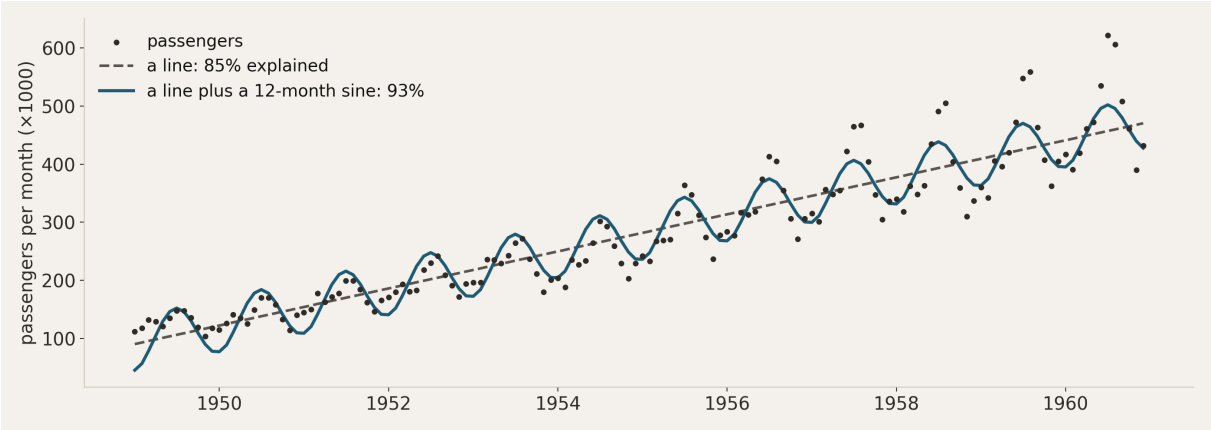


Figure 24: Airline passengers, 1949–1960: a line, and a line plus a sine.

Four numbers in total, and you can say each of them out loud: how much the trend grows per month, how big the summer peak is, and when in the year it falls. That is a summary of a phenomenon in a few parameters you can interpret. And what is left over is honest and readable: the real summer swing gets wider as the trend grows, while a sine added on top of a line keeps the same width. You can only see that because the model is simple enough to read. How would you let the swing grow? Ask yourself which number in the formula of the sine sets its height, and what that height should depend on. This is an exercise in notebook 05.4 (section 5.4.3, "a wave that grows"), so I leave it to you.

3.2.4 A better fit is good, until you fit the noise

Of course you want a model that fits better; that is what fitting is for. The problem is going too far. Every dataset is one sample, with its own accidents in it, and a model flexible enough to follow every point follows the accidents too: it models the noise. Lesson 4 gave the alternative. Noise is a factor in its own right, and you can model it: give it its own term, ε, say which distribution you expect, and check the residual against it.

$$y = f(x) + \varepsilon$$

The model then doesn't need to go through every point. As long as the data stays within the bandwidth of ε, it is a good description that respects the uncertainty.

The second point is about the goal. Sometimes the goal is prediction, and then a complex model is fine: nobody needs to read the weights of a neural network. But often the goal is people: a plot, a formula, a number with a sentence that someone else has to understand, check and act on. That artefact may well have been produced with the help of a complex model, but it still has to be understandable, because what people don't understand, they cannot work with. Fit the pattern, give the noise its own term, and hand over something people can understand.

3.2.5 A model that can fit anything tells you nothing

Twelve cars from the same dataset, spread over the weights (figure 25, top). A straight line fits reasonably: $R^2 = 0.72$. A polynomial of degree 11 has twelve parameters for twelve points, so it goes through every single one: $R^2 = 1.00$, a perfect score. This is the result that tempts students: look, machine learning, a perfect fit.

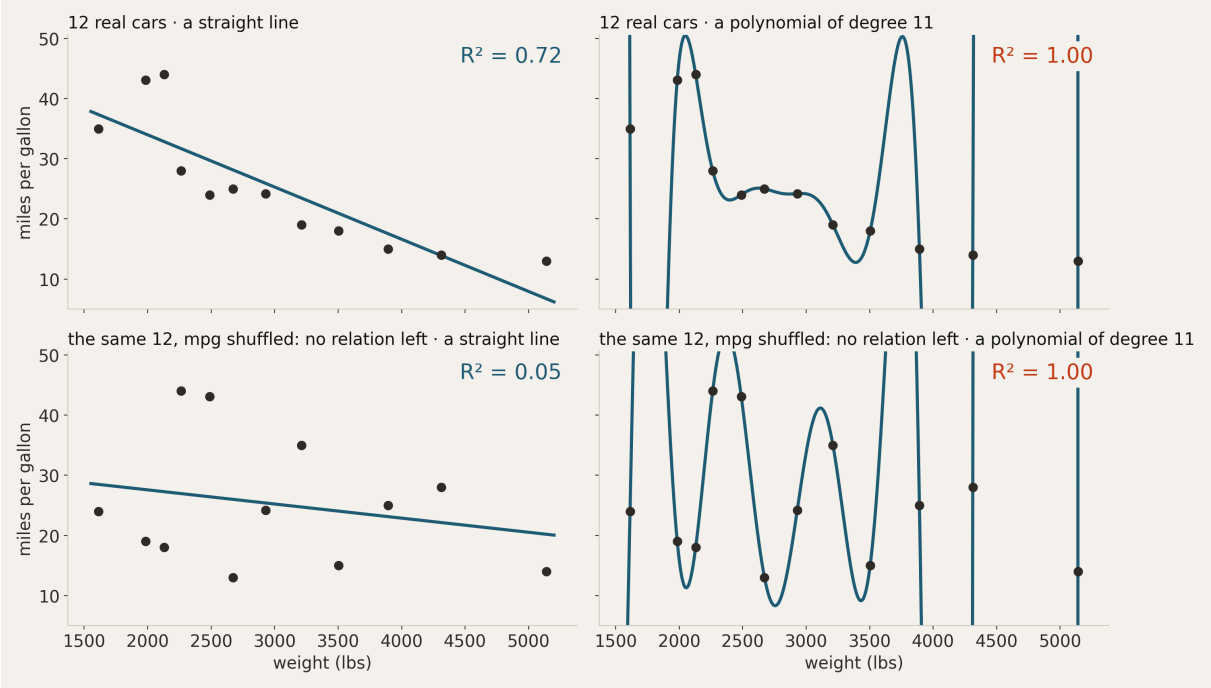


Figure 25: Top: twelve real cars. Bottom: the same cars with the fuel economy shuffled.

Now shuffle the fuel economy values between the cars (bottom row). There is no relationship left; any pattern is chance, and a good model should break. The line does: $R^2 = 0.05$. Over 200 shuffles its median is 0.05, and 95% of the shuffles stay below 0.28. The polynomial scores a perfect 1.00 again, on nonsense. You might think it is just so good that it always finds the pattern. It isn't; it is too flexible. A model that scores perfectly on real data and on nonsense cannot tell them apart, so its score is worthless. There is a paradox in this: because the line can fail, its 0.72 tells you something about cars. This is the same shuffle as in lesson 4: ask what you would see if there were nothing there.

3.2.6 Too simple, about right, too flexible

Now test the three kinds of model on the 386 cars they never saw, in the same weight range (figure 26).

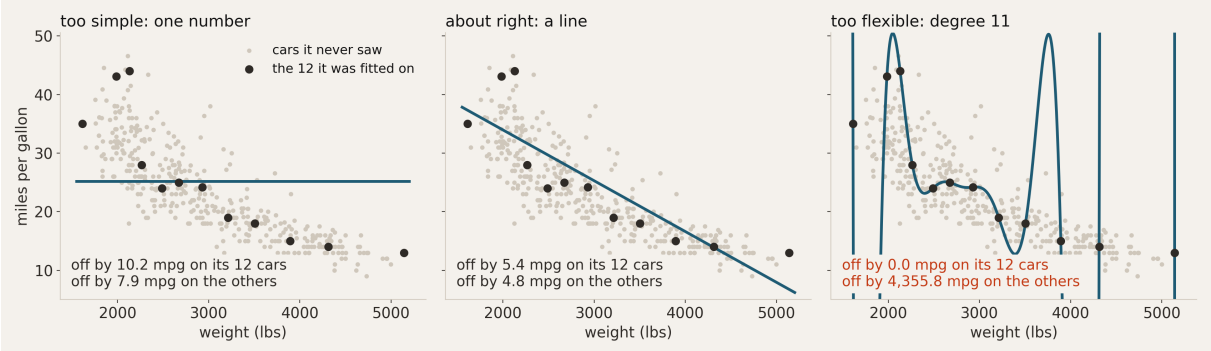


Figure 26: The same twelve cars, and 386 cars the models never saw.

- **Too simple** (*underfitting*): one number, the mean, for every car. It is about 10 mpg off on its own twelve cars and about 8 on the others. Bad everywhere, but at least

honestly bad: it misses the pattern.

- **About right**: a line, about 5 mpg off on both. It is a bit wrong on its own cars and about as wrong on cars it has never seen. It learned something that carries over.
- **Too flexible** (*overfitting*): the polynomial is 0 mpg off on its own twelve cars and on average more than 4,000 mpg off on the others, because between the points it swings wherever it likes. It learned the twelve cars, not cars.

The tell-tale sign of overfitting is exactly this gap: very good on the data it saw, bad on new data. The trap is to look at the zero and think: look how good I am. You want a model that is about as wrong on new data as on old. Lesson 6 gives this its own machinery, with train and test sets; for now the intuition is enough.

3.2.7 If you can't explain a number, don't report it

This is a real report, handed in at another university of applied sciences, by a student who had worked on it for three months. The task was to predict the power draw of an engine. At the defence, two teachers and a half-hour presentation, the student showed six slides of MAPE formulas, all generated, and this table:

Model	**MAPE**	**MAE**	**RMSE**
Random Forest	4.78×10^{13}	0.89	2.5
XGBoost	5.48×10^{13}	0.53	1.3
KNN regressor	6.48×10^{13}	1.46	3.74

and the conclusion: "These results suggest that the model is a robust and reliable tool for predicting power draw of the engine". MAPE is the *mean absolute percentage error*: on average, how many percent the prediction is off. A MAPE of 10^{13} means the model is off by ten trillion percent. If you understand what MAPE is, you cannot let that pass. So I asked: suppose I pay your salary with a model that has this MAPE. Are you worried? The student didn't really know. It means I might pay you one euro or ten billion euros, and I can't tell you which. Only then did it begin to dawn. The student failed.

The MAE column is no better. An MAE of 1.46 means nothing until you compare it with a baseline error, for instance the error of always predicting the average. If that baseline is 0.001, then 1.46 is bizarrely high; if it is 100, the model is excellent. Without the baseline nobody, the student included, can say whether the model is any good. (Why can MAPE reach 10^{13}? It divides each error by the true value, so true values at or near zero blow it up. The student could not say that, and that is the point.)

In class someone asked whether there might have been an explanation for the number. Perhaps; but that is beside the point. The problem was that the student could not explain what was worrying about it, and the defence is far too late to discover that. If you present an employer with a robust model that is off by ten trillion percent and you don't see the problem, they will stop trusting the rest of your work too. It is like looking in your rear-view mirror on the motorway and seeing chunks of rubber flying and a shower of sparks behind you, because you are driving on the metal of your wheels, and concluding that your car might need some attention. That is not a maintenance issue. You stop. If you don't notice, you are a danger on the road.

So: use the tools, including an LLM, but every number on your slides is one you must be able to explain and defend. A number you can't explain is not a result.

When the simple model is the solution. A case from practice. At the start of the pandemic, the sales of Strepsils throat lozenges shot up and everything sold out. A complex sales model went completely off the rails trying to fit that peak. What worked was the opposite: a simple model with five or six sine waves for the rhythm of the year, which by construction could not follow the peak. The difference between that simple model and what actually happened we called the *panic factor*: one parameter, fixed at zero up to the point where the simple model stopped working, and free after that. The model then had to put the whole COVID effect into that single parameter, so the effect was captured in one place. We then modelled three scenarios for the panic, that it would fade, disappear or stabilise, and gave the manager a knob to choose between them. That only worked because the model stayed simple enough to explain.

Is that an argument that simple models are usually right? Usually, yes; it is a trade-off. Simple models leave more on the table, but their inflexibility makes them reliable. There is also the opposite reflex: people so afraid of anything that cannot be explained that they stay simpler than necessary and reduce everything to a decision tree. In healthcare, for example, very complex models are used, because you can put far more information into them than any person can hold. That can be the right choice.

Again, concretely. A simple model can fail, and that is a signal you can use. Complex models find complex patterns, but you can't read them. Four shapes, each a sentence; a flexible model is not closer to the truth, it is closer to the noise; and a number you cannot explain does not belong in a report, however it was produced. Start simple. Add a term only when what is left over has a shape you recognise. If a model fails, that can be a very interesting signal about reality.

3.3 How do you build a model?

Every step goes the same way. You *look*: plot the data and notice what is strange, such as noise, a delay or a difference in size. You *think*: use what you know about the world, which is often not in the data at all. How does a disease progress? How does reporting work? When did the vaccine come? In your own chat this is knowledge like "aunt Connie was single and bored that winter". And you *add*: turn each piece of reasoning into one step in the pipeline, or one term in the model. Then you look again.

The data are the public Dutch COVID figures: positive tests and deaths per day, from October 2020 to June 2021. The model in the end is small, but the way there is the lesson. Notebooks 05.3 and 05.4 follow the same path, and `scripts/covid_pipeline.py` holds the whole loop as a script.

3.3.1 Step 0 · A running total. How many died per day?

The data as it arrives. The Dutch figures give deaths as a running total: everyone who has died of COVID since February 2020, 6,419 on 1 October 2020 and 17,632 on 1 June 2021. That line can only go up, and the number that matters is hidden in how steep it is.

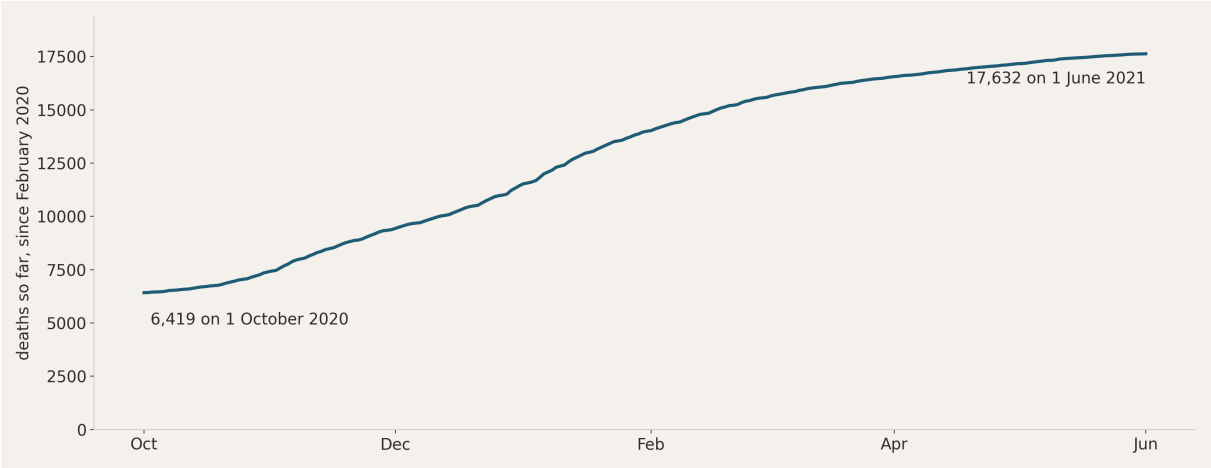


Figure 27: The deaths as delivered: a running total.

How do you get the deaths per day out of this? Take the difference with the day before: deaths today = total today − total yesterday. In pandas that is `diff`, and in the pipeline it is the first step:

```
pipeline.add(DiffValues, column="deaths")
```

3.3.2 Preattentive processing: what do you see?

In the first fraction of a second, before anyone reads an axis or a number, your visual system has already picked out what stands out: repetition, contrast, a pattern that keeps coming back. So look at the numbers exactly as reported, and ask yourself what you see before reading on.

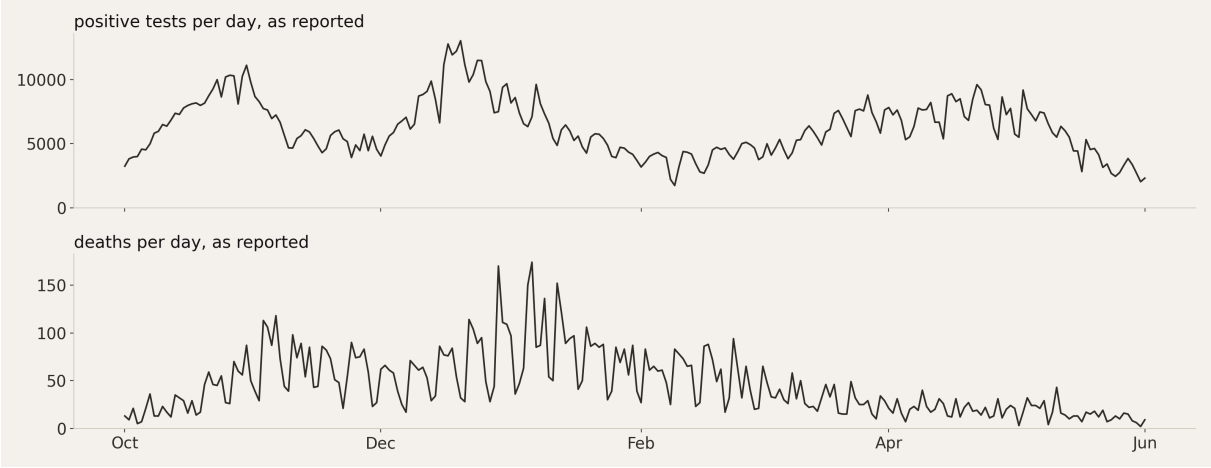


Figure 28: Positive tests and deaths per day, as reported.

What jumps out first is the *weekly cycle*, strongest in the deaths. On average about 25 deaths are reported on Sundays and Mondays, and 67 on Tuesdays. In class, the first explanations were the waves and the vaccine; those are the model, and we will come to them. The weekly rhythm is something else: people do not die less on Sundays. Registration waits for the weekend to end and catches up afterwards.

3.3.3 Step 1 · Too much noise: average over a week

Should you remove the weekly cycle? That is a decision, and it depends on what you want to model. Here we are not interested in it: it tells us more about society than about the disease. Knowing that, the fix is obvious: average over seven days, so that every average contains each weekday exactly once.

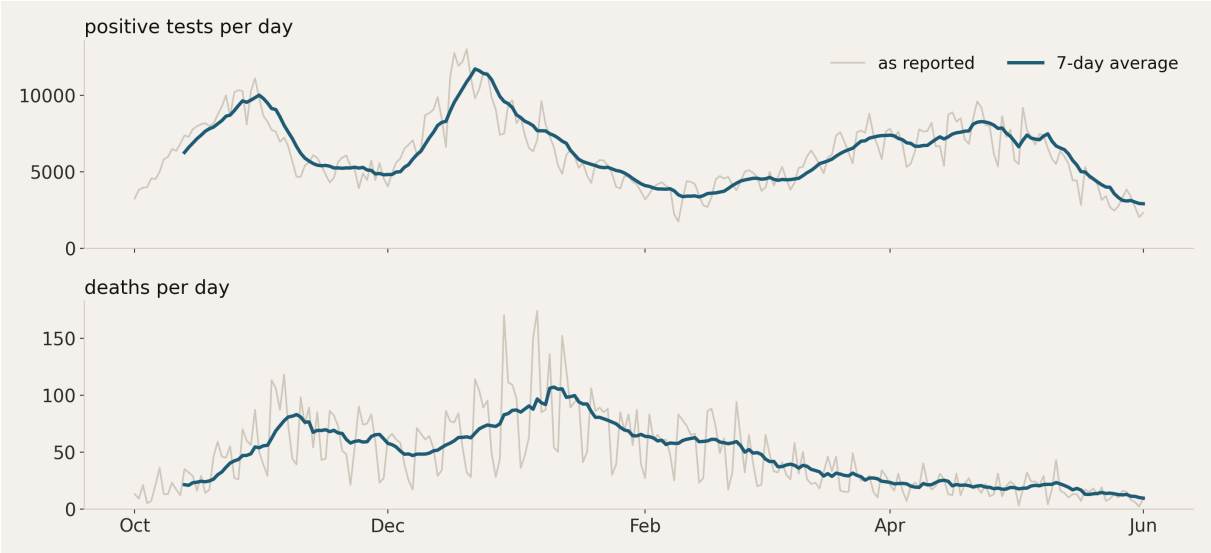


Figure 29: Grey: the numbers as reported each day. Petrol: the average over the last seven days.

The weekly rhythm cancels out, and the course of the epidemic is left.

```
pipeline.add(RollingAvg, column="positivetests", window=7)
```

and the same for the deaths.

3.3.4 Step 2 · Deaths come about two weeks after a positive test

With the weekly noise gone, something else becomes visible: the peaks in deaths come after the peaks in tests. You don't die on the day you test positive. Infection, illness, hospital, death: that usually takes about two weeks. That is knowledge from outside the data, from the medical literature (and at the time, from the news). So the deaths recorded fourteen days from now are the deaths of today's positive tests, and a model should pair them that way.

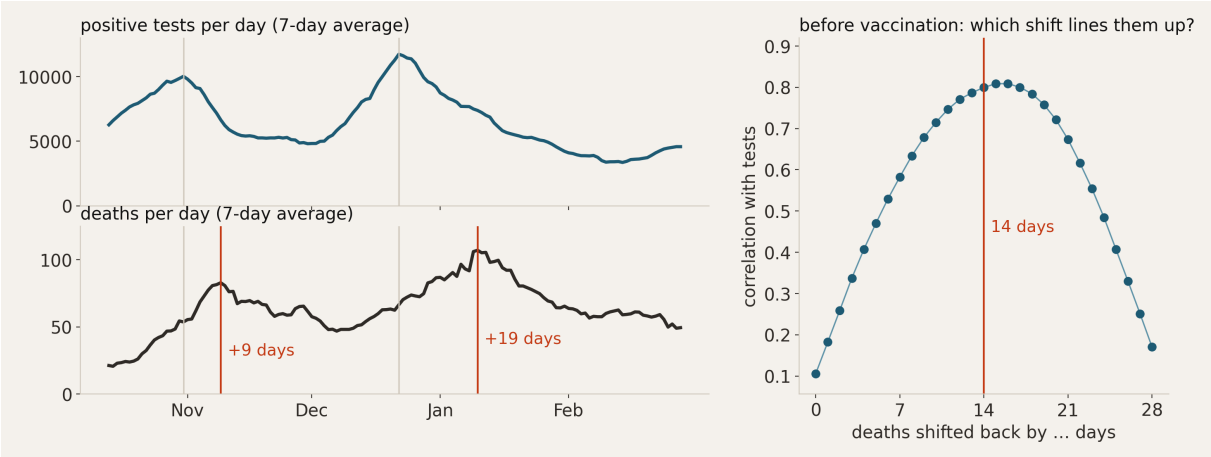


Figure 30: Left: tests peak, deaths follow. Right: the correlation between tests and deaths for every shift from 0 to 28 days, before vaccination.

The left panel shows it with the naked eye: the deaths peak 9 days after the tests in November, and 19 days after them in January. Why the difference? In class we came up with several explanations: more testing discipline when people are worried, so infections are caught earlier; the most vulnerable people being infected and dying first; the holidays, when fewer people die and patients are kept alive until after New Year. We don't know. The right panel checks the two weeks: shift the deaths back by $0, 1, 2, \ldots, 28$ whole days and compute the correlation with the tests for each shift. The best shifts are around 14 to 16 days. The data agrees.

Why keep 14, and not take 16, which fits a little better? Because I don't know the mechanism behind 16. It could be chance, Christmas, or any of the explanations above. If I tune the shift to this data, I artificially improve my model now and risk a worse fit in the future: I would be fitting noise. Note the order: the 14 came from the literature, and the data confirmed it. Searching the data for the best lag and using that would be searching again, like the search for the switch day in lesson 4. If you want 15.6 days, find it in the literature, or in data from other countries.

In the pipeline this is a shift of the deaths column, and it is also where the model starts:

```
pipeline.add(ShiftValues, column="deaths", period=-14)
```

$$\text{deaths}(t) = a \cdot \text{tests}(t - 14).$$

What is a? "The coefficient of the line" is the mathematical answer. In normal language: if a hundred people test positive, a is the share of them that dies two weeks later. Every term in a model should have an answer like that.

3.3.5 Step 3 · Tens against thousands: compare with z-scores

You want to see whether the two series have the same shape, but they live on different scales: thousands of positive tests a day, tens of deaths. On one axis the deaths are a flat line at the bottom.

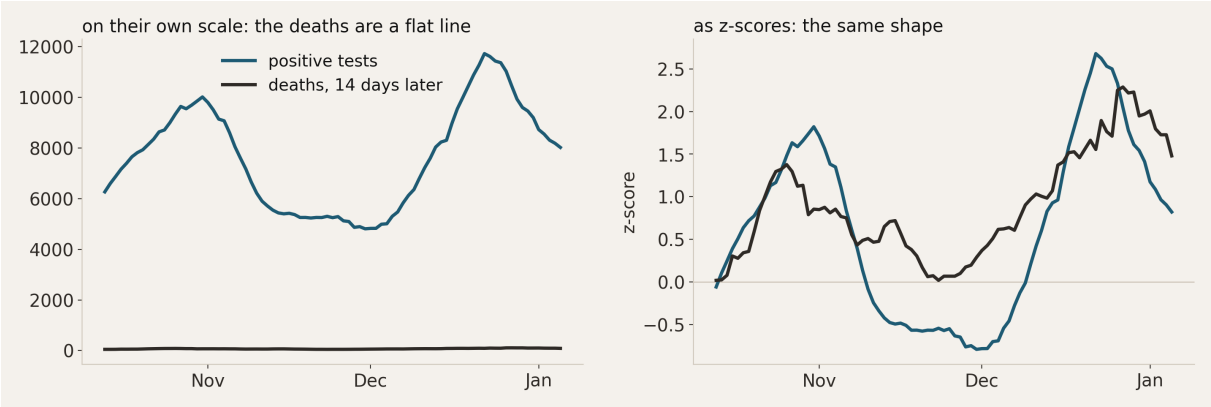


Figure 31: Left: tests and deaths on one axis. Right: both as z-scores, the deaths shifted two weeks back.

A *z-score* puts every series on the same scale: subtract its own mean and divide by its own standard deviation,

$$z = \frac{x - \text{mean}}{\text{sd}}.$$

Zero is an average day for that series; plus two is a very unusual one. In November there were two standard deviations more positive tests than average, and one and a half standard deviations more deaths. As z-scores, the deaths two weeks later follow the shape of the tests. You know z-scores already; the point here is why you use one: to compare shapes, not sizes.

```
pipeline.add(ZScaler, column="positivetests", rename=True)
```

Be careful not to put the z-scores into the model. You could, but then a would no longer be a share of people who die; it would become something like 0.8, meaning nothing you can say out loud, and you would have to calculate back. The z-scores are for looking; the model uses the counts.

3.3.6 Step 4 · Add what you know: vaccination started on 6 January

Over the whole window the picture changes. Until January the two curves move together; from February they part. The tests rise again in spring, the deaths keep falling.

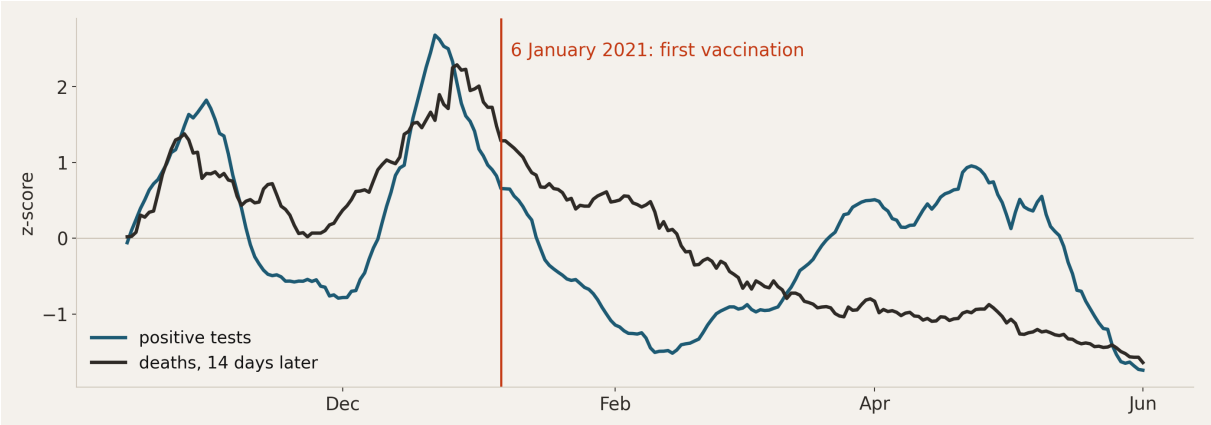


Figure 32: The whole window as z-scores, with the first vaccination in the Netherlands.

The data doesn't say why; it has no column for it. You know something it doesn't: the first vaccination in the Netherlands was given on 6 January 2021. I didn't get that date from the data but from the news. People who are vaccinated still get infected; they just die much less often. (Could it be false positive tests? Possibly, but tests are fairly reliable, and you would also have to explain why they gave few false positives before January and many after.)

The date does two things. It tells us where the relationship between tests and deaths is likely to hold, before vaccination, so that is where we fit the first model. And it gives us a guess for where something might change. The pipeline's `SelectDataRange` makes the same kind of decision for the whole study: which months to look at, here 1 October 2020 to 1 June 2021.

3.3.7 Add b: the deaths the tests don't explain

A line is $a \cdot x + b$. Instead of adding a b because lines have one, ask what it would mean here. Fit the model from step 2 on the 85 days before vaccination, and the line has to go through zero: $a = 0.0090$, the dashed line in figure 33. Now add a constant:

$$\text{deaths}(t) = a \cdot \text{tests}(t - 14) + b.$$

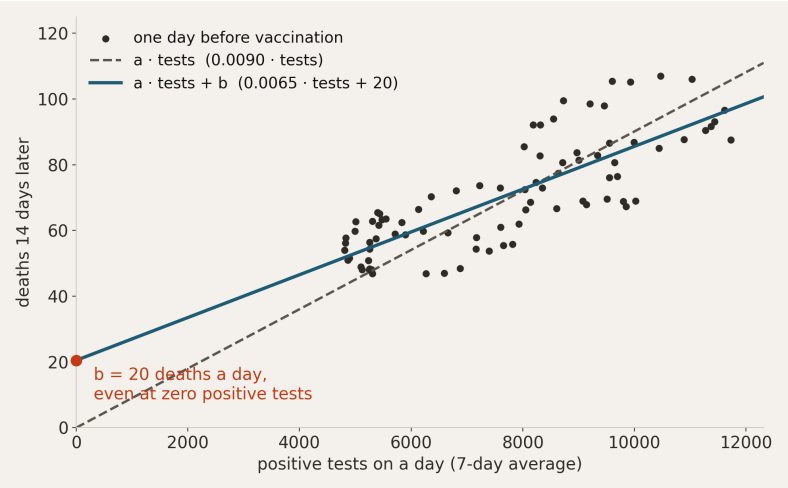


Figure 33: The 85 days before vaccination, with and without b.

The fit puts a at 0.0065: of every hundred positive tests, 0.65 end in death two weeks later. And b at about 20 deaths a day, even at zero positive tests. Those are deaths that happen whatever the number of tests: people who were never tested, or people who die *with* COVID rather than *of* it, because they were positive and old. With b the line is closer to the points: the mean squared error drops from 127 to 99. And b gives the model room for deaths that stay when a changes, which we will need shortly. One caution: no day in this window had fewer than about 4,800 positive tests, so b is where the line meets zero tests, an extrapolation. Read it as the deaths the tests don't account for, not as a measured count.

3.3.8 Step 5 · Say how wrong the model is allowed to be

The last term is the noise:

$$\text{deaths}(t) = a \cdot \text{tests}(t - 14) + b + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma).$$

Writing down ε is a claim: everything the line doesn't explain (lockdowns, demonstrations, Christmas, new variants, people having coffee together anyway) is treated as chance, normally distributed around zero. Here σ is about 10 deaths a day. That is modelling the uncertainty: the model says not only what it expects, but how far off it is allowed to be.

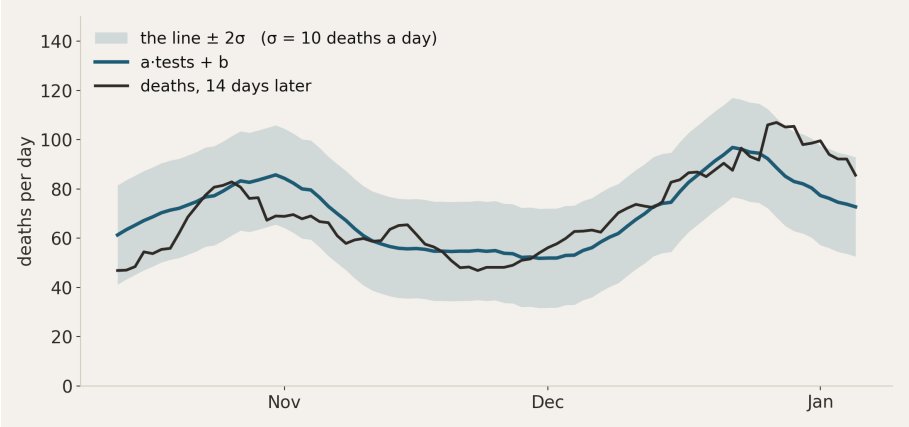


Figure 34: The model with a band of $\pm 2\sigma$, October 2020 to 5 January 2021.

How do you find ε? You know the deaths and the tests; you don't know a, b or ε. But you can fit a and b, and then what is left is ε:

$$\varepsilon = \text{deaths}(t) - \Big(a \cdot \text{tests}(t - 14) + b\Big).$$

That is the residual from lesson 4. Plus or minus two σ should hold about 95% of the days; it holds 98% of the 85 days before vaccination. And because ε is a claim, it can be checked: if the errors stop looking like chance, the model is missing something. goad's `DistributionFitter` is the tool for that check. You could argue that the bump around Christmas is not chance; then add Christmas to the model. Treating it as noise is a choice, and it is a choice you make out loud.

3.3.9 Every step of reasoning becomes a line of code

This is the part to remember. Every piece of reasoning from the last five steps ended up in one of two places: the pipeline that prepares the data, or the formula of the model.

Pipeline step	The reasoning behind it
`DiffValues`	A running total of deaths; you want deaths per day.
`ShiftValues(-14)`	Deaths come about two weeks after a test: the literature.
`SelectDataRange`	The months you want to study.
`RollingAvg(7)`	Weekends are society, not disease.
`ZScaler`	Tens against thousands: compare the shapes.

Table 2: The pipeline, `CovidDataProcessor`, and the reasoning behind each step.

In the model, tests$(t - 14)$ is the same two weeks as `ShiftValues(-14)`, now in the formula. The parameters a and b are fitted on the days before 6 January, a date from outside the data. And ε says that what the line can't explain is chance, which is itself a claim the residual can check. A `DataConfig` holds the 14, the 7 and the window, so every decision has one place where it lives. Nothing here came out of the data by itself. Each step is a decision you can explain and point at, and that is why a pipeline is built from named steps instead of one long chain of pandas.

3.3.10 Previous lesson: the residual shows what the line misses

Now take the line exactly as fitted before vaccination, with the same a and b, and run it over every day up to June. Nothing in it knows what month it is.

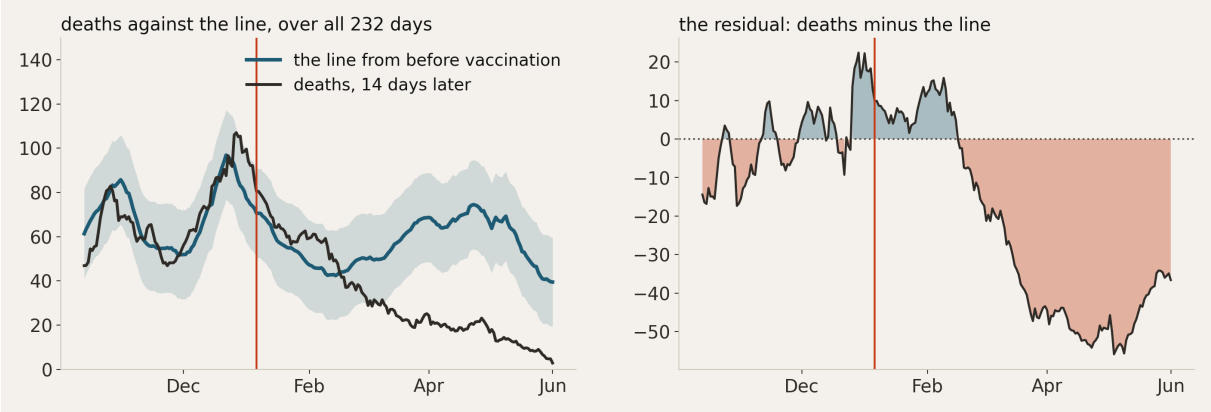


Figure 35: Left: the line from before 6 January, over all 232 days. Right: the residual.

Until January the deaths stay inside the band. From February they drop below it and never come back: 60% of the days after 6 January fall below the band. Even what we modelled as noise no longer covers it. The residual is below zero from 16 February to the end, at its worst about fifty deaths a day. Up to January you could still say: sometimes a bit higher, sometimes a bit lower, all chance. After that it is no longer chance. The errors have a shape, a long run in one direction. One fixed share of tests ending in death no longer works. Something changed the ratio, and the residual says roughly when. How do you put that into the model?

3.3.11 How do you model a vaccine? Look at the ratio

Not with a separate line per month, and not with a polynomial in time that bends until the residual is flat; that would match the picture and explain nothing. This is the moment many students say: my linear model and my random forest don't work, so now I'll try deep learning. Don't throw the next model at it. Think about what a vaccine does: it changes the share of positive tests that ends in death. So plot that share.

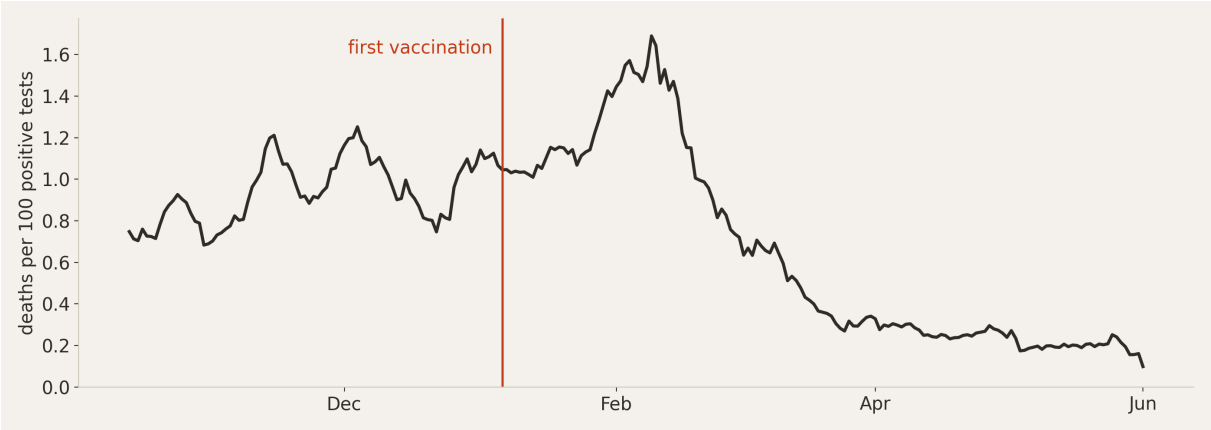


Figure 36: Deaths two weeks later per 100 positive tests.

Through the winter about one death per hundred positive tests, between 0.8 and 1, with a bump in early February. Then it falls steeply, to 0.3 in April and 0.2 in May. A vaccine doesn't work on one day. The Dutch rollout started on 6 January with care-home residents and the people caring for them, and worked down the age groups from the oldest, through February and March. Slowly at first, then everybody wanted their turn, and at the end a long tail of people who came late or hesitated. Once the most vulnerable groups are protected, the ratio settles lower.

One level, a turn over some weeks, a lower level. Which of the four shapes is that? A logistic, turned downward. Why would the model for duckweed on a pond fit vaccination? Because both grow until they run into a limit: the pond is full, or the vulnerable people have had their turn.

3.3.12 A logistic switch has two knobs: when and how fast

The vaccine gets a term of its own, a *switch* $s(t)$:

$$\text{deaths}(t) = a \cdot \text{tests}(t-14) \cdot s(t) + b + \varepsilon, \qquad s(t) = \frac{1}{1+e^{-k(t-x_0)}}.$$

Turned downward, the switch starts at 1: of every positive test, the old share ends in death. Multiplying by one does nothing, so before vaccination this is the same model as before. It ends at 0: a positive test no longer leads to death. Halfway, at 0.5, half of the deaths the tests would predict happen.

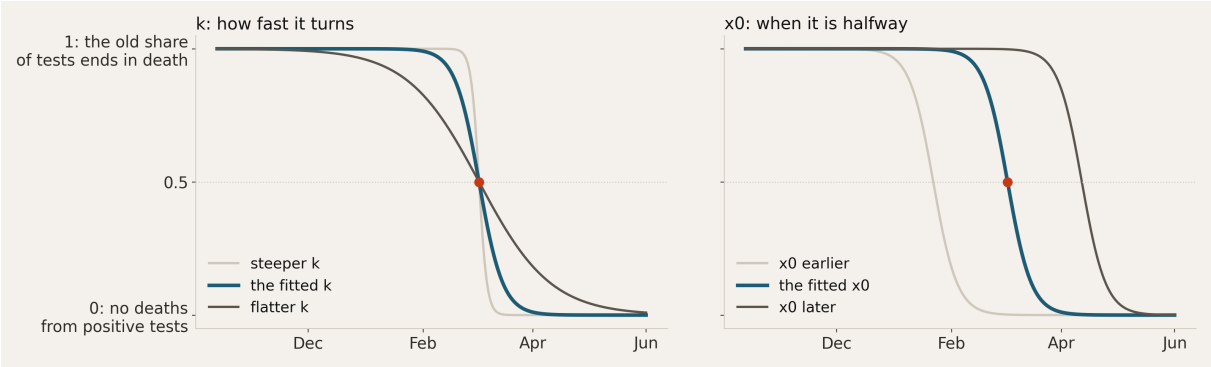


Figure 37: Left: the same x_0 with a steeper and a flatter k. Right: the same k with x_0 earlier and later.

The switch has two knobs. x_0 sets *when*: the day the switch is halfway; it slides the curve left and right. k sets *how fast*: a large k turns in days, a small k over months. We don't know either of them, and that is exactly what we want the fit to tell us: here is the formula, here is the data, find out how steep the turn is and where it lies.

Notice where the switch sits. It multiplies only the tests term. When it reaches 0, what remains is b and the noise: deaths that have nothing to do with positive tests. That is the claim of the model: vaccination breaks the link between testing positive and dying; it does not make other deaths disappear. That is not exactly how the disease works, of course. Even vaccinated people sometimes die of COVID, so you could argue the switch should not go all the way to zero. You can keep refining; at some point that is a job at the RIVM, the Dutch public health institute.

3.3.13 The switch, fitted: the date falls out of the fit

Four numbers, fitted together: a and b from the line, k and x_0 from the switch. The only help the optimiser got was a starting value for x_0, about a month after the first vaccination, and bounds that say what each parameter may mean.

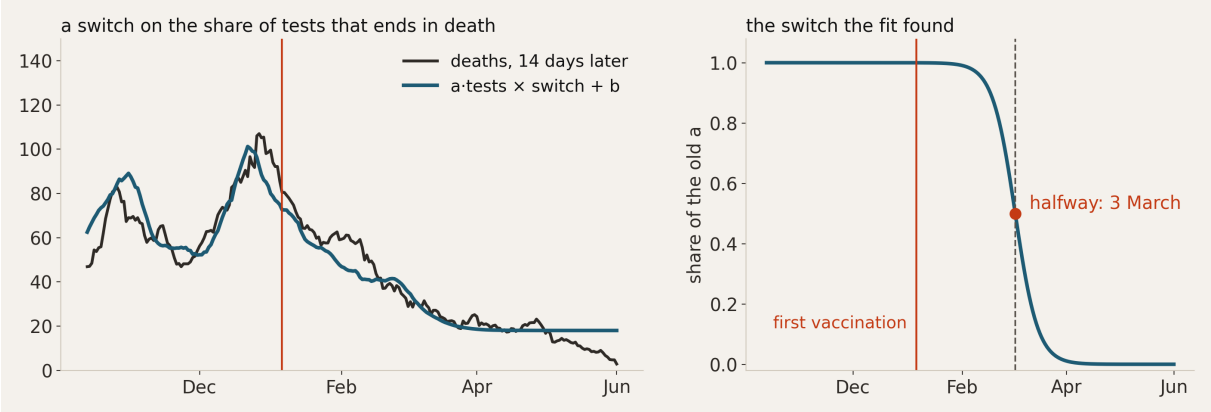


Figure 38: The model with the switch, over all 232 days.

The fit puts the turn halfway on 3 March 2021, eight weeks after the first vaccination. The turn is fairly quick: the switch is still at 0.99 on 1 February, 0.58 on 1 March, and close

to 0 by 1 April. And $a = 0.0071$ and $b = 18$, close to the line fitted before vaccination (0.0065 and 20): the switch doesn't have to bend the line, it only takes the tests term away. The mean squared error drops from 800 for the line alone to 66.

Look at what we combined. A time series, with a factor that changes over time. A linear relation between two variables. Something like categories: people who die after a positive test, and people who die anyway. Noise from lesson 4. And a logistic from this lesson. This is not deep learning. It is every lesson so far in one formula, with four numbers you can say out loud.

3.3.14 The residual looks like chance again

Is the model done? Apply the bar from lesson 4: no long structure over time, and a distribution the data cannot pin down.

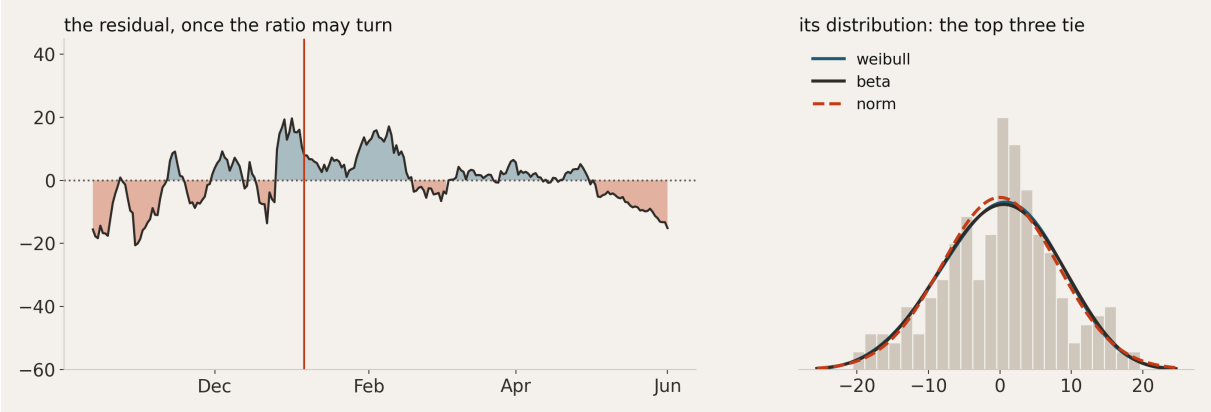


Figure 39: Left: the residual of the model with the switch. Right: its distribution, with the three best families.

The long negative run after vaccination is gone; what is left are waves of a few weeks around zero, probably lockdowns and holidays. Weibull, beta and normal fit the errors almost equally well, within about one log-likelihood point of each other, and none is rejected by the KS test. The data can't tell them apart: that is what chance looks like. From May the residual drifts below zero again: the model now says there are $b = 18$ deaths a day, and fewer happen. That is the next thing a serious model would treat. How could b be better modelled, if you think of the four shapes? Perhaps as a season: more people die of flu in December than in June (although in June some die of heat).

Then the honest part. The model has a switch on the calendar, not a term for vaccination. The end of the winter wave, a change in who was being tested (at some point people stopped getting tested) and the order in which age groups were vaccinated all sit on the same calendar, and each could produce a switch that fits. The fit shows that the ratio turned, and when. Why it turned is a separate claim. Defending it needs other evidence: vaccination coverage by age group against deaths by age group, or the same fit on a country whose rollout started on a different date. That would also average out the effect of the Dutch lockdowns. And this happened once: there is no second pandemic to test the model on.

When do you stop? That depends on your goal. In lesson 4 we saw that a mechanism that is right 51% of the time on a fifty-fifty bet can make you rich. If you predict whether someone is ill and send them home, a wrong prediction can cost a life, and you need a very different standard. That is why a hospital asks your date of birth eight times before an operation: a very simple test, repeated until it is almost impossible to cut into the wrong patient. Predicting the future is a profession of its own, and a well-paid one. During the pandemic the RIVM ran large Bayesian models with assumptions about everything: what share of people keep a distance, how honest they are when asked, how many keep to the rules at Christmas. At some point the uncertainties multiplied so much that they said they could not predict more than two weeks ahead. That is what honest modelling looks like: you are better than guessing, and you say what you don't know.

I don't expect your own model to become this elaborate. What I wanted to show is the process: you take a step, something is left over, you investigate it, you model it, and something is left over again. And then you stop when what is left looks like real noise.

3.3.15 The whole loop as a map, drawn by atlas

The loop we just built also lives in `scripts/covid_pipeline.py`: six small functions and a `main` that runs them in order.

```
def main() -> None:
    data = preprocess()
    plot_zscores(data)
    data = fit_model(data)
    plot_model(data)
    plot_residual(data)
    plot_residual_distribution(data)
    logger.success("All done.")
```

Atlas, a tool I wrote for this course[10], draws what that code does (figure 40). It doesn't show files and folders; it shows where data rests and the functions that carry it from one place to the next, in the order they run. Read it from left to right: the command starts the script, `preprocess` makes a DataFrame (the goad pipeline from table 2), `fit_model` turns it into a second DataFrame with the prediction and the residual, and the plotting functions write PNG files. A rounded pill is a function, a box is where data rests: a DataFrame in memory, a file on disk. Click a pill and atlas opens it into its own steps.

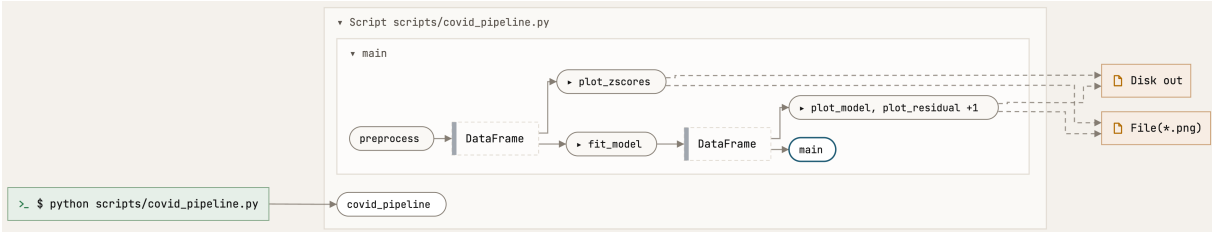


Figure 40: `scripts/covid_pipeline.py`, drawn by atlas.

The idea behind atlas is that consistent, logical code gives a consistent, simple picture. The same plot can be produced by tidy code and by messy code, and the pictures will

[10] `https://github.com/raoulg/atlas`, with an install script for macOS, Linux and Windows.

look very different: global DataFrames that get written and read somewhere along the way, steps whose order you can no longer see. A linter doesn't notice that: if you make three plots that are really one flow, only you know it should look like one flow. When the picture looks wrong, the code usually is, and the red marks show where atlas misses information: unclear inputs, unclear outputs. It works the same for Python and Rust, and you can give the picture to Claude to discuss what to change. In the final assignment you use it to show how your own code works: run `atlas-code view` in your repository, pick your entry point, and be ready to explain the picture. Section 6 says why a picture of your code helps, and how to read one.

Again, concretely. One line, then one switch. The line from before vaccination kept its two numbers; the switch added two more and found the turn eight weeks after the first vaccination, without being told. Look, think, add one step or one term, then read the residual. Each piece of reasoning has a place: the pipeline, or the formula. Stop when what is left looks like chance, and say what the model cannot tell you: here, why the ratio turned.

3.4 When should you believe a pattern?

In lesson 4 you shuffled labels to see how big a difference chance makes for one comparison. Now: what happens when you make many comparisons, and what makes a finding believable at all? The data is the Ubuntu IRC chat channel you know from earlier lessons.

3.4.1 Fifteen comparisons, one below the line

Everyone in #ubuntu-uk with at least thirty messages, 442 people, and fifteen numbers per person: message length, links, questions, posting at night, and so on. Each person also has a cohort label, A or B. Test every number between the cohorts with a t-test, and put the fifteen p-values on a log axis with the usual line at 0.05.

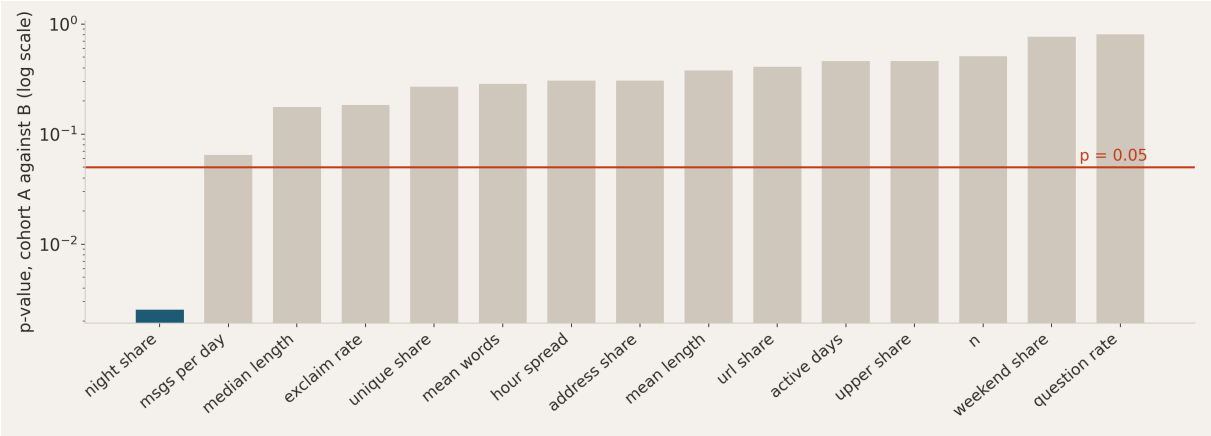


Figure 41: Fifteen measures, cohort A against cohort B. The dashed line is $p = 0.05$.

One clears the line easily: cohort A does 13.5% of its posting between midnight and five in the morning, cohort B 6.7%, $p = 0.003$, with more than two hundred people per group. That sounds publishable, and a story arrives at once: night owls, or another time zone. Notice how fast the story came. Now look at how the cohorts were made: with a random

permutation. The two groups are the same population, split at random. There is no cohort. Everything in this plot was produced by nothing at all.

3.4.2 How often do you find something when there is nothing?

So measure it instead of being surprised. Shuffle the cohort labels 300 times and hunt every time over the fifteen measures.

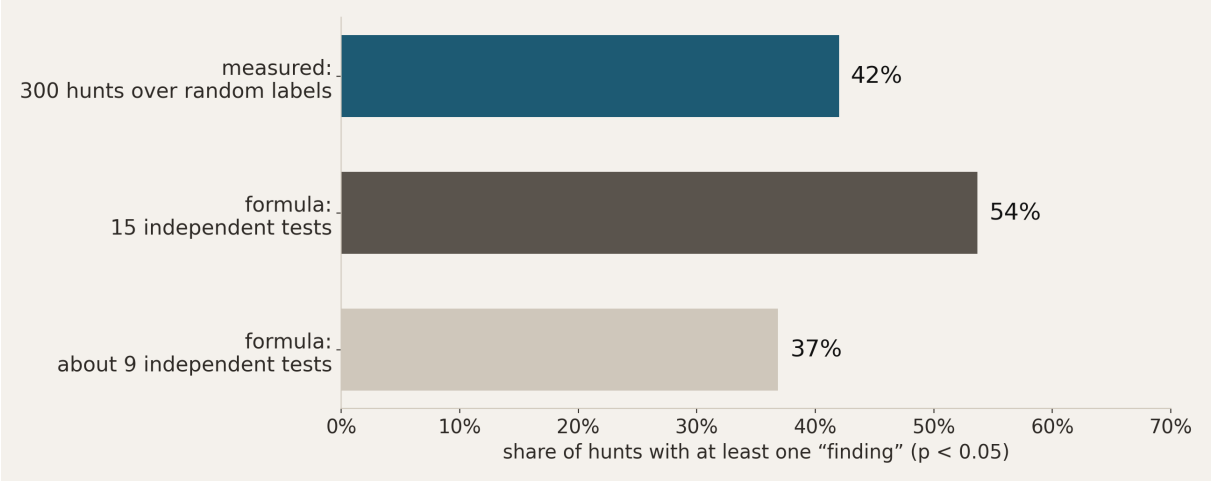


Figure 42: 300 hunts over random labels: how many of the fifteen measures cross 0.05.

In 42% of the hunts at least one measure crosses 0.05; some find two or three. That is the baseline: before asking whether a finding is real, ask how often you would find something if there were nothing to find. If the answer is almost half the time, finding something is not evidence.

The arithmetic is worth knowing. A p-value below 0.05 means a false alarm one time in twenty per test. “At least one alarm” is hard to count; “no alarm at all” is one product. Fifteen independent tests all stay quiet

$$0.95^{15} = 0.46$$

of the time, so at least one alarm 54%. The measured 42% is lower because the measures overlap: mean length, median length and mean number of words are three names for message size. The fifteen behave like about nine independent measures, and $1-0.95^{9} = 0.37$.

This is the dice from lesson 4 again. If I say beforehand that you will roll six sixes and you do, that means something. If I let a hundred thousand people roll and one of them rolls six sixes, I should expect it. With few messages and few people, as in your own chats, the chance of a lucky find is large. Ask beforehand: how many things am I testing, and what would I find by chance alone?

3.4.3 A real pattern predicts people it hasn’t seen

The strongest defence is to split the data: find the pattern in one part, then predict the other. Here with fresh random cohorts, so again there is nothing to find.



Figure 43: Left: the hunt in half of the people. Right: the same comparison in the other half.

Hunt over the fifteen measures in half of the people: cohort A writes clearly shorter messages, $p = 0.005$, a difference you would happily report. Then look at the other half, people the hunt never touched: the gap is gone, $p = 0.62$. Over 200 random splits, the hunt in the first half finds something below 0.05 in 40% of the splits, and the new people confirm it 2% of the time. A real pattern predicts new data; an accident of the search does not. With enough data you can make this a habit: use 80% of your data to search, as freely as you like, knowing that some of what you find is nonsense, and keep 20% apart to see what survives.

One caution: the held-out part has to consist of new units of the claim. Splitting the same people into an early and a late period is not enough, because the same people carry the same accidents into both periods; in this channel such a time split confirmed random cohort differences about one time in five. And a finding belongs with the search that produced it: say how many things you looked at.

3.4.4 Three legs make a finding believable

If the hunt were the whole story, the conclusion would be that nothing is ever real, which is as bad as believing everything. So what would make you believe a finding? Three legs, and statistics gives you at most one of them.

1. **A model.** A statistical model that fits: enough observations at the level of the claim, an effect bigger than chance. It can be evidence, if used well, but even with $p < 0.05$ it can still be chance.
2. **A mechanism.** A reason it would be true, that you could have named before looking. No data gives you this.
3. **Replication.** It shows up where you didn't look: another period, another group, another way of measuring.

	Survives testing	Fails testing
Mechanism	Report it, then look for replication.	"Plausible, unproven." A finished result: say what would settle it. Or the test was wrong.
No mechanism	Don't conclude yet. A fluke, or a variable you haven't named.	Drop it.

Table 3: Mechanism against testing.

The two cells on the diagonal are obvious. The other two are the lesson. A mechanism that fails testing is a finished result if you say what would settle it, and sometimes it means you tested the wrong way; the next example is one. A significant result without a mechanism is not a conclusion but a question: a fluke, or a confounder you haven't named, and the confounder is usually the more interesting finding. A mechanism also has a risk of its own. A surprising claim, against intuition, gets critical eyes. A claim that confirms what people already thought is accepted too quickly, and then the digging stops. The COVID switch, by the way: a model that fits, a mechanism yes, replication not yet.

3.4.5 The comparison you choose is part of the claim

The first claim through the grid: Ubuntu release days are busier. The mechanism is strong. Ubuntu releases on a Thursday, twice a year, on dates fixed in advance by a calendar the chat has no say in, and people in the channel have a reason to show up: have you seen the new version?

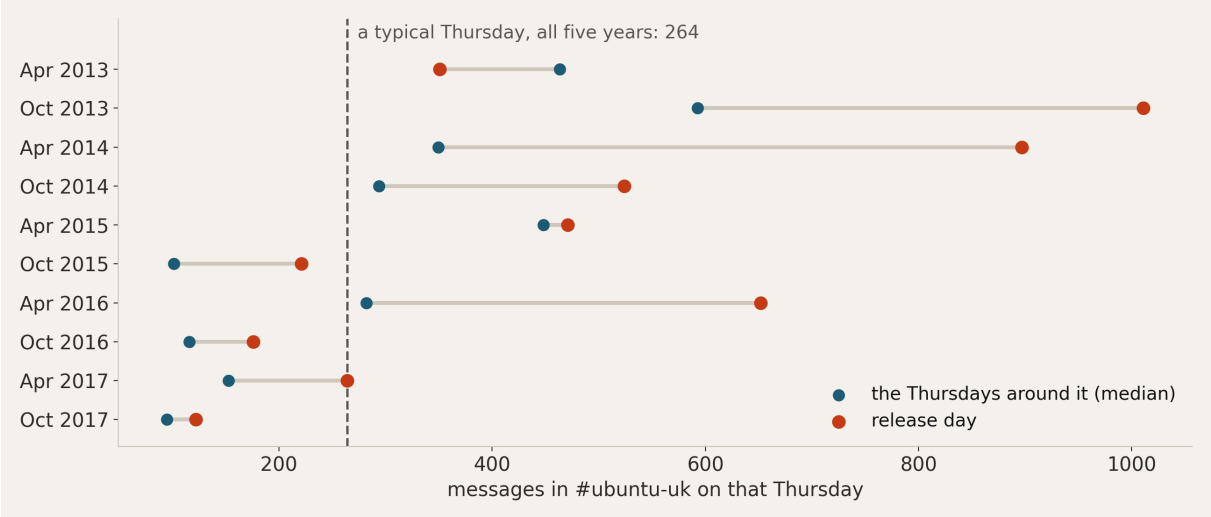


Figure 44: Ten release days against a typical Thursday of all five years (dashed), and against the Thursdays around each release.

The obvious test compares each release day with a typical Thursday. Over all five years, the median Thursday has 264 messages, and only six of the ten releases beat it: a coin flip, which would kill the claim. It shouldn't, because the channel lost most of its traffic over these years. A typical Thursday in 2013 and one in 2017 are different numbers, and the late releases are compared with a median that mostly comes from the busy early

years. The trend is a confounder for the event. Compare each release with the Thursdays within four weeks around it instead: nine of ten are busier, by 1.7 times on average, and a sign test gives $p = 0.02$. The sign test only asks on which side of typical each day falls, a question ten noisy days can answer. Same data; only "typical" changed. A correct mechanism can fail on a badly chosen reference.

3.4.6 Test at the level of your claim

The claim: people write longer messages in the weekend. Per message, the answer looks decisive: weekend messages are 49.2 characters on average against 48.2 on weekdays, with $p = 10^{-9}$ over 510,833 messages.

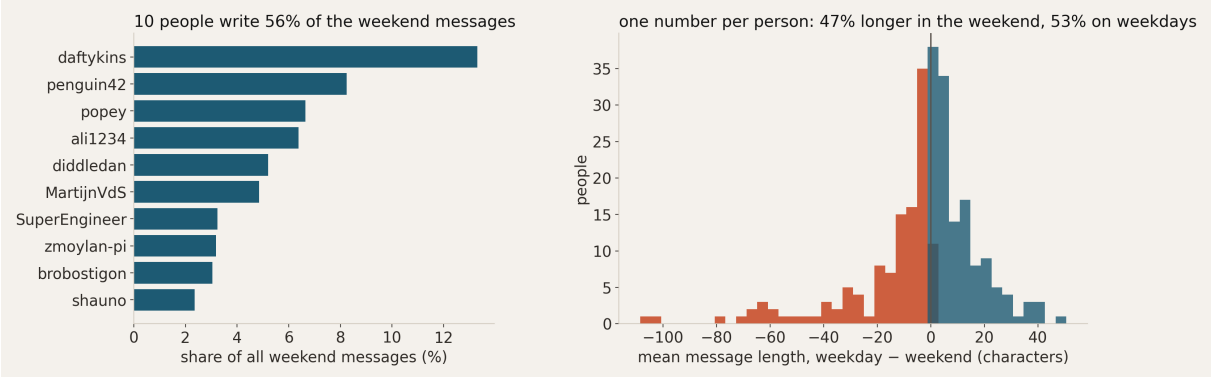


Figure 45: Left: ten people write 56% of the weekend messages. Right: weekday minus weekend message length, one number per person.

But a message average is dominated by whoever writes the most. Ten people write 56% of all weekend messages, daftykins alone about 13%, and who posts in the weekend is not who posts on weekdays. If every weekend poster wrote exactly as they do on weekdays, the weekend average would already be 48.9 instead of 48.2: most of the difference is who is talking, not how people talk. Bill Gates walks into a bar, and on average everyone is a millionaire. Nobody in the bar got richer.

Ask the question at the level of the claim, one number per person, and it splits: 47% write longer in the weekend, 53% on weekdays (perhaps during office hours, out of boredom), paired $p = 0.06$. Outside the small group of heavy posters, there is no effect at all. Report it that way: no general difference between weekday and weekend writing for the 259 people with enough messages; the per-message difference comes from who posts when. Your claim is about a few people, not about half a million messages.

This is the statistician's nightmare: dig deep enough into your data and you can always find a subgroup that makes your effect disappear, or turn around. That is not a reason not to dig. It is a reason to dig: don't stay at the surface, look for the subgroups, and state your claim at the level where it holds.

Again, concretely. Spurious correlations should be expected. With enough features and enough observations, you always find something; the right question is not "did I find something?" but "how often would I think I found something if there were nothing?" Shuffle the labels to measure it. Then the three legs: a model that beats chance at the

level of the claim, a mechanism you could have named beforehand, and replication where you didn't look. Applied to the COVID switch: it fits and has a mechanism, but one country on one calendar cannot separate vaccination from everything else that happened that spring.

3.5 What can a correlation hide?

Pearson's r says how two columns move together, scaled between -1 and 1, and the correlation matrix is the first plot everyone makes. But a correlation on its own is not reliable. It can look strong and mean nothing, or look weak and hide a real relation, and the number alone doesn't tell you which. Two habits help. Always ask which groups are mixed in your data: a subgroup you didn't separate can turn a correlation around. And with many features, use a trick: a regression with a penalty on its weights finds the useful features better than correlations do.

3.5.1 The same penguins, twice: the line turns around

The penguins from lesson 4, 324 birds with complete measurements. Bill length against bill depth: is a longer bill also a narrower one, or is a big bill usually a thicker one? Think about what you would expect before you look.

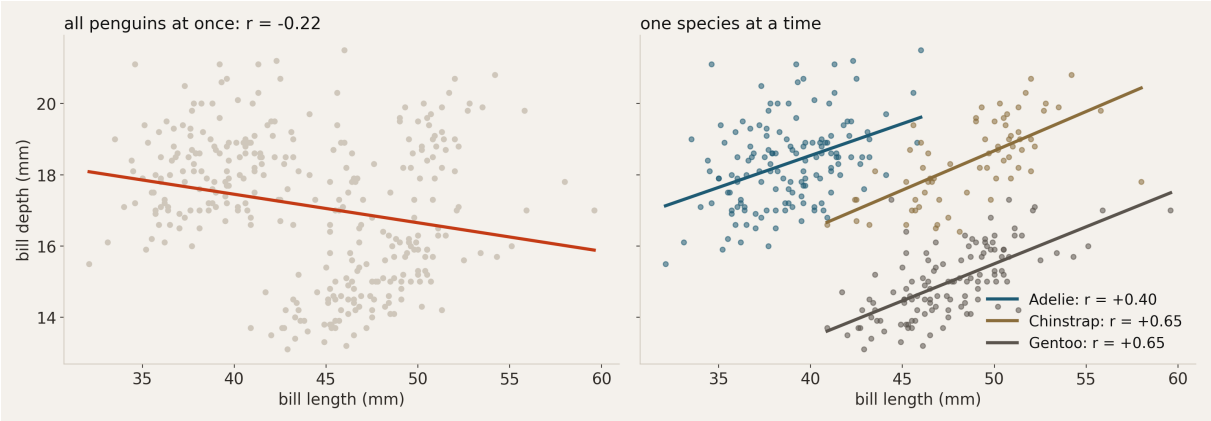


Figure 46: Bill length against bill depth. Left: all penguins at once. Right: one species at a time.

All penguins at once: longer bills are shallower, $r = -0.22$. One species at a time: in every species a longer bill is deeper, +0.40 for Adelie, +0.65 for Chinstrap and +0.65 for Gentoo. Nothing about the birds changed. Gentoos have long, shallow bills and Adelies short, deep ones, which lays the three clouds along a downward diagonal; the pooled line measures the difference between species. This is Simpson's paradox, which lesson 2 met as a table of admissions. As a scatter you can see the mechanism: three clouds, and one line drawn through them. Note how easily this happens: a correlation matrix is computed on the numeric columns only, so species, island and sex are dropped before a single correlation is computed. Look at a few bills and you discover there are three kinds of penguin.

3.5.2 Seven of fifteen correlations change sign

Now all fifteen pairs of measurements, pooled and within species. "Within species" means: subtract each species' own mean from both columns, then correlate. That removes the differences between species and keeps the differences inside them.

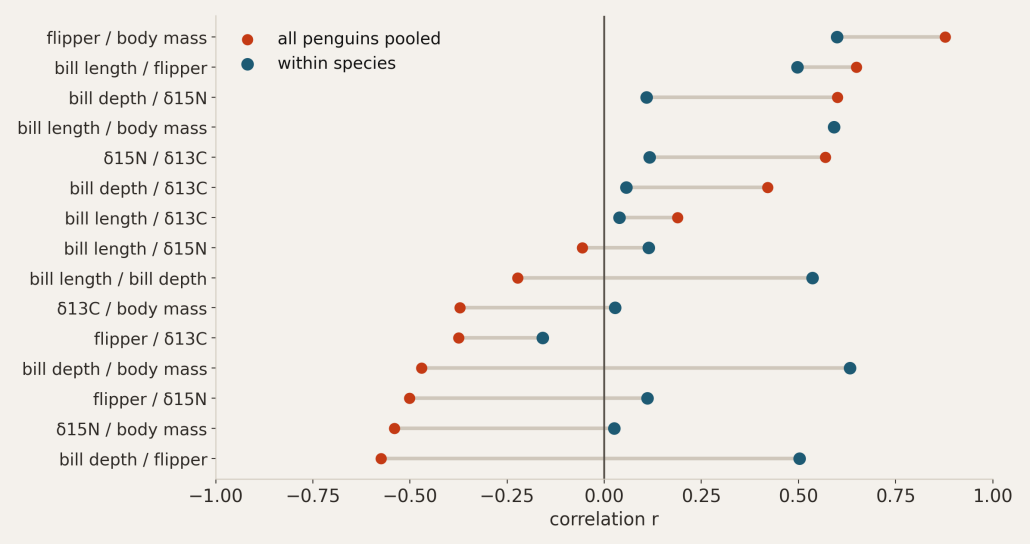


Figure 47: Fifteen correlations. Red: all penguins together. Petrol: within species.

Seven of fifteen change sign. Almost half of the claims you would make turn around when you go one level deeper. Two are worth pointing at. Nitrogen-15 and body mass: −0.54 pooled, a number you would report; within species +0.03, nothing. The correlation was not weakened by controlling for species; it was made entirely of species. δ^{15}N rises with what an animal eats, and the species eat differently and happen to differ in size. Bill length and body mass: +0.59 pooled and +0.59 within. A bigger bird has a bigger bill, in every group. So the lesson is not "distrust all correlations".

There are three ways to handle a confounder, in increasing effort: report per group, take off the group mean first, or put the variable in a model. And then the part no method fixes: the confounder has to be in your data. If nobody had recorded the species, every number would be just as wrong, and nothing would tell you.

3.5.3 A penalty on the weights: what L1 and L2 do

The trick I promised. I first used it on a dataset of 150 people with two kinds of cancer, and for each person 10,000 DNA markers. The question was whether any of the markers says something about the kind of cancer. If you know a little statistics, you know that you will find patterns in 10,000 features, guaranteed, just as in the 300 hunts over the chat channel. This is the recipe for overfitting, and a correlation matrix will hand you noise. The solution was *regularisation*.

First, how do you measure how wrong a model is? Suppose I am a model guessing your height. I guess 4 metres for one person, far too high, and 20 centimetres for the next, far too low. On average my error is close to zero, because the two errors cancel. So you square each error before averaging: both become positive, and big errors count much more

than small ones. That is the *mean squared error*:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

with y_i the real value and $\hat{y}_i$ the prediction.

A linear regression gives every feature x_j a weight w_j, and picks the weights with the smallest error. With 100 features and 500 rows it will happily give every feature some weight, also the useless ones. A penalty adds a cost for large weights to the error, and α sets how strict it is:

$$\text{L1: MSE} + \alpha \sum_j |w_j| \qquad\qquad \text{L2: MSE} + \alpha \sum_j w_j^2.$$

Of two models that predict equally well, the one with the smaller weights wins.

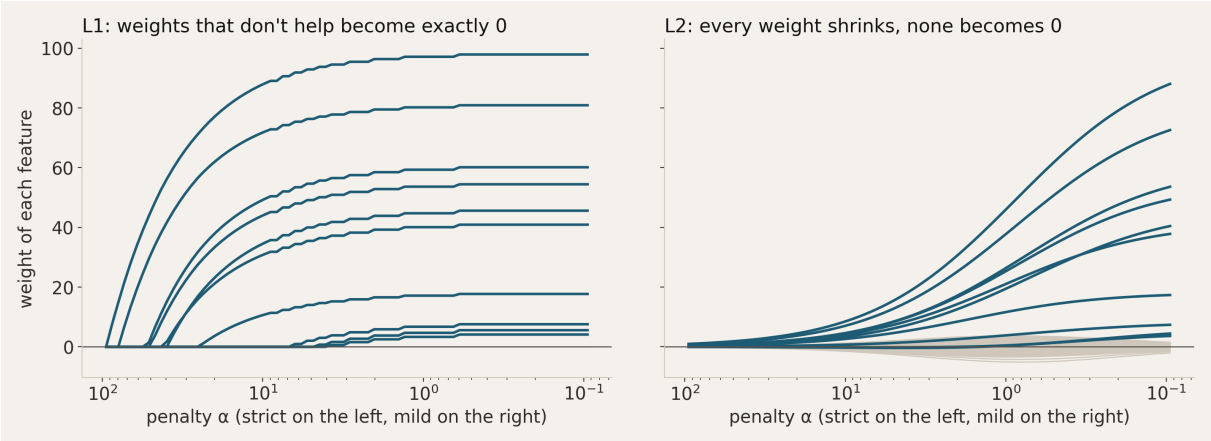


Figure 48: The weights of the same simulated data as the penalty goes from strict (left) to mild (right). Left panel: L1. Right panel: L2.

L1 (also called *lasso*) pushes the weights of features that don't help to exactly zero, and as the penalty grows, the useful ones drop out one by one, the least useful first. L2 (*ridge*) shrinks every weight smoothly, but keeps them all; none ever becomes exactly zero. That is why L1 can select features: what survives the penalty is what matters. *Elastic net* mixes the two, and its `l1_ratio` says how much is L1: 1 is pure L1, 0 pure L2.

Two analogies from class. A meeting with a specialist in the room: on their subject, their vote may count more than the others', but not ten times as much, because then they can overrule the whole group by saying no. How much extra weight still gives good decisions? That is regularisation. Or my daughter of fourteen, going shopping for Sinterklaas presents: give her a budget, and she has to make choices, hopefully relevant ones. Squeeze the budget, and only what matters gets money; the rest goes to zero.

3.5.4 A penalty on the weights finds the useful features

A simulated dataset: 500 rows and 100 features, of which ten really matter and ninety are pure noise. Can you find the ten?

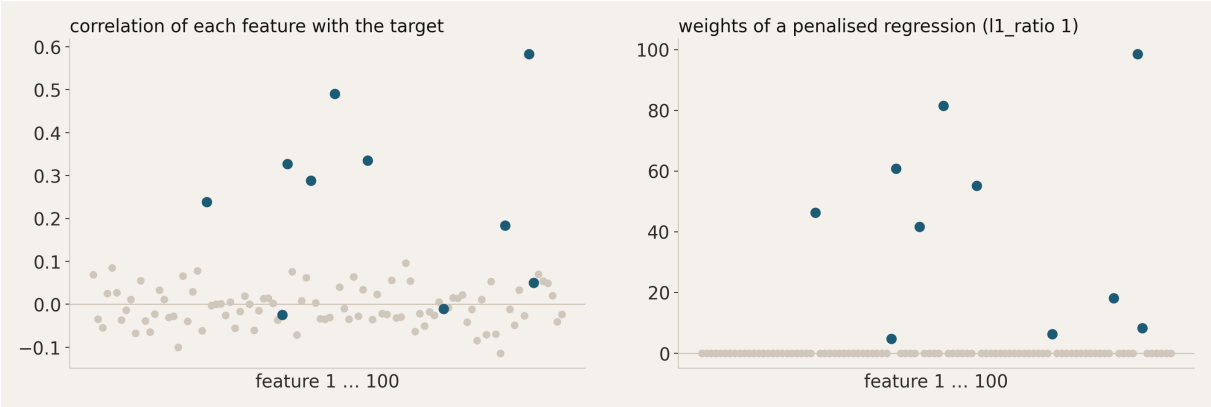


Figure 49: Left: the correlation of each feature with the target. Right: the weights of a penalised regression. Petrol: the ten largest weights.

Plot each feature's correlation with the target, and six stand out; the rest looks like noise, and you can see how noisy it is. Now misuse a linear regression on purpose: the goal is not the model, it is the weights. A grid search over the strictness α and the mix picks pure L1, `l1_ratio = 1`. Almost every weight is exactly zero, and the real features pop out. Three of the ten largest weights belong to features that have almost no correlation with the target on their own: a feature can matter in combination with the others while its correlation alone is near zero. The correlation matrix is well known; it is not very reliable. (The grid search is a preview: hyperparameters are next semester's topic.)

Again, concretely. Pooled, longer bills are shallower. Within every species they are deeper. Same birds, opposite sentence. Before you report a correlation, ask what else is different between the groups you compare. The confounder has to be in your data: species was, and one `groupby` fixed it. If nobody had recorded it, nothing would tell you. With many features, let a penalised regression do the selecting, and read its weights.

3.6 Can you recognise someone by how they type?

The last question takes one claim all the way through: *people are more identifiable by how they write than by what they write about.* The mechanism is plausible, and it predicts something specific: habits should be stable, while subjects drift. Three warnings go with it. Accuracy only tells you there is something to look at; it is not the finding. The weights tell you what the model learned: a habit, or just a topic. And count the right thing: 218,670 messages, but eight people. For a claim about people, n is eight. A model that works can still be right for the wrong reason; a stopped clock gives the right time twice a day.

3.6.1 47% right, but why?

A model reads one message from the eight most active people in the channel and guesses who wrote it: tf-idf over all words and a logistic regression. It is right 47% of the time. What is the baseline? Not one in eight, because some people write much more than others. The simplest honest model always guesses the busiest person, and is right 22% of the time. So 47% is twice the baseline: a gate passed, not a destination. Half the messages still go to the wrong person. What did the model learn? That is in its weights.

Person	Words the model leans on	What that is
ali1234	n900, pidgin, qml, mythtv	subject matter
MartijnVdS	dimpy, mungbean, neuro	other people's nicknames
zmoylan-pi	ireland, dublin	where he lives
daftykins	0o, xd, hrmm	typing
popey	dont, didnt, thats	typing: no apostrophes

Table 4: The strongest words per person in the one-message model.

Read them, and they are not all the same kind of thing. Devices and software are a topic. Other people's nicknames are not a writing habit but a social position: who you talk to. Ireland and Dublin are where someone lives. Only the last two are about how someone types. The rest is a topic detector wearing a stylometry costume, and a topic detector fails the moment someone changes the subject, stops talking to Dimpy, or moves house. That is exactly the situation an authorship model is for. When a model works, read its weights and ask what each one really is.

3.6.2 Three ways to write a smiley, and nobody mixes them

So stop using the model to find the fingerprint, and measure the habits directly. Nothing here needs machine learning: five `RegexFeature` steps and a `groupby`, the same transform you used in lesson 1 for URLs, now pointed at smileys and apostrophes.

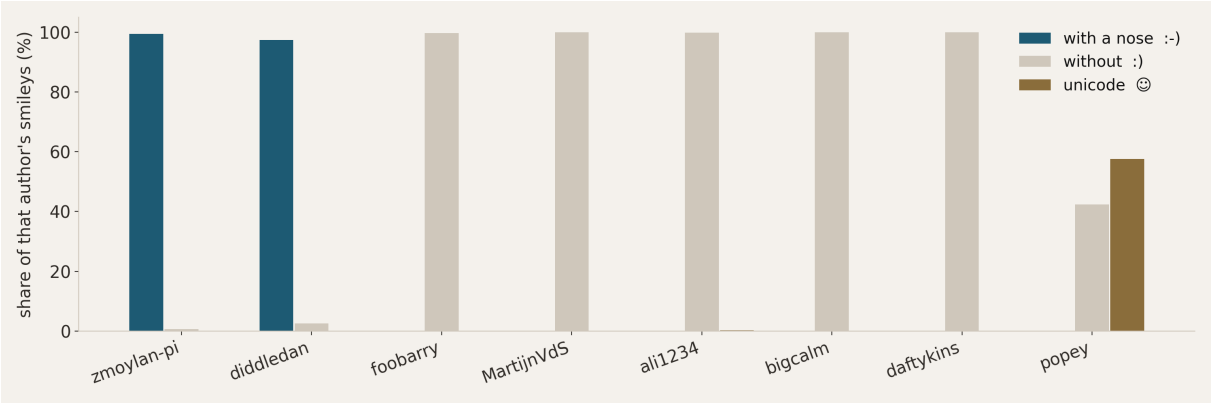


Figure 50: Smileys with a nose `:-)`, without `:)`, and the unicode smiley (U+263A), per person.

Smileys come in three dialects, and nobody mixes them. Two people use the nose almost every time; five never do; there is nobody in between. popey writes the unicode smiley, which nobody else types. Most differences between people are a matter of degree. This one is a choice, made once and kept for years, by people who read each other's messages every day. Apostrophes, capitals, addressing someone by name and posting at night look the same: a couple of people at one extreme, everyone else at the other. There is no typical author in this channel.

3.6.3 Fifty messages at a time: ten habits do what 17,918 words do

If the habits are this decisive, why only 47%? Because of the unit of analysis. Three quarters of all messages carry no habit at all: “ok”, “thanks”, “brb”. Nobody can tell from “ok” who wrote it, so a model reading one message usually has nothing to go on; 47% is what a strong signal looks like when it is sparse. So stop asking about one message: pool fifty messages per person into a block. That needs a small pipeline step of our own, `BlockMessages`, because what counts as one row is a decision worth naming.

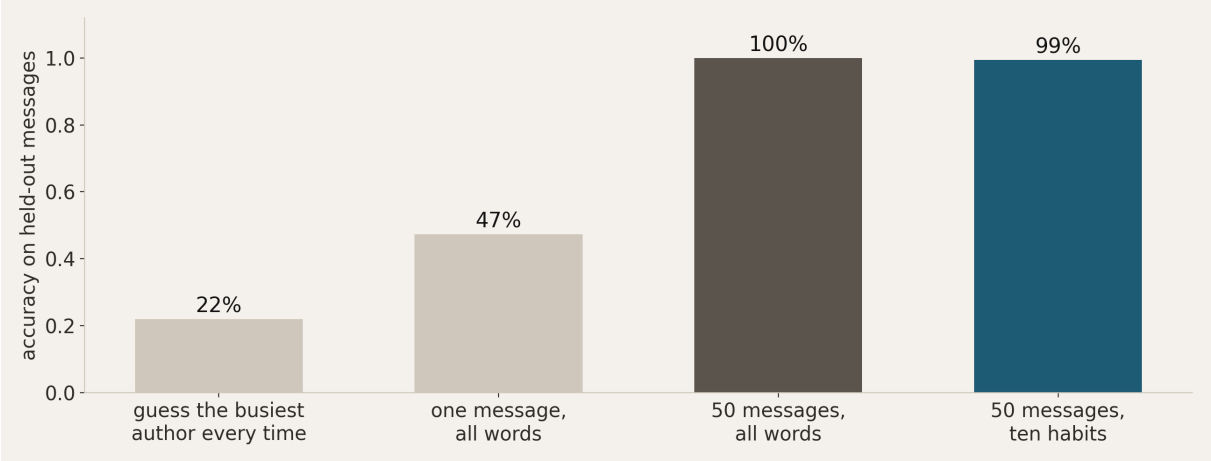


Figure 51: Accuracy on held-out messages, for four ways of guessing the author.

Then race two models on the same 4,368 blocks. One knows every word, 17,918 of them, and gets 100%. The other knows ten habits and nothing else, and gets 99%. The one with no topics, no nicknames and no place names is within a point of the one that knows everything. And its ten weights can be read aloud: MartijnVdS is addressing people, daftykins is noseless smileys at night, foobarry is recognised by never posting at night.

Which is the better model? In class someone defended the big one: test it on a held-out set, a thousand times if you like, and if it scores 100% against 99%, take the 100%. I don’t think a train-test split saves it, because the data changes over time, and the student named the reason themselves: other words, and other people who also mention Ireland one day. Someone who moves or stops talking to Dimpy breaks their own feature, and for Dublin and Ireland that is to be expected, not exceptional. The 17,918 words bring an enormous amount of noise into the model for one extra percent: they score 100% today, and after a year in production the model drifts away, and then the verdict is “your model doesn’t work”. I can’t prove that here; it is an intuition. But if we made this bet a hundred times, I would put my money on the ten habits every time, and you would need a lot more analysis to convince me otherwise. Sometimes there is no simple model, and then a complex one is very useful and you accept its noise. When there is a choice between ten features at 99% and 17,918 at 100%, take the ten. Another suggestion from the room fits the whole lesson: start with two habits and add them one at a time, instead of starting at the maximum.

3.6.4 The same eight people, four years later

The third leg: does it hold where nobody looked? Fit nothing; measure the same habits in 2013–2014 and again in 2016–2017.

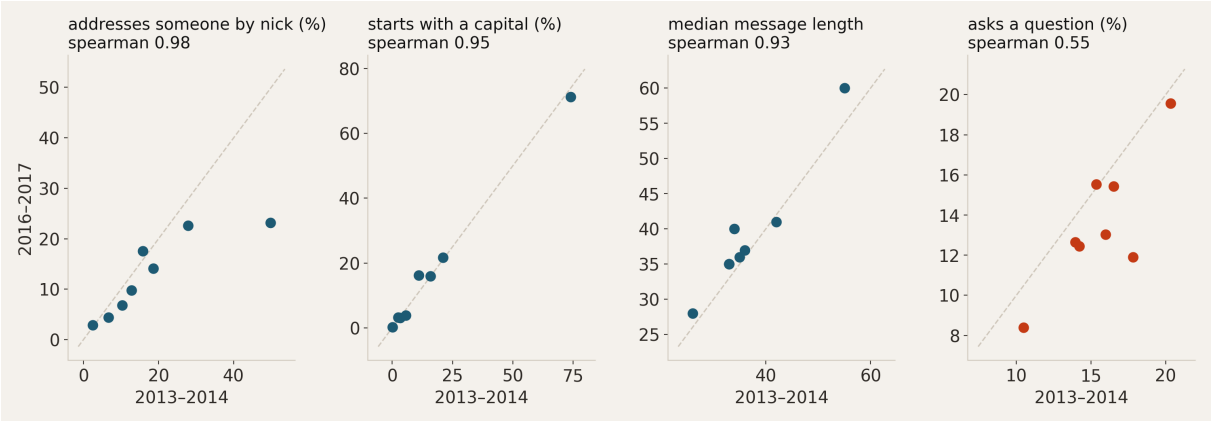


Figure 52: Each dot is one person: a habit in 2013–2014 against 2016–2017. On the diagonal means unchanged.

Addressing people by name, starting with a capital, message length: the order of the eight hardly changes, with Spearman correlations of 0.93 to 0.98 after a four-year gap. If you started 75% of your messages with a capital in 2013, you still do in 2016. Two people became a little more long-winded, and one addresses people less than before, but the habits stay. That is replication, and it explains why ten habits are enough for 99%. Asking questions does move (Spearman 0.55), and drops over time: whether you ask questions depends on whether you have a problem right now. That is not a weakness of the finding. A measure that could say no makes the others believable, because it shows the measurement could have failed.

Again, concretely. Ten habits recognise eight people, four years apart. But it is eight people. Through the grid of question four, this claim lands in the best cell: a model that fits, a mechanism, replication. How strong is eight out of eight? If staying stable were a coin flip, all eight would stay stable with probability $(1/2)^8 = 1/256$: a fairly strong signal. But 218,670 messages don't make n bigger. The claim is about people, so $n = 8$, and large data does not fix a small n: a hundred people with fifty thousand messages would say much more. These habits might be specific to Linux users; children, teenagers and twenty-five-year-olds, or writers in the UK, Ukraine, South Africa or on a Chinese keyboard, might behave differently. The fix is cheap: the habits run on anyone with thirty messages, so recompute the stability for everyone with enough history in both periods. The accuracy was a gate; the weights were the finding. Reported honestly: among the eight most active people, ten habits classify blocks of fifty messages at 99%, stable over four years. Others: untested.

3.7 The six questions, answered

Question	Answer
What is a model?	A simplification you choose. Not true, not objective, useful for what it leaves out.
Why keep a model simple?	You can only understand simple shapes, and a model that fits anything tells you nothing.
How do you build a model?	Look, think, add one step or one term. Then read the residual.
When should you believe a pattern?	When it beats chance, has a mechanism, and shows up where you didn't look.
What can a correlation hide?	The variable you left out. It has to be in your data.
Can you recognise someone by how they type?	These eight people, yes. Large data does not fix a small n.

Notebooks 05.1 to 05.5 work through the questions: 05.1 the line as a claim, the hunt and the three legs; 05.2 correlation and Simpson's paradox; 05.3 the COVID model as a question you answer yourself, and 05.4 the worked answer and the script; 05.5 how people type. In your own chat: pick a habit you can write as a regex, measure it per person, and report n at the level of your claim.

4 Lesson 6: Many dimensions

Until now every dataset had a handful of columns, few enough to plot against each other. This lesson's data does not: a handwritten digit is 784 numbers, a sentence from a language model 384, a piece of text counted by its three-letter sequences tens of thousands. Nobody can imagine a space with that many dimensions, and that is exactly the point. The assignment for this week is to take your own data up into such a space, at least twenty dimensions or so, and to bring something back down from it that shows a pattern.

It is probably the most complicated lesson of the course, because we are trying to visualise something that cannot be seen. And the plots people hand in for it look much alike: a cloud of dots on two axes called PC1 and PC2. For one student that plot is excellent, for another it is a fail. The difference is not in the plot. It is in the process around it: what you are actually visualising, and what happened on the way.

The lesson is organised around six questions:

1. **What** does PCA show you? The directions in which your data varies most, in the units you gave it.
2. **When** is a flat picture not enough? When the data lies on a curved surface. t-SNE unrolls it, at a price.
3. **How** does text become a vector? Count which three-letter sequences it contains. Then check what did the work.
4. **Why** use a model somebody else trained? It is expensive to run and cheap to reuse. Pay once, then train on a laptop.
5. What is **one row**, when a row is "haha"? A picture can only show what the rows contain.
6. Why did **k-means** find three clusters? Because you asked for three.

The first two are about looking at data with many columns: PCA and t-SNE are two different tools, and you have to know what each one gives up for the other. The next two are about getting into such a space: by counting, or with a model somebody else trained. The last two are about the mistakes I see most in your plots: a row that is too small to carry what you ask of it, and a clustering method that always finds what you ask for.

4.1 What does PCA show you?

PCA, principal component analysis, is the oldest way to look at data with many columns, and a statistical method with many pitfalls. What goes wrong most often is that people know the reflex ("many dimensions, so PCA"), run it, and then either see nothing and don't know why, or see a pattern that isn't reliable.

PCA works in two steps. First it fits new axes. It looks for a line through the data, the line the points lie closest to, which is also the line with the most spread along it. Then the best line at right angles to it, and so on, until there are exactly as many new axes as you had columns. Fifty columns in, fifty axes out. So PCA on its own is not dimensionality reduction: nothing is lost yet, it is the same data described along different axes, and those axes describe it more efficiently. Only then do you reduce, by dropping the last axes, the

ones that carry the least spread. That always loses some information, and PCA tells you exactly how much.

Why does that work at all? Because columns often say the same thing. Temperature and hours of sunshine go up together; humidity and millimetres of rain too. Describing a point along two such axes means turning two knobs at the same time, both by about the same amount. One tilted axis is one knob that does the same. What is left over, the small distance from each point to that axis, is what you give up.

4.1.1 The line with the most spread loses the least

Reducing two columns to one means choosing a line and dropping every point onto it (figure 53). Each point splits into two pieces: where it lands along the line, which PCA keeps, and how far it sits from the line, which it throws away. By Pythagoras, the squared distance of a point to the centre equals the squared position along the line plus the squared distance to the line, and the left side does not depend on which line you chose. So summed over all points, more spread along the line means exactly less distance to the line. The line with the most spread is the projection that loses the least. That is the first principal component.

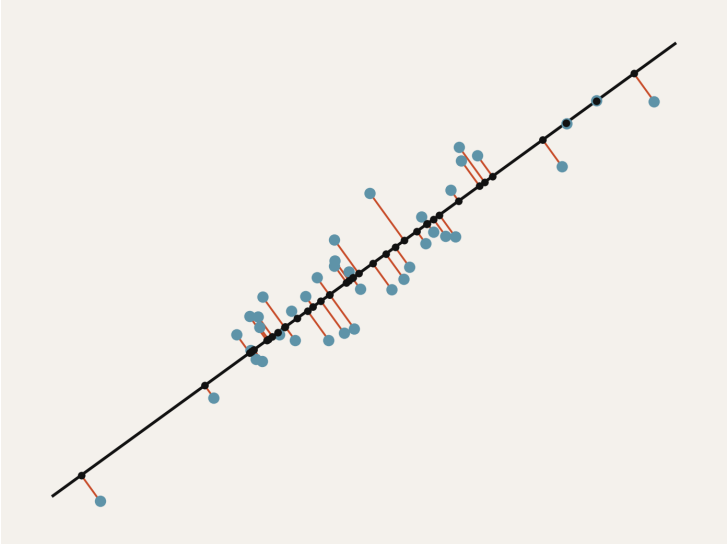


Figure 53: 40 points and one line. Black: every point dropped onto the line. Vermilion: the distance to the line, which is thrown away.

Nothing here is about labels or groups. PCA only looks at spread.

4.1.2 PCA finds new axes, and you choose how many to keep

The same idea one dimension up, on data built so the answer is known (figure 54). A hundred points on a curved strip: three columns, but the third is almost a sum of the first two, so every row lies close to one tilted plane. PCA finds three new directions, and none of them is one of the original columns: each is a mixture of all three. That is what people mean by “PCA finds new axes”.

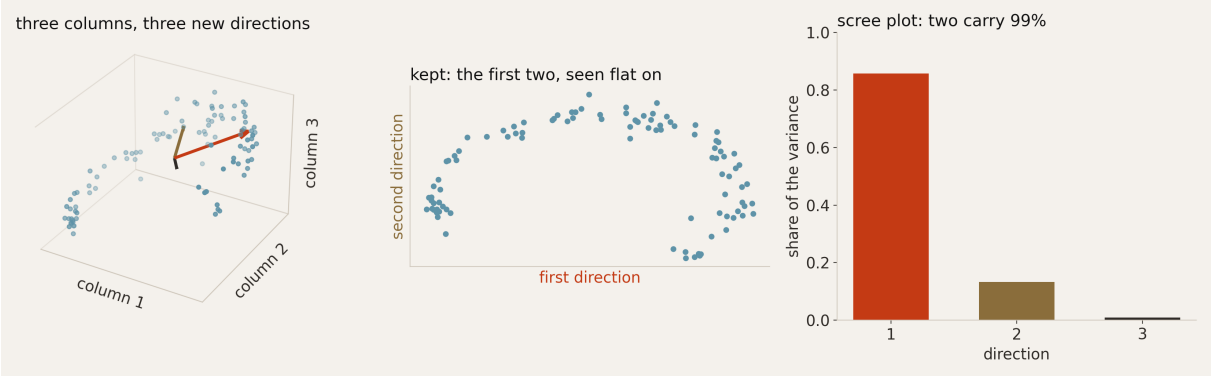


Figure 54: Three columns that are really two. Left: the data with PCA's three directions. Middle: the first two directions, seen straight on. Right: the scree plot.

The *scree plot* on the right shows how much of the spread each direction carries: 86, 13 and 1 percent. The first two carry 99%, so the flat picture in the middle is a picture of nearly all the data. The third axis does something, but very little; drop it and you lose almost nothing. In the original axes you could not drop any one of them without distorting the data much more.

What does "86% of the variance" mean? In class someone asked whether it means that 86% of the points lie on the first axis. It doesn't. It means that 86% of what differs between the points can be expressed along that direction. Think of describing food. One axis, "sweet or not", says something about food, but much food isn't sweet at all, so it describes only part of the variation. With the five basic tastes you can describe a lot more, perhaps 60% of the variation in food, and the rest needs more axes: cinnamon is not just sweet plus spicy. Sometimes the conclusion is that the data really needs all its axes. On the strip it doesn't.

4.1.3 Change the unit, and the first component changes

The first way PCA goes wrong, with the penguins from lesson 2 (figure 55). PCA maximises spread, and spread has units. Body mass is measured in grams and varies by about 800 of them; bill depth varies by 2 millimetres. So almost all the spread in this table is grams, and the first component is the grams axis: 99.99% of the variance. Not because body mass describes a penguin so well, but because it is in grams. That number in a report looks like a triumph. It is the symptom.

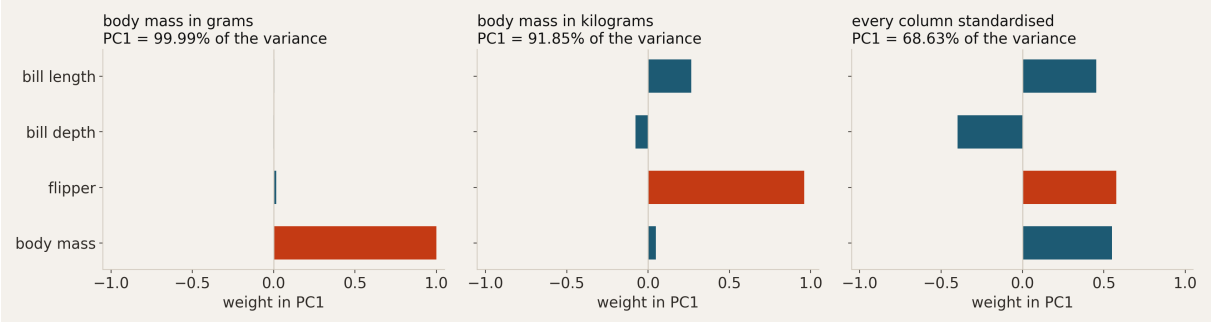


Figure 55: 333 penguins, four measurements. What the first component is made of, in grams, in kilograms, and standardised.

The clearest way to see it: divide body mass by a thousand and call it kilograms. Same birds, same information. Now the first component is flipper length. A unit decided the answer. The fix is to *standardise*: subtract each column's mean and divide by its standard deviation, so one unit of every column is one standard deviation of that column. Then the first component reads as a sentence: bill length, flipper and body mass together, so a penguin high on PC1 is a big penguin. The second component, not shown, is bill shape.

4.1.4 Standardise first, or PCA measures your units

It is not only about interpretation: the unscaled version is worse at the job (figure 56).

```
PCA().fit(StandardScaler().fit_transform(measurements))
```

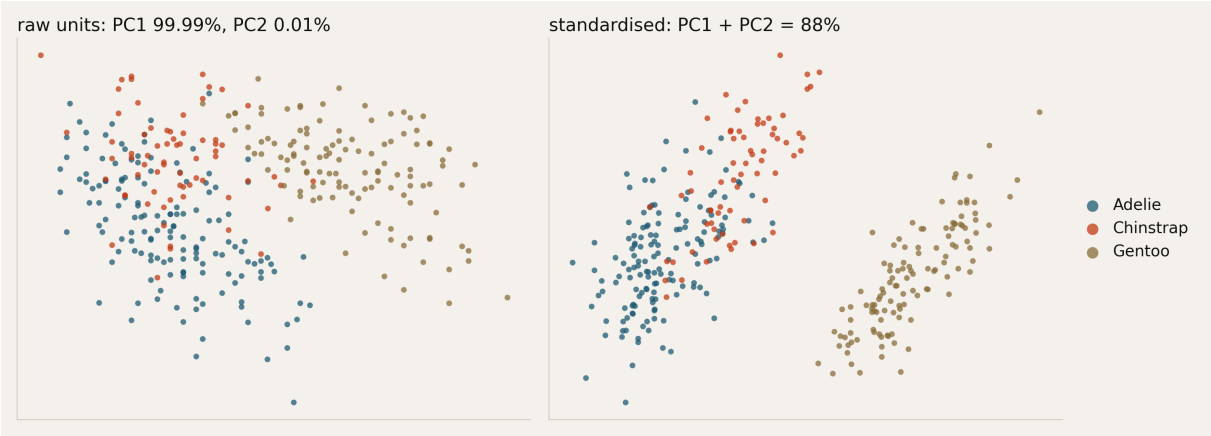


Figure 56: The penguins on two components, in raw units (left) and standardised (right).

On the left, Gentoo sits apart because they are heavy, but Adelie and Chinstrap stay mixed: what separates them is bill shape, in millimetres, which the grams drown out. On the right all three species come apart. And look at what the left picture hides. Its two axes carry 99.99% and 0.01% of the variance. Drawn honestly it would be a horizontal line; matplotlib stretched the second axis to fill the panel, so a component of 0.01% looks like half the picture. A projection always fills its frame, which is why the explained variance belongs in the title of every PCA picture you make.

Notice that the share went *down*, from 99.99% to 88%, and the picture got better. A higher number is not a better analysis; here it was the sign of a mistake. Never stare blindly at a number. The rule: standardise before PCA, unless every column is in the same unit and you want the columns that vary more to count more. Pixel intensities, all 0 to 255, are the usual exception.

Again, concretely. PCA turns many columns into a few new axes, each a mix of the old ones, ordered by how much of the spread they carry. It does exactly that and nothing more: it does not know about groups, labels or meaning. Two habits follow. Standardise, unless you have a reason not to, and write the reason down. And read the scree plot before the picture: two components of four columns can hold 88%, two components of 784 pixels hold 17%, and the reader cannot guess which. If your two components hold 6%, PCA did its job, but it could not do it well: most of your data is not in the picture. Then consider other units, or the tool of the next question.

4.2 When is a flat picture not enough?

PCA is linear. Think of it as a lamp that casts a shadow of your data on a wall. For flat objects that works well: the shadow of a flat thing is recognisable. For a complicated object there may be no angle from which the shadow shows what it is. A lot of real data is like that: it does not lie close to a flat surface, but on a curved one, and no viewpoint undoes a curve.

You can often not see that, but you can suspect it when the data has many interactions in itself. A linear relationship says more is more: more square metres, a higher price. A curved one says it is better in some cases and worse in others, depending on other factors. Body mass index is an example: a fine measure for most people, and a poor one for someone who trains, because muscle weighs more than fat.

The bet that most real data with many columns lies on a curved surface with far fewer dimensions is called the *manifold hypothesis*. The method built for that case is t-SNE. It does not look for a viewpoint; it looks for an arrangement in which each point keeps its nearest neighbours. It was developed to visualise, to solve the problems of PCA, and it pays a price for that: the distances between groups, their sizes and the gaps between them mean nothing.

4.2.1 PCA casts a shadow; t-SNE unrolls the sheet

The *swiss roll* is the standard dataset for seeing the limit of a flat shadow: a flat sheet, rolled up in three dimensions, like the cake (figure 57). Someone made it up, and it is useful precisely because you can understand it as a person. The two numbers that really describe a point are where it sits along the sheet and across it. Can we unroll the cake?

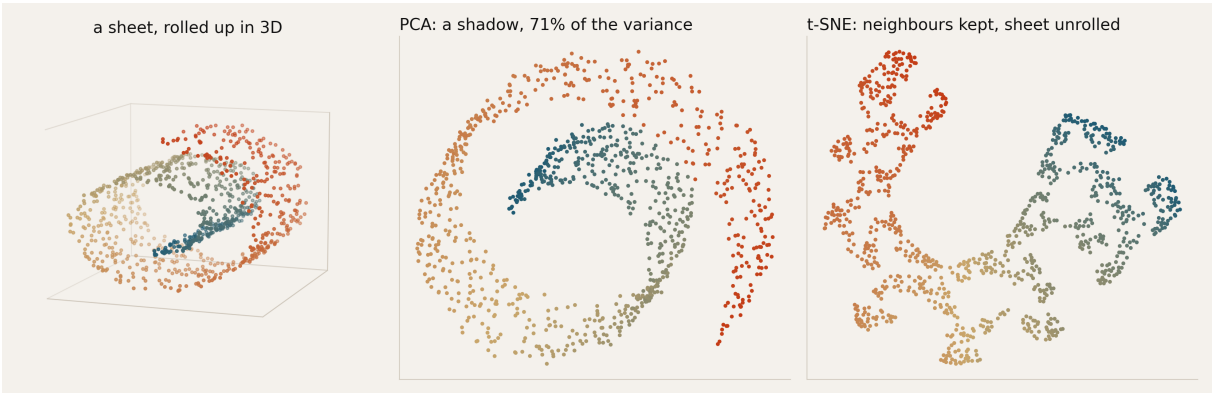


Figure 57: 1,000 points on a rolled-up sheet. The colour is the position along the sheet. Left: the data. Middle: PCA. Right: t-SNE.

PCA keeps 71% of the variance, which sounds like most of the data, and shows the roll from the side. Look at where the colours land: the two ends of the sheet, as far apart along it as two points can be, sit next to each other. The shadow is faithful to the 3D shape, and useless for the question which points are near each other on the sheet. t-SNE asks that question directly. The colour now runs from one end to the other: the sheet is more or less unrolled. The price is visible too: the sheet is torn into clumps with gaps between them, and those gaps are not in the data.

4.2.2 The manifold hypothesis: many columns, few real dimensions

A manifold is a space that is curved as a whole but looks flat close up, like the earth: round, but flat as seen from your street. The easiest way I picture it is a towel. A towel is two-dimensional, lying crumpled in a three-dimensional room. If you know how to spread it out, two numbers per point are enough, and you don't need the whole room.

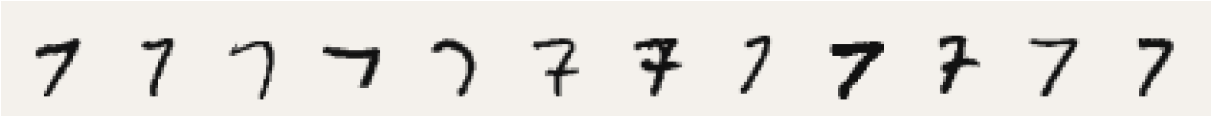


Figure 58: Handwritten sevens from MNIST.

A handwritten digit is 28×28 pixels: 784 numbers, so one point in a space with 784 dimensions (figure 58). But almost every point in that space looks like television static; the points that look like a seven are a tiny, particular region. And sevens differ in only a few ways: slant, thickness, size, a bar or none, what happens at the top. You don't need 784 axes to describe what is different about them, because pixels are not independent: they come together in strokes and directions. So the sevens lie on a curved surface with a few dimensions, and the same goes for every digit. If the bet holds, a method that follows the surface should show ten digits as ten groups, and a flat shadow should not. It is a hypothesis: it holds for images and text, it is not a law of nature, and you check it every time.

4.2.3 On 784 pixels the shadow is a smear; t-SNE finds ten groups

5,000 digits from MNIST (figure 59). The pixels are not standardised: all 784 columns share one unit, and a pixel that varies more between digits should count more.

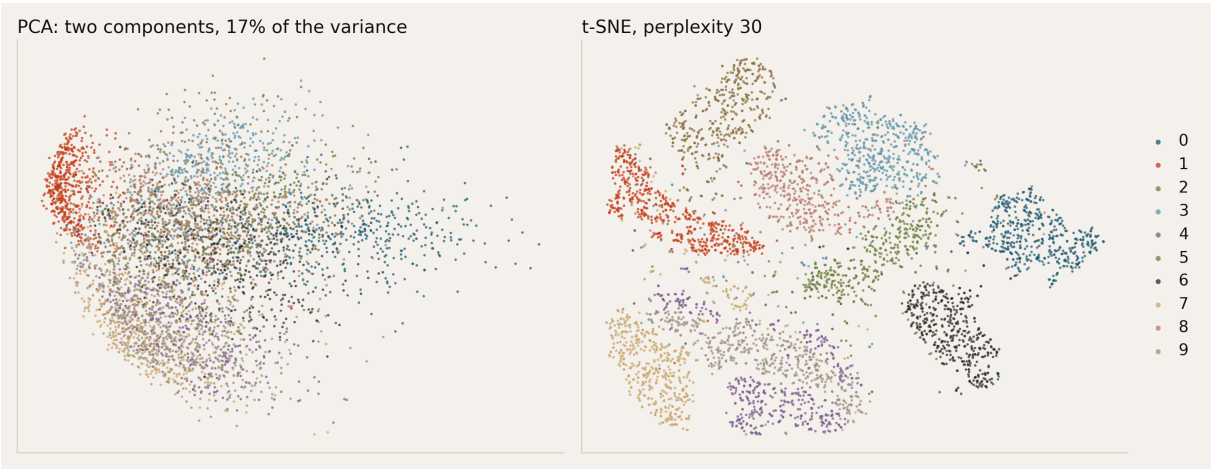


Figure 59: 5,000 handwritten digits. Left: PCA, two components hold 17% of the variance. Right: t-SNE.

PCA first. Two components carry 17% of the variance; you need 148 for 95%. The digits overlap in one smear. That is not wrong: it is an honest picture of the 17% that fits on paper. Then t-SNE with its default setting: ten groups, one per digit, and the digits that touch are the ones people confuse too, 4 and 9 most of all. So there is more structure in this

space than a flat projection can show, structure more complicated than two dimensions; the axes you would need are real. t-SNE gives you a look into a space we cannot see. The bet paid off.

4.2.4 t-SNE: the map must pick the same neighbours

How does t-SNE do it? You don't need to remember this, but it helps to have an idea of what happens inside. Take the data, N points with D numbers each, and a map, the same N points with d numbers each, usually two. In the data, t-SNE computes for every pair how likely point i is to pick point j as its neighbour, with a normal curve around i. In the map it asks the same question, with a curve that has heavier tails: the "t" in t-SNE. Then it moves the points of the map until the two sets of likelihoods match.

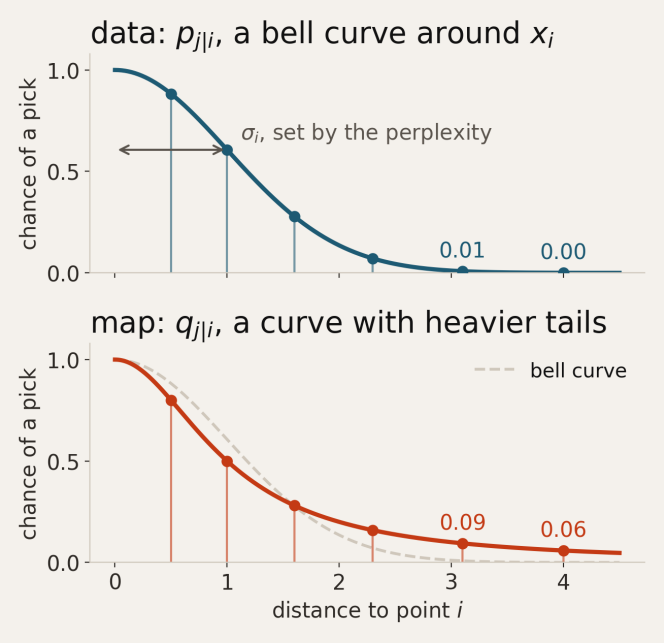


Figure 60: How likely a neighbour is picked, against its distance: a normal curve in the data, a heavier-tailed curve in the map.

Why the heavier tails? In a space with many dimensions there are so many ways to be neighbours that almost everything is somewhat near something. Compare the characters of everyone in class on one trait, introvert or extrovert, and the class splits in two. Compare them on 36 traits, and any two people have something in common: you are both conscientious, or both open to new experiences. With a thousand traits, "are these two alike in some way?" no longer says anything. The heavier tail pushes a point that is only moderately likely to be a neighbour further away in the map, which makes it harder for everything to end up together.

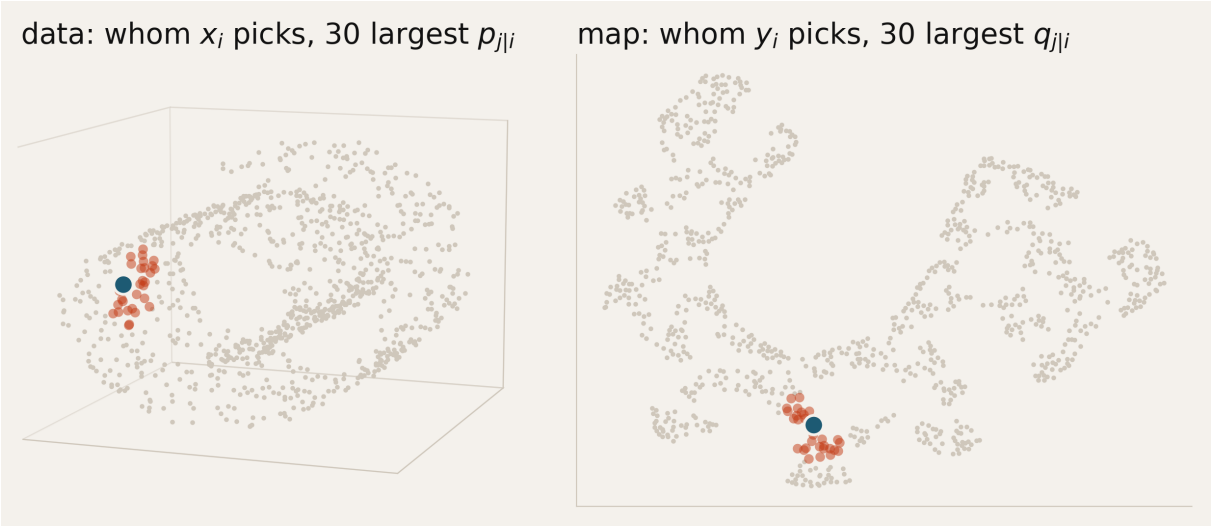


Figure 61: One point on the swiss roll. Left: its 30 likeliest neighbours in 3D. Right: its 30 likeliest neighbours in the finished map. 25 of the 30 are the same.

Figure 61 shows the result for one point on the swiss roll: of its 30 likeliest neighbours in 3D, 25 are among its 30 likeliest in the map. That agreement is what t-SNE optimised for, and it is *all* it optimised for. Where a point far away on the sheet ended up in the map was never part of the question.

4.2.5 Which points group together survives; where the groups sit does not

The parameter. *Perplexity* is roughly how many neighbours each point tries to stay near. The default is 30. You can turn it to 5 or to 100, and you get a different picture, while nothing in the data changed. It is a choice, not a display option, so run several (figure 62).

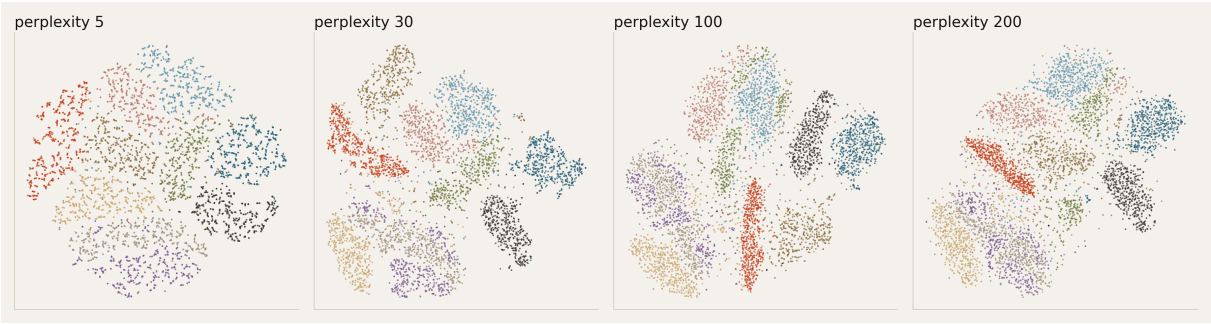


Figure 62: The same 5,000 digits at four perplexities.

Two answers, and you need both. Which points group together is stable: at every setting the digits come out as groups. To put a number on that instead of your eye, use the *silhouette*: for every point, how much closer it lies to the other points of its own group than to the nearest other group, averaged over all points, between 1 (clearly apart) and 0 (fully overlapping). It stays between 0.20 and 0.28 at every perplexity. Where the groups sit is not stable: at perplexity 5 the two digits furthest apart are 0 and 1, at 200 they are 0 and 7. How far apart two groups sit is a fact about the setting, not about handwriting.

t-SNE also contains chance: run it twice and the map differs. Fix the seed and you get the same map every time, but it remains a random process. And there is no right perplexity. If you know your data has ten groups, you can look for the setting that shows them best, but that makes it even more clearly a way of visualising, not a statistical method.

4.2.6 Preattentive processing: what do you see?

Look at figure 63 before you read on.

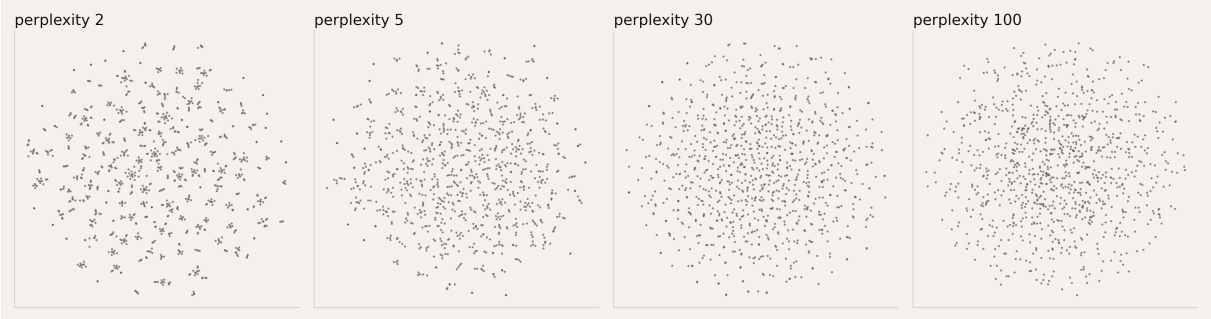


Figure 63: 1,000 points, 50 dimensions, four perplexities.

There is nothing there. Every value is an independent random draw. Still, t-SNE draws clumps at low perplexity: it was asked to keep each point near its two nearest neighbours, and in fifty dimensions of noise every point has two nearest neighbours, so it obliged. You look at it and think: I see clusters, something is happening in the data. Nothing is. Run k-means with five groups on the raw data and the silhouette is 0.016, correctly reporting that there is nothing there. Run the same k-means on the two-dimensional picture and it is around 0.31, which anywhere else you would read as decent structure. So never cluster a t-SNE picture, and never measure its distances. Distances in it are not the data's distances; anything measuring distance there is measuring the paper.

4.2.7 PCA or t-SNE? Two tools that answer different questions

	PCA	**t-SNE**
keeps	the overall shape: large distances, main directions	neighbourhoods: who is close to whom
loses	curves: it can only flatten	distances between groups, their sizes, the gaps
use it to	see the main directions of spread, and what they are made of	see whether points group together
report	the share of the variance in the picture	the perplexity, and one other you tried
never	read two components as the whole data	cluster the picture, or read its distances

Table 5: PCA flattens; t-SNE unrolls.

The two "never" rows are the mistakes I see in projects every year. In practice the two are often used together: PCA first to bring hundreds of dimensions down to fifty, then t-SNE on those fifty.

What can you still say with a t-SNE picture, if not where the groups sit? Return to the treehouse. You lived with a few friends in a treehouse, where you were neighbours, and now everyone has to move to the ground. The whole organisation is different: up there you lived above each other. Perplexity is roughly how surprised you would be to find you are still neighbours. If you see a group of friends that set up camp together again, that is fairly reliable information: they were probably a group in the tree too. Where their camp is, and how far it is from the next camp, says nothing about the tree. And you can check it: if you know who was a group in the tree, give them a colour and see whether the colours stay together in the map. With MNIST that is what the labels do: one route gives the colour, the other the place, and they agree.

A student asked why we learn this at all; is it a preparation for machine learning? It is data analysis. If you can only analyse data in two or three dimensions, you are limited, because most data has more. This course went slowly: comparing categories was one dimension, time was two with one of them fixed, a distribution is one dimension seen through a lens, and lesson 5 was explicitly two. A lot of data has many dimensions, and then PCA and t-SNE are the tools you will meet. If you use them without knowing what they give up, you lose your way and report clusters that aren't there. In deep learning you will work almost only with vectors of many dimensions, and rarely bring them back to two, because too much is lost. Doing it once with data you know, your own chat, is good preparation.

Again, concretely. PCA flattens; t-SNE unrolls, and forgets distances. Data on a curved surface, and images and text usually are, needs a method that follows the surface, and t-SNE keeps neighbours together. You buy that with everything at a larger scale. What survives another perplexity you may report: which points group together. Everything else in the picture is decoration. Never cluster a t-SNE picture or read its distances: on pure noise it draws groups too. In a report, write down whether you scaled and why, the variance your PCA picture holds, and the perplexity and one other you tried.

4.3 How does text become a vector?

Before any neural network, the cheapest way to turn text into numbers is to count. Which runs of three characters does a text contain, and how often? That gives a row of counts per text, and two rows can be compared. No GPU, no download, seconds on any laptop, and you can read back what the numbers mean. Trigrams pick up how someone writes: commas, full stops, their own spellings, the words they always use. Students before you have used this to separate in-laws from their own family in a family chat, dialects, age groups. It also has a trap, and a published claim shows it.

4.3.1 A trigram is any run of three characters

`CountVectorizer(analyzer="char", ngram_range=(3, 3))` finds every trigram that occurs anywhere in the texts, makes each one a column, and counts how often each text contains it. It can also count words, or runs of two to four characters; that has other effects. Take "yellow banana" and "papagena papaya banana": together they contain 23 distinct trigrams, and `ana` occurs twice in each. Most of a row is zero, and that stays true at scale: the posts in the next example span 27,179 trigrams.

To compare two rows, the *Manhattan distance* adds up the absolute difference per column. On counts it reads directly: these two strings differ by 21 trigram occurrences. It grows with length, so texts you compare must be the same length: a text twice as long has counts twice as big, and would be far from everything for its length alone.

4.3.2 "QAnon is two different people"

QAnon was an anonymous person, or persons, who spread conspiracy theories on online forums, claiming a high security clearance in the American government and leaving predictions and puzzles. It had real consequences: many of the people who stormed the Capitol on 6 January 2021 followed Q. In 2020 the Swiss company OrphAnalytics, which does authorship analysis for court cases, counted character trigrams in every post by Q and concluded: two authors, one on 4chan in 2017 and a second after the move to 8chan. Their software was secret, but they described the method well enough to rebuild it.

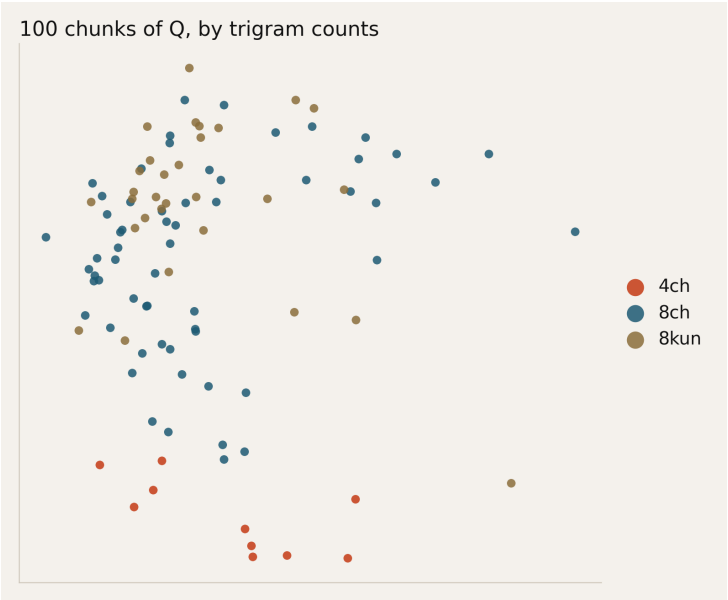


Figure 64: 4,966 posts in 100 chunks of equal length, trigrams counted, compared by Manhattan distance, drawn with PCA. The colour is the site, and a site is also a period.

Rebuilt in a few lines: 4,966 posts, cut into 100 chunks of equal length, counted, compared, and drawn with PCA (figure 64). The ten 4chan chunks sit apart, as in the paper. When I had rebuilt it and got the same picture, I was pleased: it works. Now lesson 5's question: what does this picture support?

4.3.3 What separates the 4chan months: questions, and links

The advantage of a vector you made yourself: you can read it back. Per trigram, take its share of a chunk on 4chan minus its share after the move, and list the extremes.

More common on 4chan	␣wh, ?␣w, is␣, ␣th, hat, wha: *what*, *why*, *is*, question marks
More common after the move	ttp, htt, ://, com, tps, .co: links, and nothing else

Table 6: The trigrams that differ most between the 4chan months and the rest. ␣ is a space.

The 4chan side is writing: the early posts were long strings of questions. The other side is links. A link is pasted, not typed, and its domain names the platform rather than the writer. A writer who moves to another forum and starts answering with links changes their trigram counts without changing who they are, and a counter cannot tell the two apart. So remove the links and count again: one extra step in the pipeline.

```
pipeline.add(NormaliseText, column="text", strip_urls=True, letters_only=True)
```

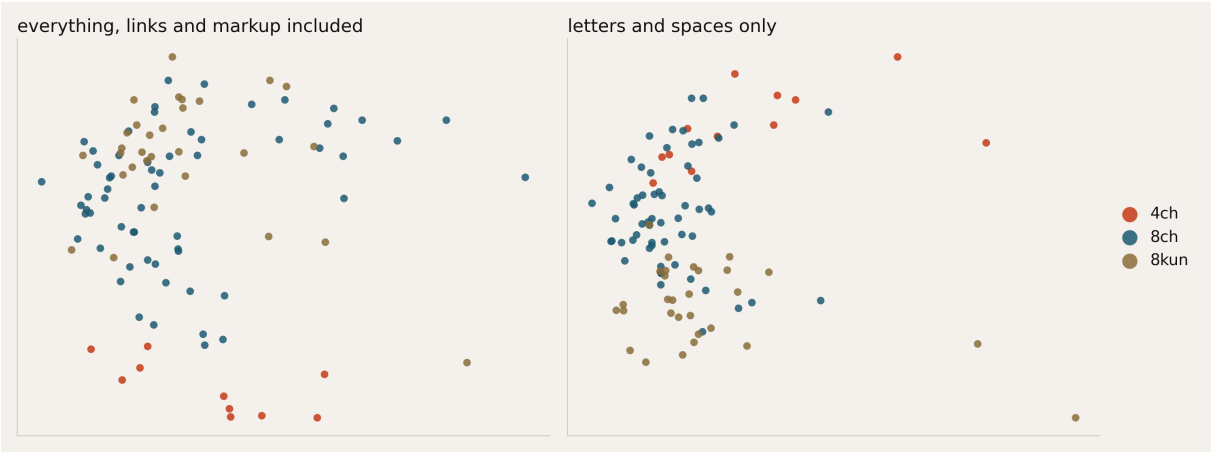


Figure 65: The same posts, counted twice: everything (left), and without links and punctuation (right).

The 4chan chunks no longer sit apart (figure 65). Some are at the edge, several among the 8chan chunks. The split in the first picture was mostly markup. I should say that I didn't find this myself: Claude did, while we were rebuilding the method. I was happy that it worked.

4.3.4 The number that did not move answered another question

Two pictures are hard to compare by eye, so put a number on whether the groups stand apart: the silhouette from question two. For every chunk, it compares the average distance to its own group with the average distance to the other group. Around zero means the groups do not separate.

Silhouette, 4chan vs. after	everything	letters only
mean over all 100 chunks	0.083	0.087
mean over the 4chan chunks	0.214	0.073

Table 7: The usual summary barely moves; the one that answers the question drops by two thirds.

The first row is the number you would usually report, and it barely moves. But 90 of the 100 chunks are after the move, so it mostly measures them. The question is about the 4chan chunks, and their own separation drops by two thirds. The picture was right; the mean hid it. Use the number that answers your question.

4.3.5 Every stretch of time resembles itself

A silhouette of 0.073 is still above zero. Above zero compared to what? Shuffled labels are the wrong comparison here. The chunks are in time order, and text written in the same weeks shares topics, so any run of consecutive chunks resembles itself more than a random handful would. The fair comparison for the first twelve chunks is every other run of twelve consecutive chunks (figure 66).

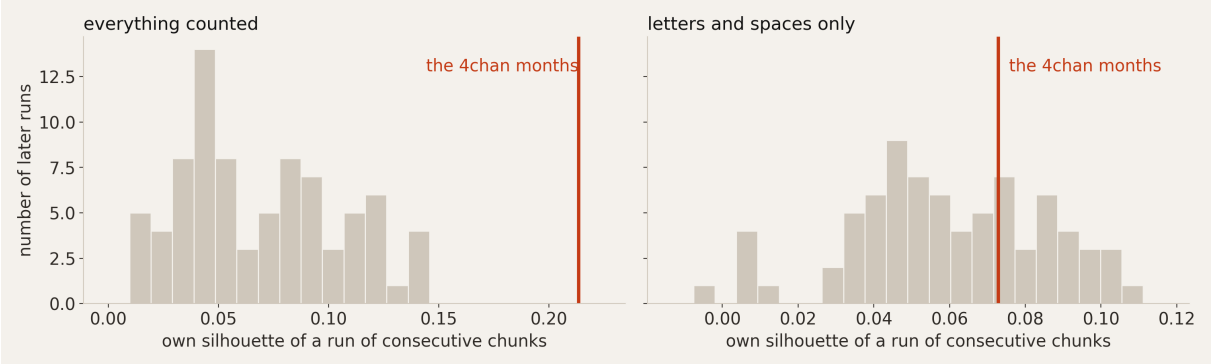


Figure 66: Grey: the same measurement for every later run of consecutive chunks of the same length. Vermilion: the 4chan months.

With everything counted, the 4chan months stand out from every other stretch of the corpus: that is the link effect. With letters only, 25 of the 77 later stretches separate better. The 4chan months are an ordinary stretch of time. What is left after the markup is the drift every text corpus has: what someone writes about changes from month to month.

4.3.6 This picture is not evidence for two authors

Through lesson 5's grid. The model fails: without links, the 4chan months are no more distinct than other stretches of time. The mechanism has two candidates: a new author, or the same author with a new habit and drifting topics. Both predict this picture. Replication: none, one corpus with one switch. That is the cell "plausible, unproven", with a note on what would settle it. A better picture would not; a different question would. Not "do the chunks differ?" but "whose known writing do they resemble?" In 2022 two teams did that, comparing Q's posts with texts by candidate authors, and both pointed at the same two people. The claim may be true; this method could not show it. So I am not sure we should hire them for our court case.

The general lesson is the one that makes high-dimensional work tricky. You do something complicated to your data, splitting it into 27,000 trigram dimensions and projecting them, and it looks as if you have proven that there are two authors. What you have actually done is count URLs in a very complicated way. One regular expression would have shown that the later posts contain more links. With 27,000 columns you cannot look at everything,

so you have to read back what did the work. Your own chat has the same trap: if you exported it without media, every photo became a line saying the image was omitted, and those trigrams, or the peak they make in message length, will show up as a pattern.

Again, concretely. Counting characters is the cheapest way to make text into numbers, and its advantage is that you can always read back which counts did the work. That reading exposed the links. Two checks then took the claim apart, and both are habits for any project. A number has to answer your question: a mean over a hundred chunks did not answer a question about ten of them. And compare your finding with what a split that means nothing gives, chosen so it keeps everything but the thing you claim: here other stretches of time, not shuffled labels. Trigrams still work for authorship; just think about the links.

4.4 Why use a model somebody else trained?

Trigram counts are a vector you design yourself. Most vectors you will use from now on were designed by somebody else: a large neural network, trained by a company with far more data and computers than any of us, turns a photo or a sentence into a few hundred numbers. Hugging Face hosts more than half a million such models, open, many of them put there by researchers.

Those numbers are a kind of summary. Just as you can turn a taste into salty, sweet, sour and spicy, a model can turn a photo into 384 numbers, each a direction it learned. What you do with them is up to you. To predict something from them, such as which flower is in the photo, you put a small model on top: the *head*. Think of a vacuum cleaner you rent. The motor, the heavy part, was built by Meta. All you do is change the nozzle for your own job: you only want to recognise flowers, and you don't need everything the motor knows. Reusing what a model learned for a new task is *transfer learning*; you keep the big model as it is, frozen, and train only the head.

4.4.1 The encoder runs once; the head trains on what it stored

The pipeline of notebook 06.3: the Oxford flowers dataset, 5,000 photos of 102 species. The encoder is DINOv2-small, a network of 22 million weights that Meta trained on 142 million photos. It has never seen these flower labels. It turns each photo into 384 numbers.

```
VectorCache.from_hub("pttrn-io/oxford-flowers-dinov2-small",
                     split="train")
```

Here is the point that matters for your own projects. A frozen encoder gives the same 384 numbers for the same photo every time. So there is nothing to recompute: run it once and store the result in a *cache*. I ran the 5,000 photos through it once, on my laptop, and published the vectors on the Hugging Face hub; you download them in under a second, without a GPU. If you do this yourself, it is just as smart: pay for the encoder once, and do everything after that on a much lighter laptop.

4.4.2 Without a cache, the encoder does nearly all the work, every epoch

How much does the cache save? Count multiplications (figure 67). One photo through the encoder: about 5.7 billion. The head: 384×102, about 39 thousand. The encoder costs

144,000 times more.

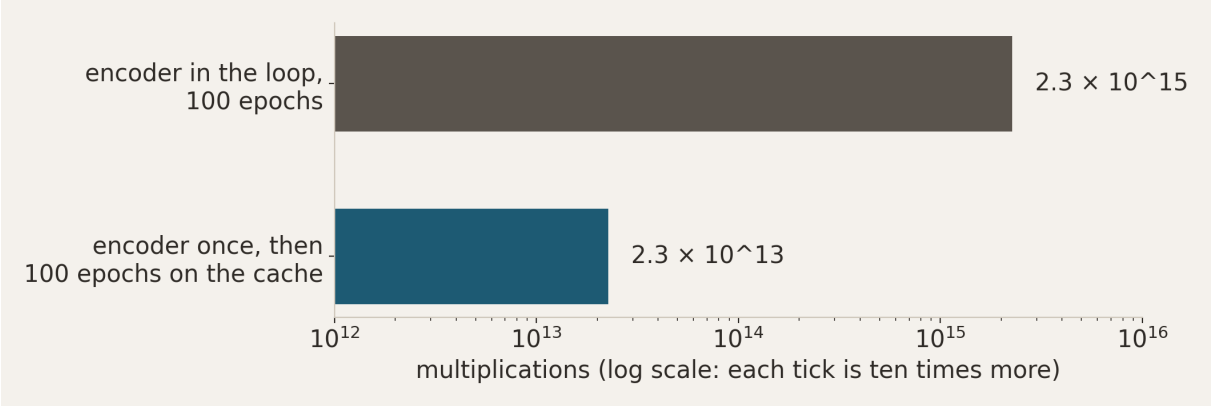


Figure 67: Multiplications for 100 epochs over 4,000 training photos, with the encoder in the loop and with a cache. The axis is logarithmic.

Training means showing the head the training photos many times; one pass is an epoch. With the encoder in the loop for a hundred epochs, it computes the same vectors a hundred times over. With the cache, a hundred epochs cost about as much as one pass of the encoder. Measured in the notebook: a hundred epochs in about four seconds on a laptop. That is also how the machine learning course works: models trained by others, their output computed once.

4.4.3 An organised space, before anything is trained

If the encoder did its job, photos of the same species should already sit near each other. Figure 68 is a PCA of the 384 numbers for eight species, coloured by species. I did nothing: I trained nothing, learned nothing, only turned each photo into a vector. Photos of one species already sit together.

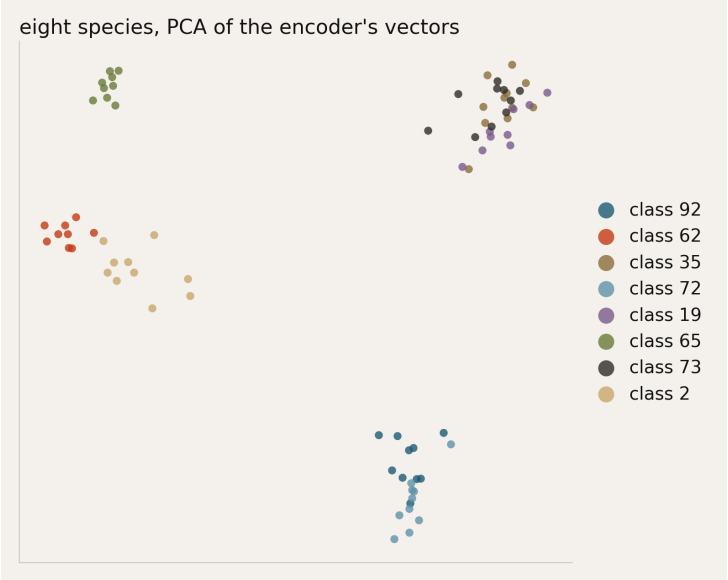


Figure 68: Eight of the 102 species: PCA of the encoder's 384 numbers, coloured by species.

A few species overlap in these two components; they are probably flowers that look alike, and there are 384 dimensions, not two. Compare MNIST's raw pixels: the same two-component PCA gave a smear. The projection does not create the groups. The encoder had already organised the space: it learned patterns from 142 million photos, colours and shapes, the way the sevens had their few ways of varying, and it recognises them in flowers it never saw. PCA only lets you see it.

4.4.4 Cosine similarity compares the direction, not the length

"Near" needs a measure. Make it concrete with vectors you can read: count two words per message, *install* and *haha* (figure 69). $a = (1, 2)$; b is a three times over, $(3, 6)$; $c = (2, 1)$, another mix.

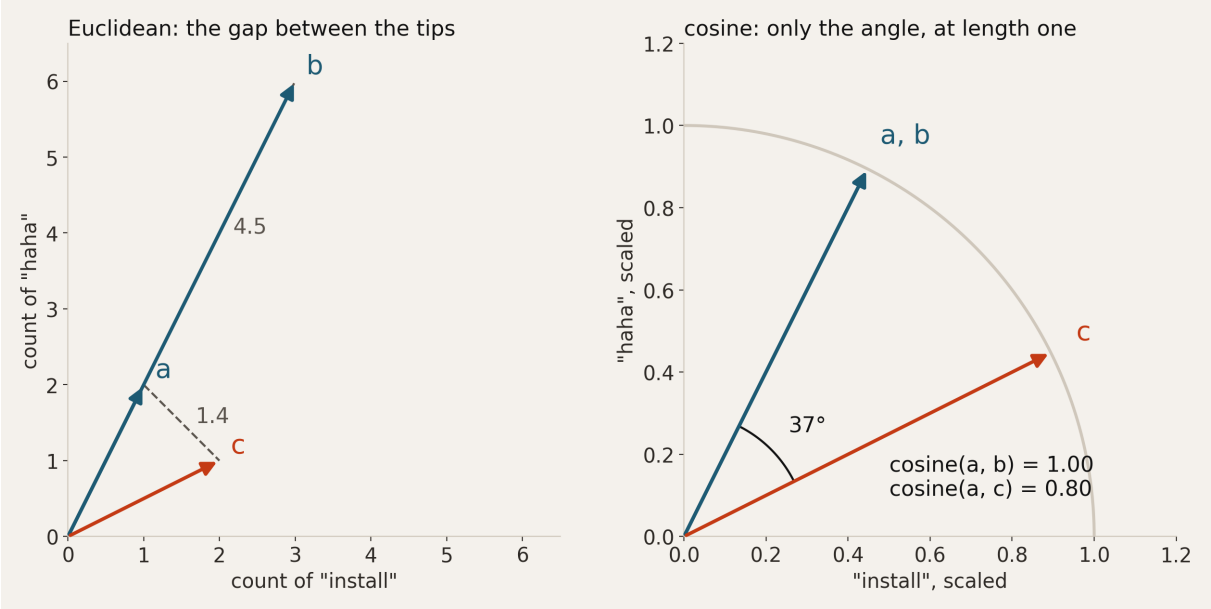


Figure 69: Three messages as arrows. Left: Euclidean distance. Right: the angle.

By Euclidean distance, a is closer to c (1.4) than to b (4.5): the message that says something else wins over the one that says the same thing three times, because length decides. Cosine similarity looks at the angle instead, as if you draw an arrow from zero to each point and ask how close the arrows lie. a and b point the same way: cosine 1. c is 37° off: 0.80.

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

Scale both vectors to length 1 first, and the cosine is just the dot product. Why is this better with many dimensions? Because of the problem from question two: with 384 properties, two things are always alike on one of them by chance. The direction in a 384-dimensional space gives a much better picture than any one axis. That is all you need to remember: many dimensions, use cosine similarity.

4.4.5 Nearest neighbour: copy the label of the most similar photo

Now the classifier that needs no training. Half the photos are a training set, whose labels I know; the others are a test set, whose labels are secret. To label a test photo, find the most similar training photo and copy its label. If it is closest to a daisy, it is a daisy.

```
(test_unit @ train_unit.T).argmax(dim=1)
```

Scale every vector to length 1, and one matrix product gives every cosine at once: a row per test photo, a column per training photo (figure 70).

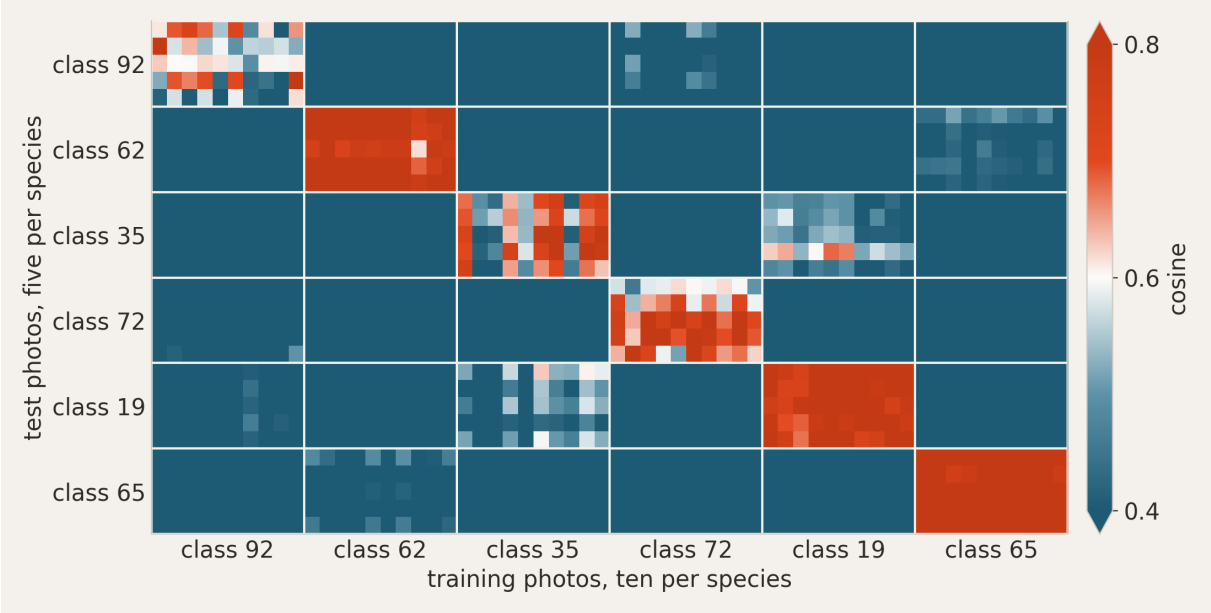


Figure 70: A corner of the cosine table between test and training photos, sorted by species.

A photo is far more similar to its own species (0.70 on average) than to others (0.08), so the most similar photo is nearly always the same species. Copying its label: 99.1% correct on 102 species, with nothing trained. That is a model almost for free. The lighter blocks off the diagonal are species that look alike, which is where the few mistakes come from. Someone asked why class 92 goes wrong sometimes, when its numbers are always the same: the numbers are fixed, but a daisy and a marguerite both have large white petals and an orange centre, and some of those photos simply look more like another class. The trained head reaches 99.3%. On a space this well organised there is little left to learn, and you only know that because you computed the baseline first.

This is a very different way of working from what we did until now. So far you invented your features yourself: regular expressions for URLs, emoji, punctuation. Here you turn the text or the photo into 384 numbers and look at what happens. Sometimes, as with the flowers, you find a very good separation of what is actually in them without doing anything. It doesn't always work. A model is trained on certain data, and that matters: there are models trained on legal texts, but European or American ones? A model trained on aerial photos from China has seen other trees and other houses than one trained on American ones.

Again, concretely. The expensive part of working with a large model is running it, and its output for the same input never changes. So run it once and store the result; everything you try on top then costs almost nothing. That is transfer learning in practice. In a good vector space, near means similar. Cosine similarity is the angle between two vectors, which ignores their length; after scaling to length 1 it is a dot product. Compute

the baseline first: nearest neighbour reached 99.1% without training, and the head added little. Not every laptop can run these models, which is why you can download the vectors.

4.5 What is one row, when a row is “haha”?

The flowers worked because one row was one flower: a clean photo with a clean label. Chat data offers no such luck. What happens to most of you is this: you embed every message, draw the picture, colour it by author, and get a blob. Then you try to fix the picture. The better question is the one lesson 1 started with: what is one row? We follow it on the IRC channel #ubuntu-uk, six people who wrote a lot.

4.5.1 Who wrote these?

Ten random messages from the six busiest people in the channel:

> so a small alarm bell is ringing to me ! • not compared to other policitians ofc • from memory nearly 12 hours • because I’m a developer • yeah that’s definitely not good • like it? based on what? • i was more expecting a sort of terminal server for CAD users :D • what’s the M stand for?

Could you tell who wrote which, if these people were in your group? For a few, perhaps; for most, no. There is nothing in these lines about their writer, and 45% of all messages have five words or fewer. You can tell that they are about computers. You cannot tell who is talking.

4.5.2 One message per row: no authors in the picture

Now the picture most students make first. Every message goes through a small sentence encoder, `all-MiniLM-L6-v2`, which turns a sentence into 384 numbers the way the flower encoder did for photos, so that sentences with similar meanings end up close together. Then PCA, coloured by author (figure 71).

```
SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2").encode(messages)
```

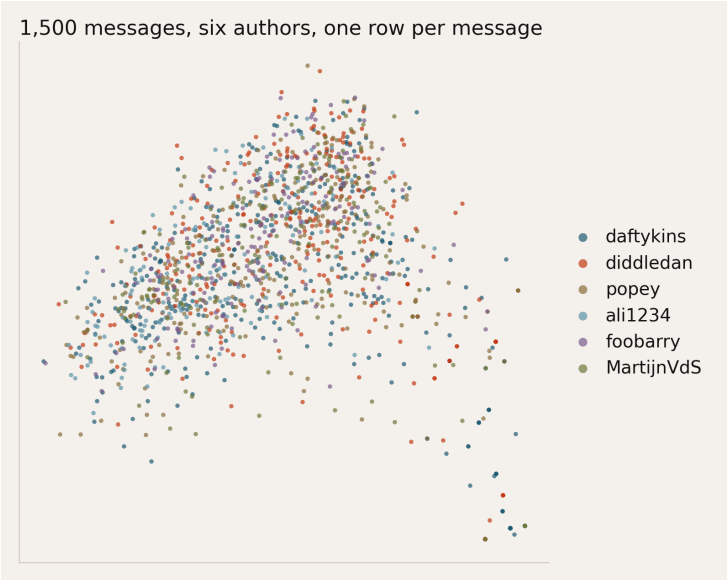


Figure 71: 1,500 messages, embedded one at a time, PCA, coloured by author.

A blob, and the silhouette by author is 0.00 in all 384 dimensions, so this is not the projection hiding something. That is the right answer, and nothing failed. The encoder puts similar meanings close together, and "thanks!" means the same thing whoever types it. If you cannot tell the authors apart by reading the messages, a model that gets nothing but those messages cannot either, however many dimensions it has. Even with an encoder, the information is just not in five words. You may conclude that the model doesn't work on your data. The better move is to change the row.

4.5.3 Three ways to merge messages into one row

Merge consecutive messages from the same person into a larger unit. Figure 72 shows three ways on one person's day: 47 messages, one tick per message, and a colour change where a new unit starts.

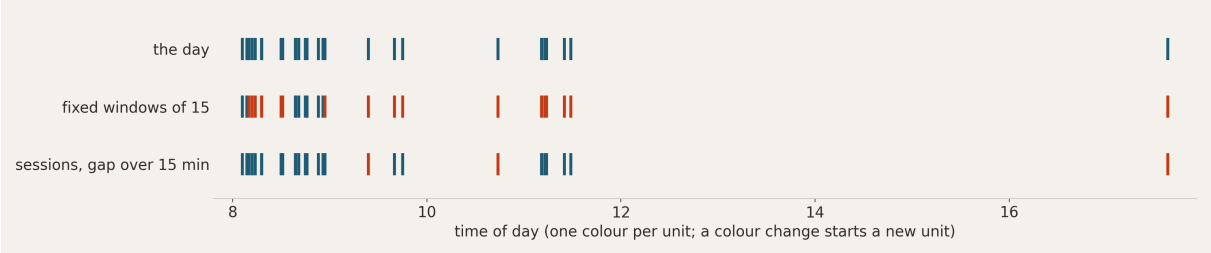


Figure 72: One person, one day, cut three ways: per day, per fixed window of 15 messages, and per session.

- **Day**: everything one person said on one calendar day, however many conversations that was.
- **Fixed window**: every 15 messages from one person, cut wherever the fifteenth falls.
- **Session**: a new unit after a pause of more than 15 minutes. That number comes from lesson 4's exponential fit to the gaps between messages, not from a guess.

```
sessionize(uk, "timestamp", "author", fit_session_threshold(...))
```

Three defensible answers. Which one is best is something you can test.

4.5.4 Does the second half of a unit belong with its first half?

There are no labels to score the units against. So think about what a good unit should be: about one thing. If someone spends ten minutes on a broken printer, the last five minutes are still about the printer, and you could match the two halves even if they were shuffled among hundreds of others. Cut every unit in half by time, embed both halves, and let each first half look for its own second half among 400, by cosine (figure 73).

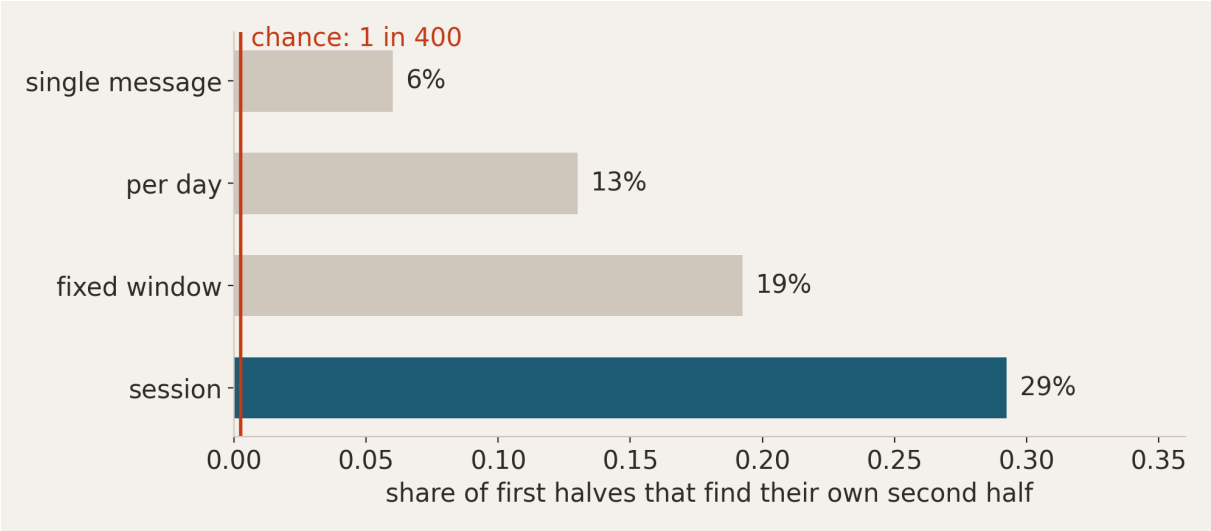


Figure 73: How often the first half of a unit finds its own second half among 400.

Predict first: a session is one conversation, a window cuts through one, a day mixes several. Sessions win with 29%, then fixed windows with 19% and days with 13%; single messages find their other half 6% of the time. All are far above chance, which is one in 400. Days hold more text and still lose: what counts is whether the text belongs together. A student in class had cut their chat into five-minute chunks and found the topics still mixed. That is the same question, and the right size depends on the people: in one chat a conversation is five minutes, in another thirty, some people reply after two hours and others after thirty seconds, some write long messages and others short lines. So test the unit on your own chat; don't copy mine.

4.5.5 Near in meaning, not in habit

What is a merged, embedded corpus good for? Finding what is similar: a session about running out of memory finds, as its nearest neighbour, a session about reserving memory for a GPU, written by someone else. That is retrieval by topic. But do the sessions group by author? No: the average session vectors of the six people all point in nearly the same direction, cosine 0.82 to 0.94 (figure 74). Not so strange: they are all in an Ubuntu channel, talking about Ubuntu.

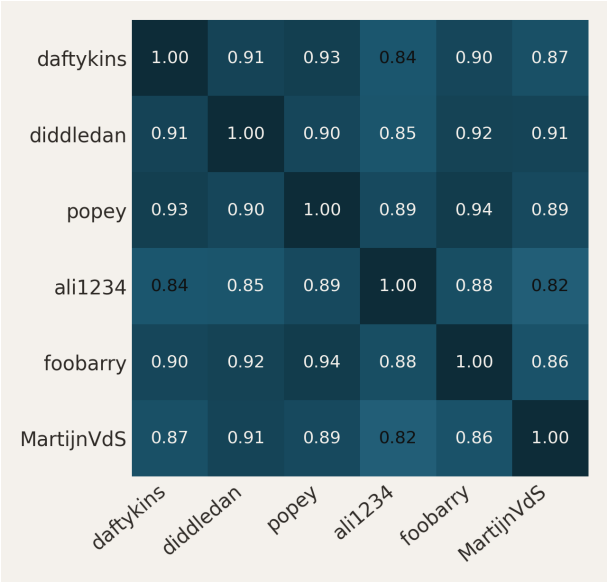


Figure 74: Each author's average session vector, compared by cosine.

Lesson 5 told these same people apart cleanly with a few regexes: a smiley with a nose or without. That signal is style, and a sentence embedding is trained to ignore style: "dont" and "don't" mean the same, so they should land in the same place. A model measures what it was trained to measure.

4.5.6 A race: which vector knows who is writing?

Turn that impression into a number. The task: read one session and name which of the six wrote it. The classifier: logistic regression, as in lesson 5, scored on sessions it never saw. Only the vector differs (figure 75).

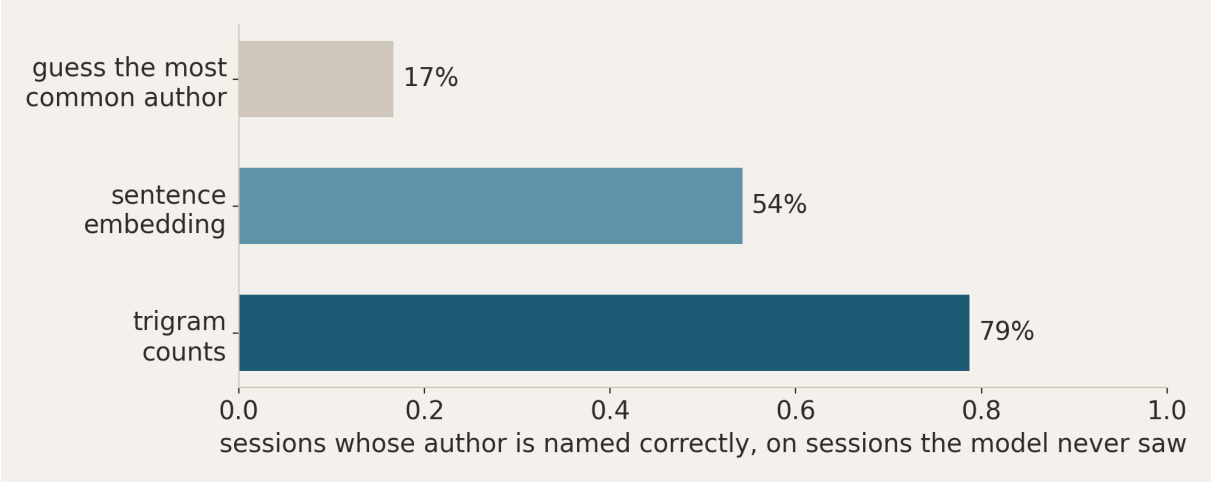


Figure 75: Same task, same classifier, two vectors: who wrote this session?

First the baseline. There are six authors; always guessing the most active one is not a model, it is smart guessing, and it is right 17% of the time. Anything below that is not impressive at all. The sentence embedding reaches 54%, a real improvement. The trigram counts reach 79%. All that deep learning, beaten by counting characters. And the trigram

weights are lesson 5's habits, found by the model on its own: one person's `:-p` and `lol`, another's `:d`, a third person's missing apostrophes.

A student asked: one person always starts with "hey" and the name of their partner; should you take the names out? That is the same as the links in the QAnon posts. If someone always starts with "hey Claudia", that may be the best predictor of who it is, but it is not a writing style; it is one feature you could find with one regex. Ask what you are trying to find. This course is not really about WhatsApp: you already know you are talking to your father. It is about finding patterns in data, and the same thinking holds when you analyse 100,000 helpdesk messages or the sensors in a machine. Where am I looking? Is there information leaking in, something like "hey Claudia", that makes me think I can predict something that doesn't matter?

4.5.7 Check a model on messages where you know the answer

One more model somebody else trained: a sentiment classifier. The tempting use is to run it over everything and report who is the most negative person in the channel. First check it on cases where you know the answer. Writers label their own mood: `:)` or `:(`. Take 600 of each, remove the smiley, and let the model read only the words (figure 76). Here one message per row is right, because a mood belongs to a message: the right unit depends on the question.

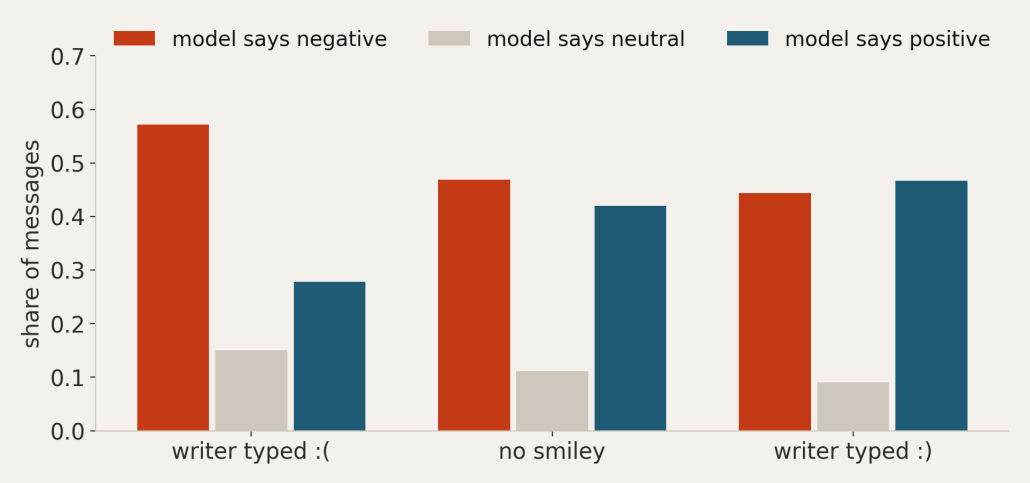


Figure 76: The share called negative, for messages that had a frown, a smile, or no smiley.

It calls 57% of the frown messages negative, but also 44% of the smile messages and 47% of plain ones. It reads a little of the mood: not proof, but an indication that it measures something. Too little to compare people, because their differences would be smaller than its own confusion. Sentiment models sound attractive, and students often use them, but they are usually trained on tweets or product reviews. They work reasonably there, and much less well on chat messages.

Again, concretely. A picture shows only what the rows contain. One message is too small to carry who wrote it, and the blob was not a failure of the model but the correct answer to a badly posed question. How you chunk your data makes a large difference to the result: sometimes you find nothing, and with another unit you do. Choose the row

for the question, sessions for topics and single messages for moods, and check that choice with a test whose outcome you can predict, like the two halves of a session. An embedding measures meaning; for who is writing, count characters. Always know your baseline, and check a borrowed model on cases where you know the answer.

4.6 Why did k-means find three clusters?

Because you said $K = 3$. Methods that always give an answer are tricky. Sorting is the simplest one: count something and sort, and there is always something that scores highest. "I counted something and this person scores more than that one." And what does that tell you? Often the honest answer is: I don't know, I was glad I had a plot. k-means is a more advanced version of the same: my data went into a space with many dimensions, the plot said nothing, so I let k-means loose on it, and now I have three colours. A result we see in projects every year: sentence vectors, a PCA picture of two components, k-means with $K = 3$, and the conclusion: three clusters. Three questions catch it.

4.6.1 k-means finds three groups in anything

The rescue, rebuilt on the 1,500 messages from question five: the blob, cut by k-means into three tidy groups (figure 77, left). Silhouette 0.48: halfway to 1, which in any report would read as decent cluster structure.

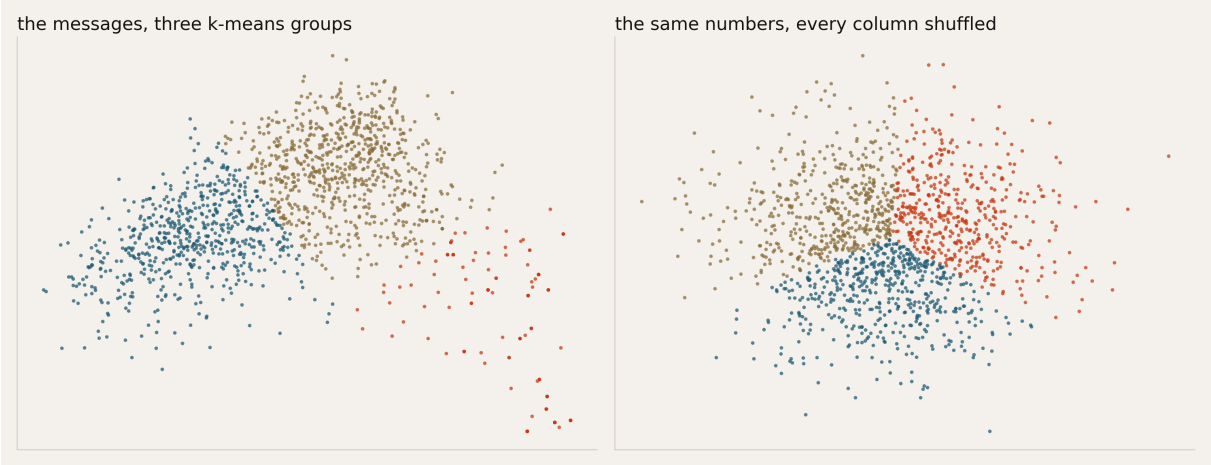


Figure 77: k-means with $K = 3$ on the PCA picture of the 1,500 messages (left), and on the same vectors with every column shuffled (right).

Now make data without structure from the same numbers: shuffle every column on its own, so each column keeps its values but which message a value belongs to is scrambled.

```
np.column_stack([rng.permutation(column) for column in X.T])
```

Same PCA, same k-means: three tidy groups again, silhouette 0.34, not much lower. And k-means with $K = 3$ on all 384 numbers of the real messages, instead of on the picture: its three groups score 0.01. In the full data they overlap almost completely. A low silhouette does not prove there is nothing there, but the tidy groups in the picture are no evidence for anything.

4.6.2 The three groups are short, medium and long messages

What are the groups? Check them against something measured in another way. The cheapest measure here is the number of words.

Group	Median words	Examples	Authors
short	1	"!", "hmm", "nah"	all six
medium	5	"like it? based on what?"	all six
long	9	"i was more expecting a sort of terminal server for CAD users :D"	all six

Table 8: What the three k-means groups contain.

k-means did its job: it cut the cloud along the direction that varies most, how much there is in a message, and handed back three slices because three is what it was asked for. That is a true fact about the data, and a dull one, and it is not the three topics or three people a report would claim. In your project the check is the same: put the groups next to a regex, a word count, a label you trust, and read ten messages from each group. If the groups show up nowhere but in the picture, they belong to the picture.

4.6.3 A picture of 7% is a shadow of 7%

The first question, and the statistics of question one. Two PCA components of these sentence vectors hold 6.8% of the variance (figure 78); you need 54 components for half of it.

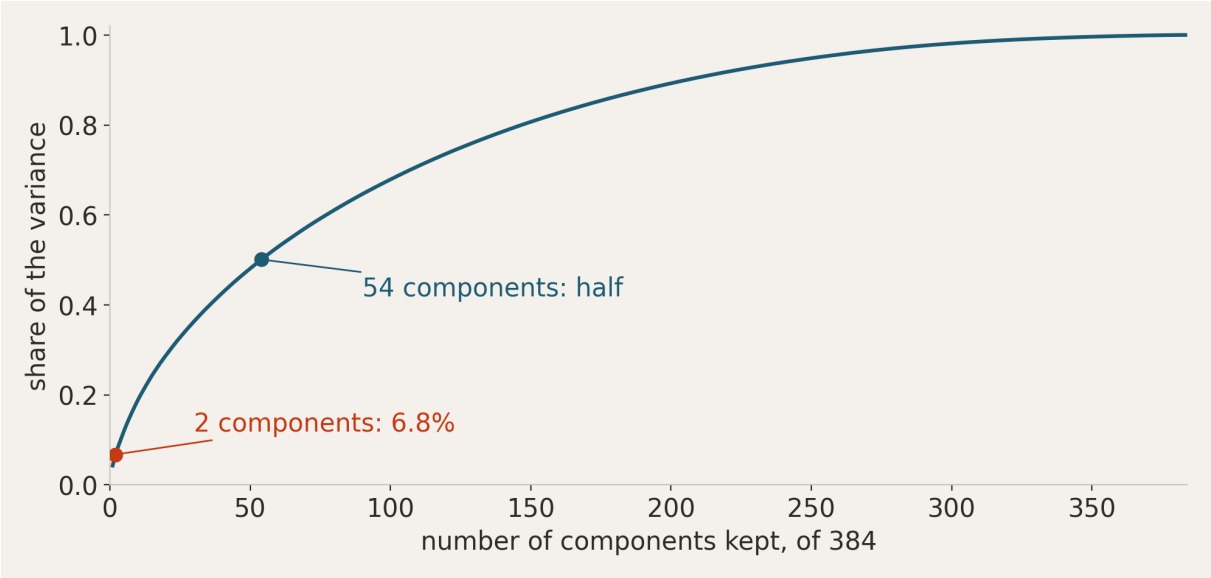


Figure 78: The scree plot of the sentence vectors: the share of the variance per component, and cumulative.

A projection only shortens distances. Points far apart in the picture are far apart in the data; points close together in the picture may be far apart in the other 93%. So you are running k-means on a shadow of what is going on. Projecting to two dimensions destroys

so much of your data that k-means on a PCA picture is almost always a bad idea. Cluster in the space with many dimensions instead, and answer this first question before you cluster anything.

4.6.4 If a tool always finds something, finding something proves nothing

It is the same trap three times. In lesson 5 we shuffled the labels and tested fifteen measures, and in 42% of those hunts something came out significant. In this lesson, t-SNE drew clumps in pure noise, and k-means on that picture scored 0.31 where the data scored 0.016. And k-means found three groups in the sentence vectors, and three in the same vectors with every column shuffled. Each time, the tool produced a result because producing a result is what it does. Two questions catch all three: what would this method show me if there were nothing there? And does what I found show up in something I measured another way?

Again, concretely. k-means always returns K groups, so "$K = 3$ gives three clusters" is a sentence about your input, not a finding. A clustering becomes a finding only after three checks: how much of the data is in the space you clustered, which for two PCA components of sentence vectors is about 7%; what the same steps give on data without structure, which here was three groups too; and what the groups contain, held against something measured another way, which here turned three clusters into short, medium and long messages. For this lesson I am strict on this point. Every chat you put through a model and plot with PCA gives a cloud. Always. Getting that far is the technical pipeline, and that is not worthless; but the plot says nothing yet.

4.7 Up, and back down: what to do with your plot

The last two slides of the lesson are what I want you to do with your plot this week.

4.7.1 Map data up to many dimensions, then back down to two to look

Take data that is not numbers yet, documents of any kind, and map it up into a space with many dimensions (figure 79). That can be a model from Hugging Face, trigram counts, or done by hand: some of you wrote 25 regular expressions, and then every message becomes a point in 25 dimensions. That is fine too. In that space near means similar: searching, comparing and classifying happen there, as with the flowers. Then, to look at it, map it back down to two dimensions with PCA or t-SNE. Two axes always hold only part of the space, so say how much.

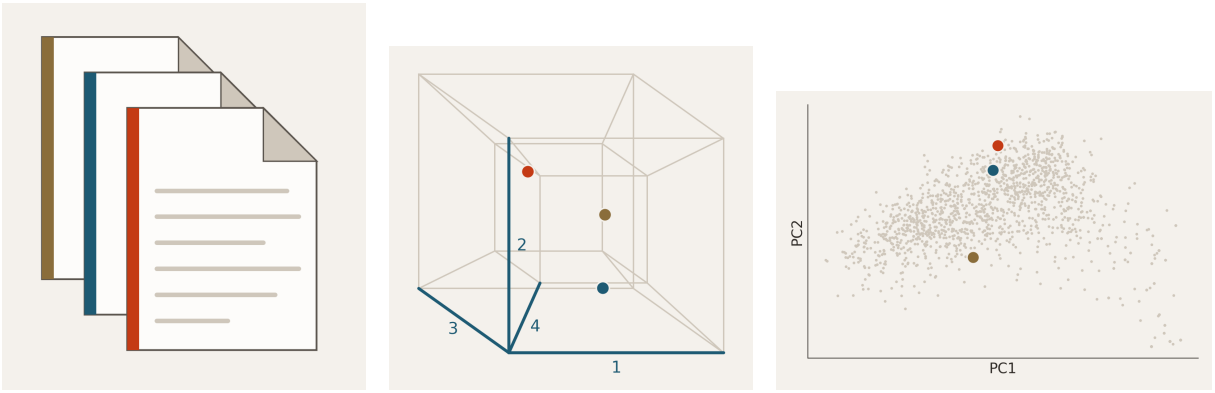


Figure 79: Documents, mapped up to 384 axes (four of them drawn as a hypercube), and back down to two.

4.7.2 A picture tells a story when two routes agree

Many projects end in a cloud of points that says nothing. A pattern that does work: take two routes from the same documents and see whether they agree (figure 80).

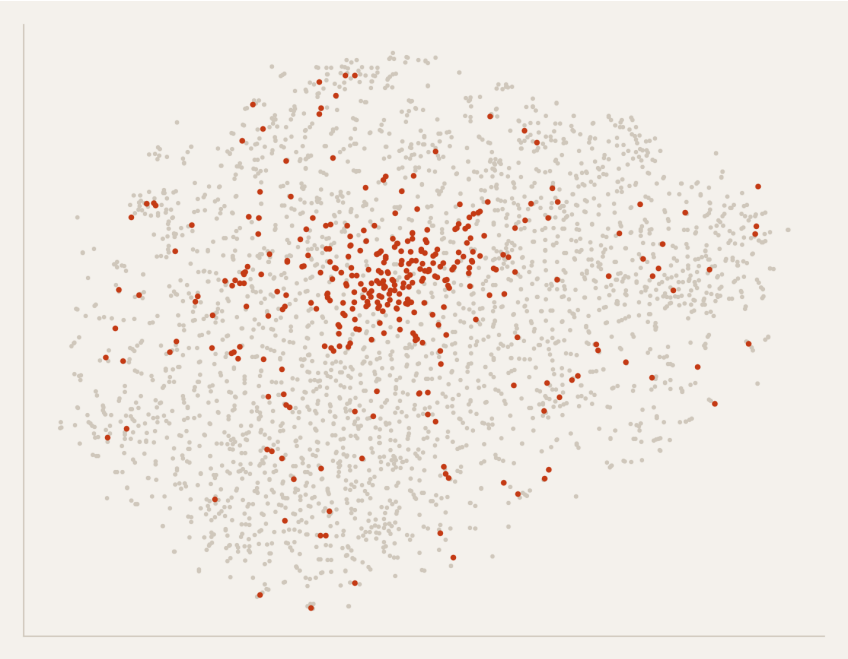


Figure 80: 3,000 sessions placed by a sentence encoder and t-SNE, coloured by a regex: does the session mention a phone?

Route f gives every document a label: with regexes, by hand, or by asking a language model. Route g gives every document a place: a sentence encoder with 384 numbers, or your own 25 regex features, projected down. The two routes do not know about each other. The picture is where they meet: position from g, colour from f. Here the label is "mentions a phone", and those sessions gather in one region: 52% of them have another phone session as their nearest neighbour, against 11.5% by chance. Two independent routes point at the same group, and that is a finding you can defend: the encoder never saw the regex, and the regex never saw the encoder. A scattered colour means f sees something g does not.

Why is that useful? Because one route is often expensive and the other cheap. A former student did this in a master's thesis on fraud. The usual way is to search a dataset with search terms and read the suspicious documents. The student labelled 9,000 e-mails by hand as suspicious or not, and put the same e-mails through an embedding model. The fraudulent e-mails lay close together in the cloud: not all of them, but far more often than chance. Starting from one suspicious e-mail and reading its neighbours, you had to read 30 e-mails instead of 200, a speed-up of about eight times; at 175 euros an hour, that is a very interesting speed-up. The thesis showed that the structure of the embedded documents carries information about fraud, which is exactly what the flowers showed for species. Showing it with labels you already have makes it plausible that the cheap route finds the same thing when you don't have them.

The same thinking carries over to predictive maintenance. A factory where every machine has 50 sensors sends you a point in 50 dimensions every 100 milliseconds. How do you find, in that space, whether a machine needs repair? Can you simplify it, find the patterns, and label the past by when machines broke down? Then see whether the two routes agree. If you can take your data up into a space with many dimensions and back, and you have a story about what it shows, you have practised enough for that.

4.8 The six questions, answered

Question	Answer
What does PCA show you?	The directions of most spread, in your units. Standardise, and read the scree plot.
When is a flat picture not enough?	When data lies on a curve. t-SNE keeps neighbours, not distances.
How does text become a vector?	By counting. Then read back what did the work: in the Q posts, links.
Why use a model somebody else trained?	Pay for it once, store its vectors, train your part on a laptop.
What is one row, when a row is "haha"?	The unit your question needs. Check it with a test you can predict.
Why did k-means find three clusters?	Because you asked. Ask what the same steps find in nothing.

Notebooks 06.1 to 06.4 work through the questions: 06.1 PCA, the penguins, the swiss roll, MNIST, perplexity and t-SNE on noise; 06.2 trigram counts and the QAnon claim checked twice; 06.3 the flower cache, cosine similarity, nearest neighbour and a head trained on the cache; 06.4 your own vectors, from the blob through sessions, the race and the sentiment check, ending in five cells on your own chat. For your project: every picture of many dimensions gets the share of the data it holds in its title, and every group you report gets the two questions: what would the method show if there were nothing there, and does it show up in something you measured another way.

4.9 Feedback on your high-dimensional plots

In class we discussed several of your plots. The same points came back, so I collect them here.

Do a few dimensions do all the work? One plot counted trigrams in a long chat, 5,680 dimensions, and found one PCA axis holding 45% of the variance. A creative touch: that axis drawn as a time series instead of a cloud. But the axis is very likely English against Dutch: the person switched language after two months in the Netherlands. Then 30 or 40 trigrams such as *the*, *you* and *it* do the work, and the 5,000-dimensional space is mostly a handful of hidden regular expressions, just as the QAnon picture turned out to be one regex for URLs. That does not make it a bad plot; it is a critical question worth asking of your own.

Don't throw dimensions away before you start. Another plot grouped 80 regexes into four themes and ran PCA on the four. The result showed stripes. Stripes in a PCA are suspicious: there is so much structure that PCA flattens everything along the axes, which means the data was not high-dimensional enough. Use every regex as a column, 0 or 1 or a count, and you have an 80-dimensional space for PCA or t-SNE. Then think about the colour: your name and your partner's, the topic, or the four themes as route f against a Hugging Face model as route g.

A null result needs tension. One plot coloured trigram chunks by whether the team captain wrote them, and found nothing. Not finding a pattern can be interesting, but only when everybody was convinced there would be one: "for ten years we all thought this, and it isn't so". Nobody expected the team captain to stand out in a 3,000-dimensional space. A single PCA component near 90% in that plot was suspicious too: something, probably a unit, dominated. In a second version the student did see something: the captain writes more formally, in announcements. That is finding something, however modest, and it is very different from finding nothing.

Fewer, better dimensions. One student went from 10,000 trigrams to the 25 that best separate their own writing from their partner's, mapped those 25 down to two, and the two of them came apart. That is a convincing example of two routes: one gives the label, who wrote it, and the other the place, 25 trigrams that identify them. With more than two people it gets harder, but for this assignment this is exactly the exercise.

Never k-means on t-SNE. One plot embedded five-minute chunks with BGE-M3 (1,024 dimensions), projected them with t-SNE, and coloured them by a topic label from a language model: football, over the years. Without k-means that is a good plot. The moment you mention k-means on a t-SNE picture, it is wrong: t-SNE distances mean nothing. And for football, a few regular expressions (club names, *keeper*, *competition*) are good enough as a label, even if they don't catch everything.

5 Lesson 7: Networks

Until now a row was a thing: a message, a person, a penguin. This lesson's data is about how things are connected: who talks to whom, which topics come up together, which stations share a line. That kind of data has its own structure, the graph, and its own pitfalls. A graph sounds very basic, things and relations, but you can do a lot with it, and nearly every step on the way is a choice you have to be able to defend.

The lesson is organised around six questions:

1. **What** is a graph? A set of things and the relations between them. You choose both.
2. What are the **nodes and edges** in your chat? People, but also places, hobbies and objects. Every edge is a rule.
3. What does a graph **measure** add? Sometimes something deeper. Sometimes a column you already had.
4. **Graph or heatmap**? Both, side by side. Sometimes one is much better.
5. Can an algorithm **find the groups**? Yes. But sometimes it finds groups that aren't there.
6. Are the groups **real**? Ask what the method finds in a network without any.

The first two are about what a network is and what it can be made of, in the world and in your own chat, where people are only the most obvious nodes. The third is about the numbers you can compute on a network, and the one question to ask of each: what does it tell me that I did not have already? The fourth is about looking. The last two are about groups: an algorithm finds them, and then you check whether they are there. That last question is the same one as k-means in lesson 6, and the same one that has come back in every lesson: a method that always gives an answer is no evidence on its own.

5.1 What is a graph?

A graph, or network, is the simplest structure for data about relations: a set of things, the *nodes*, and a set of pairs of things that are related, the *edges*. The nodes are whatever your question is about: people, train stations, topics, bank accounts, posts. The edges are pairs that are related: replied to, connected by a train, mentioned together, paid.

There is a third ingredient, and it is the one to remember. A chat log does not contain a network; it contains messages. To get a network you need a rule that turns messages into edges, and that rule nearly always has parameters you can turn: how close in time, how often, with or without a direction. Different parameters on the same data give a different network. Nodes, edges and the parameters of the rule together make your picture.

5.1.1 A graph is two sets: nodes and edges

The whole definition fits on one line (figure 81):

$$G = (V, E), \qquad V = \{A, B, C, D, E\}, \qquad E = \{AB,\ AC,\ BC,\ CD,\ DE\}.$$

V is the set of nodes (the letter comes from *vertices*), E the set of edges, each edge a pair of nodes. That is the entire mathematical object; everything else is a way of showing it, or a number computed from it.

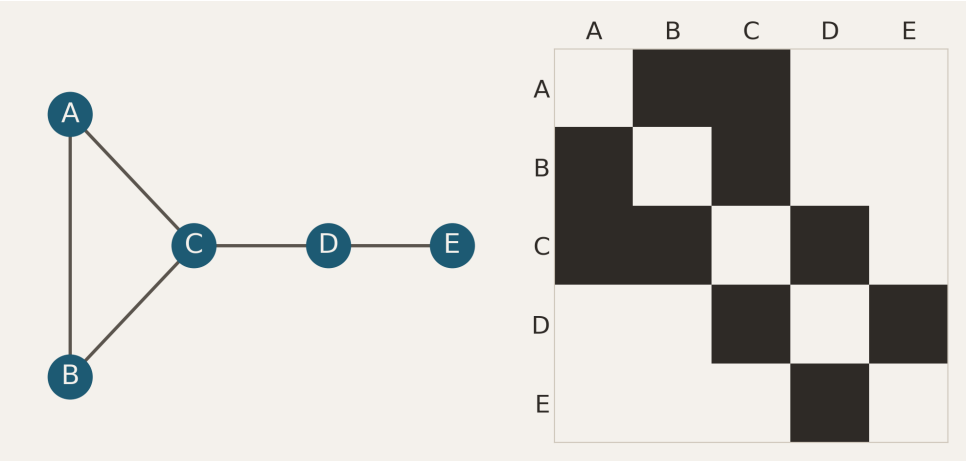


Figure 81: Five nodes and five edges, as a drawing and as a matrix.

The two ways of showing it come back all lesson. The drawing has a dot per node and a line per edge, and it is good at paths: you see at once that to get from A to E you go through C and D. But where the dots go does not follow from the data. That A, B and C sit together in a triangle at the left is simply where the drawing program felt like putting them. The matrix has a row and a column per node and a dark cell where two are connected: the same information, and no layout.

What do you notice about the matrix? It is mirrored in the diagonal. That tells you what extra information a matrix can hold: a relation from A to B and one from B to A are two different cells, and they need not be equal. Very often they are, but not always. On some social networks you follow an account and it does not have to follow you back; in a chat, me addressing you is not the same as you addressing me. Then the edges have a *direction*, the matrix is no longer symmetric, and in the drawing an edge becomes an arrow. Whether your network has a direction is something to think about.

The first number you can compute on a graph is the *degree*: how many edges a node has. C has three, E has one. Question three is about numbers like that, and what they add.

Much data is a graph in disguise. A photo is a set of pixels, and the edges say which pixel lies next to which: a graph, but a very rigid one, a perfect grid. A time series is a graph too, a one-dimensional one: time step 1, then time step 2, and so on. We force a lot of data into a neat line or a neat square, and then forget that it is a graph. Most data is not that tidy, and a graph lets you work with data that does not fit in a spreadsheet.

5.1.2 Nodes and edges can be anything

Two networks that have nothing to do with people talking (figure 82).

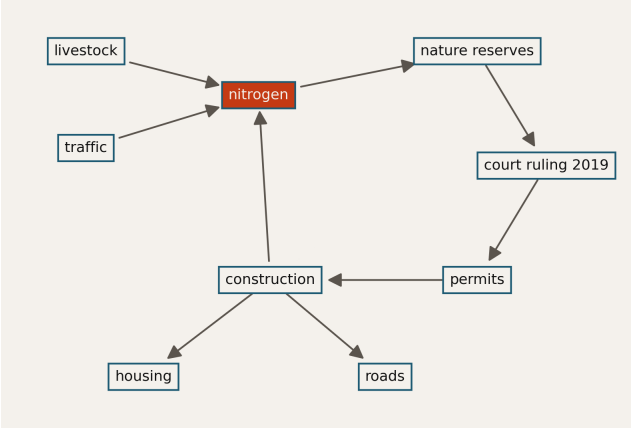




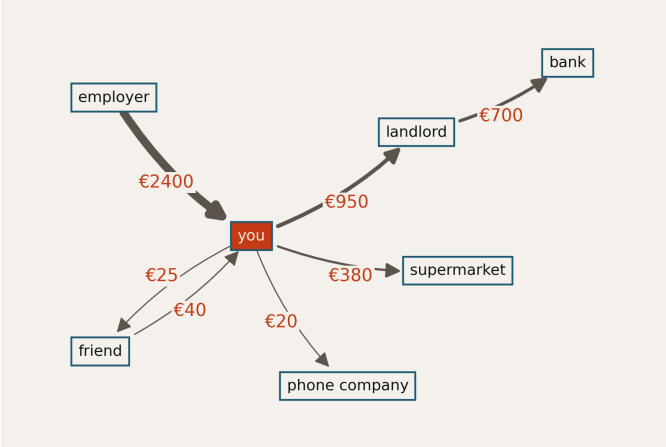
Figure 82: Left: Dutch nitrogen policy, topics as nodes and "affects" as a directed edge. Right: intercity stations as nodes, a direct train as the edge, drawn on the map.

On the left, a knowledge graph of the kind a policy analyst draws: topics as nodes and "affects" as the edge, so the edges have a direction. It is the nitrogen problem in a few boxes: emissions from livestock and traffic reach nature reserves, a court ruling in 2019 stopped the permits that had allowed them, and without permits construction stopped, of houses and of roads, which in turn produce nitrogen. The value of drawing it is the loop: a policy that is a network has consequences that come back.

On the right, the intercity network, drawn on real coordinates. Here the layout is not made up: position is geography. That is rare, and question four comes back to it. Utrecht has the most direct connections, which anyone who travels by train in the Netherlands knows already.

5.1.3 Edges can have a weight, and nodes a kind

Two more, each adding something to the definition (figure 83).



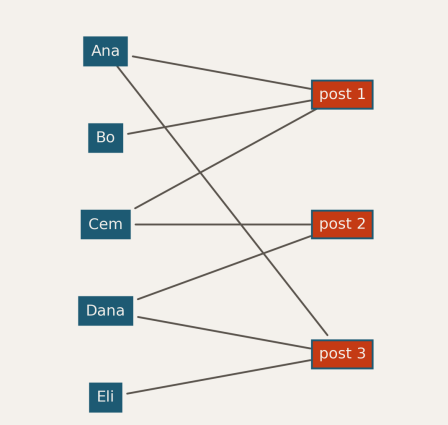


Figure 83: Left: accounts as nodes and payments as directed edges, the amount as the weight. Right: two kinds of node, people and posts, with a line for every like.

Money: bank accounts as nodes and payments as edges, with a direction, who paid whom, and a *weight*, how much. Not every edge is just an edge: more money goes through one than through another. The matrix of this graph is not symmetric, because if you give me money, I do not necessarily give you money. A weight immediately gives you a knob to turn: show only payments above some amount, or only pairs between whom money went at least three times. Banks and tax authorities analyse exactly this graph; money laundering shows up as money that travels in a loop and comes back to where it started, which no single row in a table of transactions shows you.

Likes add a second idea: nodes of two kinds in the same graph. People like posts; people never connect to people directly. From that graph you can make another one: connect two people if they liked the same post. That is the sense in which a network is a decision. The same likes give a people-to-posts graph, a people-to-people graph and a posts-to-posts graph, each with its own centre, and which one you need depends on the question.

Again, concretely. A graph is a set of things and a set of relations between pairs of them, drawn as dots and lines or written as a matrix. The things can be anything, and so can the relations, which may have a direction and a weight. The point to carry into the rest of the lesson: a network is never just in the data. You apply a rule, the rule has parameters, and different parameters give a different network. The rule is part of what you report.

5.2 What are the nodes and edges in your chat?

People are the obvious nodes, and there are already several networks of people in one chat: who answered whom, who was writing at the same time, who mentioned whom by name. But nodes need not be people. Many of you have already made good graphs of topics or places: the places you talk about, the hobbies, the games, the food, the objects. Whatever a word list or a regex finds can be a node, and two of them are connected when they come up together, in the same message, the same ten minutes, the same day.

Every edge is a measurement, so it needs a metric you could hand to someone else. "Mentioned within ten minutes of each other, at least three times" is an example of how you could define that there is a relation. It is a choice you make.

The data for this question is an IRC channel, #ubuntu-uk in 2015, 95,518 messages. It has a convention your WhatsApp chat lacks: people type the name of whoever they are answering. That lets us compare a rule you can use on your own chat with one you mostly cannot.

5.2.1 Two metrics to define a relationship

Two rules on the same messages (figure 84). The *nearby* rule connects two people when they wrote within ten minutes of each other: 3,265 edges between 443 people. It is the rule you can always use, because it needs nothing but timestamps. The *named* rule connects two people when one typed the other's name: 1,146 edges. That has to be lower: you cannot name someone more often than you send a message, because naming someone is a message.



Figure 84: Each dot a person. Across: how many people the nearby rule connects them to. Up: how many the named rule does.

The dashed line is where the two rules would agree, and every dot sits well below it. The people who wrote within ten minutes of each other were often not talking to each other at all; they were simply in the channel at the same time, in different conversations. The extreme case is the bot, lubotu3, which posts automatic messages: it was near 42 people, and exactly one ever addressed it. zmoylan-pi, one of the busiest people in the channel, was near 206 people and talked by name with 51. So the nearby rule is mostly a measure of being online at the same time. That is not wrong; it is a different relation, and it needs to be called what it is. Whichever rule you choose, you always lose something.

5.2.2 The window is a metric too

The nearby rule has a number in it, the window, and the number changes the network (figure 85). A longer window finds more pairs: 2,308 edges at one minute, 4,104 at thirty. The question is who they are.

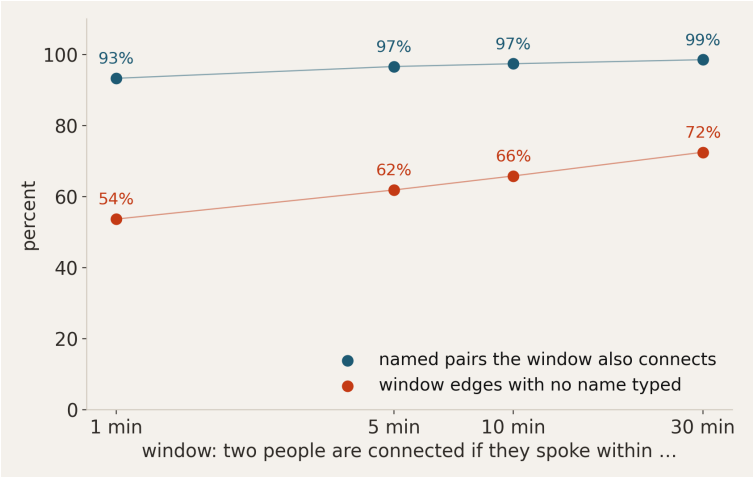


Figure 85: Against the window length. Petrol: the share of named pairs the window also connects. Vermilion: the share of window edges with no name typed.

Here we use the named rule as a check, because in this channel we can. The petrol line asks: of the pairs who addressed each other by name, how many does the window also connect? Almost all of them, 93%, already at one minute: people who talk to each other write close together, so a short window already has the real conversations. Then whatever a longer window adds must be something else, and the vermilion line says what: the share of edges with no name behind them climbs from 54% at one minute to 66% at ten and 72% at thirty. Make the window big enough and you vacuum up everyone who was online at the same time. A longer window does not find more conversation; it finds more bystanders.

When you read a chat yourself, you know whether someone is still answering in one conversation or has started another. From a timestamp alone that is very hard to tell. In your WhatsApp chat you cannot draw the vermilion line either, because there are no names to check against. That is exactly why the window goes in the caption: it is the decision that made your network, and the reader needs it to judge what an edge means.

5.2.3 Topics: what is mentioned together

The same channel with things as nodes instead of people (figure 86). The nodes are 23 words from a word list, and two of them are connected when both were mentioned in the same ten minutes at least eight times.

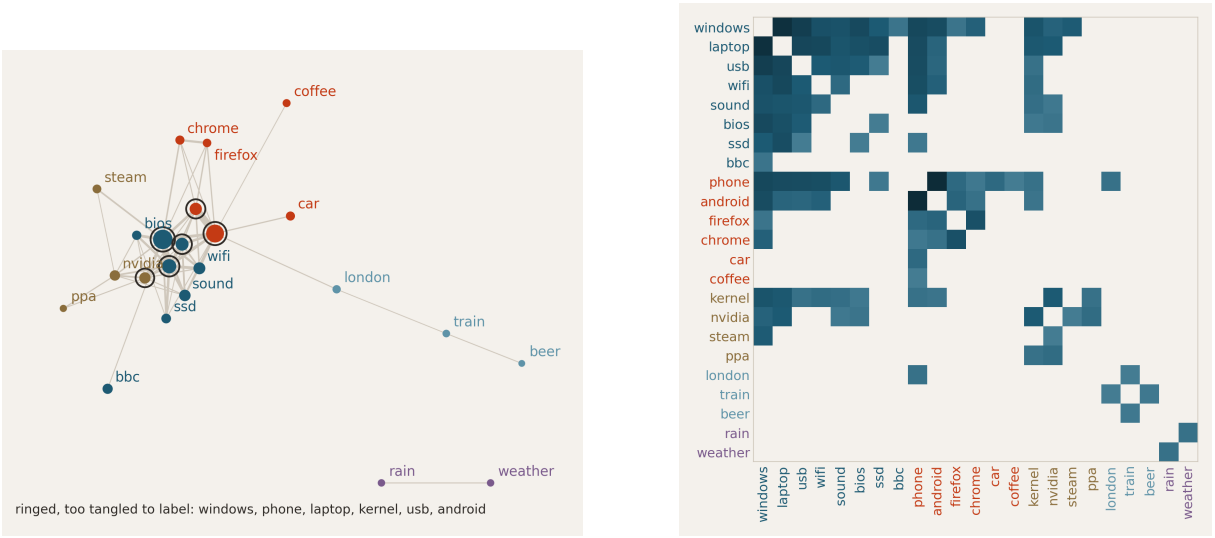


Figure 86: 23 things mentioned in #ubuntu-uk, 59 edges, as a drawing and as a matrix sorted by group.

A network appears. Some edges look random: it is not obvious why wifi and coffee would belong together, and that may well be chance. Others make sense at once: internet and browser, Chrome and Firefox, a tangled knot of hardware words. And then, as a kind of outlier, London, train and beer, and a pair of their own, rain and weather: the off-topic life of an Ubuntu help channel. The clusters you see in the drawing come back in the matrix as blocks. The colours in both are groups found by an algorithm, louvain; question five explains how.

This is the kind of network you can build from your own chat: places you mention together, hobbies, the things that come up on the same evening.

5.2.4 Different lenses on the same dataset

One dataset, many networks. Before anyone writes code, list several:

Nodes	An edge when	Measured as
people	they wrote within ten minutes of each other	times, at least three
people	one @-mentions the other (groups only)	mentions, with a direction
places	both mentioned in the same message	messages
hobbies, games, food	both mentioned on the same day	days
people and places	a person mentions a place	mentions; two kinds of node

The table is a start for a WhatsApp chat, not a list to copy. For people, the nearby rule is what you have; @-mentions exist only in group chats, and they have a direction. For things, every row is a word list plus a window. The last row has two kinds of node, like the likes example. The point of listing several before choosing is that the choice becomes something you can defend: you can say why this network answers your question and the others do not.

Again, concretely. Your chat contains many networks, of people and of things, and every one of them is a set of knobs: what a node is, when two are related, how often, with or without a direction. It is not always obvious which setting is a good one, so you are allowed to search. Two things to search for. First, holes: if everything is connected with everything, there is no story. Networks become interesting through their gaps, where clusters form and some groups are connected and others are not. Second, honesty about the rule: the rule you will use most for people, being active within a window of each other, mostly measures being online together, so name it for what it is, and put the window and the threshold in the caption. They made the network.

5.3 What does a graph measure add?

Once you have a network, there is a whole toolbox of numbers to compute on it, and networkx gives you each in one line. Three things those numbers can reveal:

- **Hubs.** *Betweenness* shows who or what is an intermediary in the network, a switching point that can often control a lot.
- **Triangles.** The *clustering coefficient* says something about how tightly knit the network is, and can reveal social dynamics.
- **Outliers.** Sort nodes by a measure, and see who stands out: especially a node that is high on one measure and low on another.

The question to ask of each is what it adds to the picture you already had. Some measures add something real; some add nothing, and the second half of this question is about recognising which.

5.3.1 The Medici were the broker

Padgett and Ansell (1993) collected the marriages between the elite families of fifteenth-century Florence. Marrying off sons and daughters was politics, and there was a fierce struggle for power between the families, some of whom were bankers with a lot of money at stake. The question was how the Medici, who were not the richest or the oldest family, came to rule the city (figures 87 and 88).

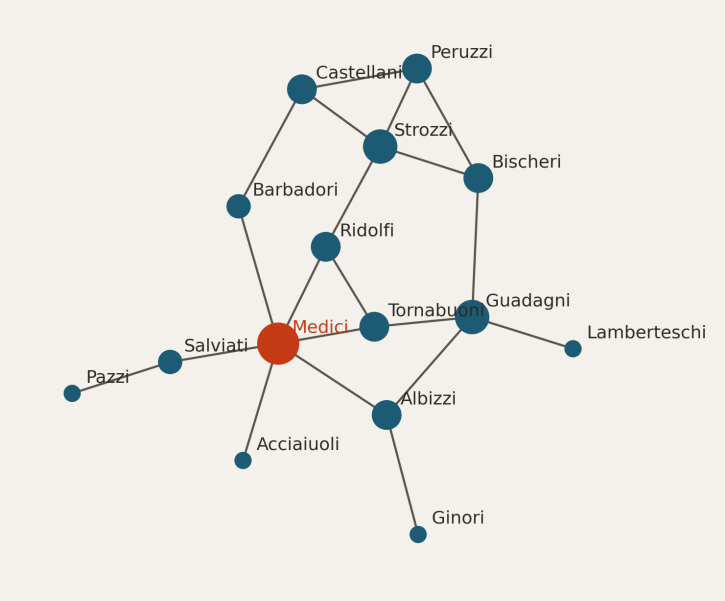


Figure 87: The marriage network of 15 Florentine families, the Medici in vermilion.

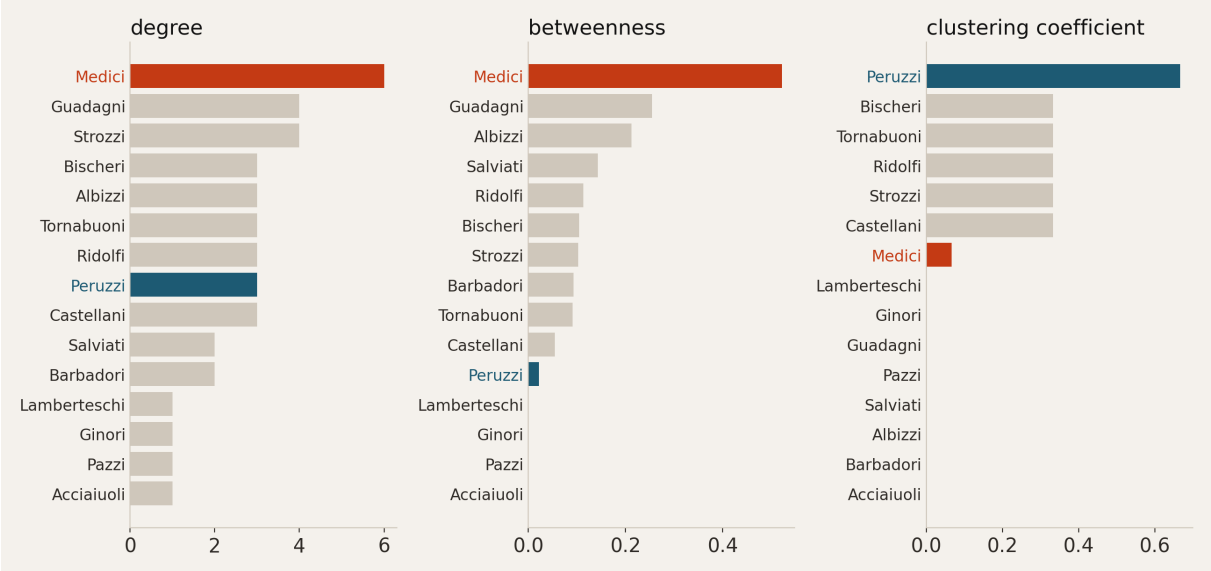


Figure 88: All fifteen families, sorted by degree, betweenness and clustering coefficient.

Degree is simply how many lines you have, and that alone is an interesting measure. The Medici are at the top with six marriages; Guadagni and Strozzi have four. More, but not a different kind of family.

The spider in the web is measured better by *betweenness*. Take two nodes, find the shortest path between them, and ask whether that path has to go through a particular node. Do that for every pair, and a node's betweenness is the share of shortest paths that run through it, a number between 0 and 1. The Medici score 0.52, twice the next family: about half of the shortest routes between other families run through them. If you are with the Pazzi and want to reach the Lamberteschi, you go through the Medici. Families with low betweenness are ones you rarely need: from the Castellani you can reach most families just as quickly without them. Some families score exactly zero, because no route ever needs them. A hub position is a strong one: a middleman who can control what passes, and so has power. That is the outlier, and it is an outlier in a column that only exists in the network.

The third measure is about triangles: are your friends friends with each other? The Peruzzi are connected to the Castellani, the Strozzi and the Bischeri, and those families mostly married each other too: two thirds of the pairs among the Peruzzi's in-laws are connected, so their *clustering coefficient* is 0.67. That embeds them strongly in a closed bloc of old families. The Medici score 0.07: the families they married hardly married each other. So the Medici were the middleman, but not embedded in a tight network. Every alliance went through them, and their partners could not coordinate without them.

In class someone asked whether betweenness belongs with the hubs or with the outliers. With the hubs: betweenness is the measure that tells you what the hubs are, and the clustering coefficient measures the triangles. The outliers come from putting measures side by side: the Medici are an outlier on betweenness and not at all on clustering, and that contrast is the finding. A broker has high betweenness and low clustering.

One caution about the data. The marriages are not all the ties there were: the families also formed business alliances, and those are missing from this graph. To really explain what happened in Florence you would need them too. A network can only show the relations you put in it.

5.3.2 Are your friends friends with each other?

The clustering coefficient of a person counts the pairs among their friends who know each other, and divides by the number of pairs there could be (figure 89):

$$C_i = \frac{\text{pairs of friends of } i \text{ who know each other}}{\text{all pairs of friends of } i}.$$

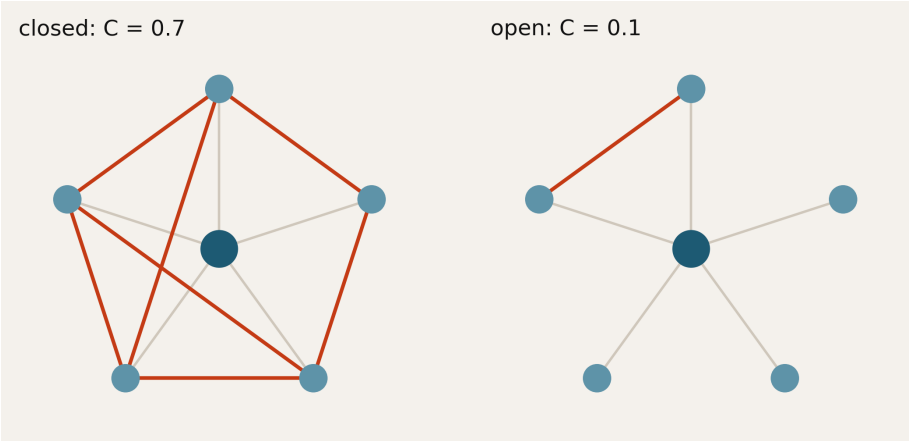


Figure 89: One person, five friends. Closed: 7 of the 10 pairs know each other, $C = 0.7$. Open: 1 of 10, $C = 0.1$.

Among five friends there are ten possible pairs. On the left seven of them know each other: a closed circle. On the right one does: an open circle, held together only by the person in the middle.

The reason this matters is *triadic closure*, one of the oldest observations in sociology: two people with a friend in common tend to become friends. They meet at that friend's birthday, they hear about each other. So real social networks have far more triangles than random ones. And a closed circle has its advantages: your birthday party is not a room of strangers who start arguing, and you can easily arrange to meet the whole group.

Sociology reads the difference between closed and open in two ways. James Coleman (1988) saw closure as social capital: when your friends know each other, they share norms, trust each other and notice when something is wrong. Closed gives support and trust. Ronald Burt (1992) found social capital in the open circle instead: whoever sits between people who do not know each other hears more, and decides what passes between them. Open gives control. The Medici are Burt's case. The same coefficient can mean opposite things, so a network measure says nothing without knowing what its ties carry.

5.3.3 A small world: closed circles, and a few shortcuts

Hubs and triangles together give one of the best-known results in network science (figure 90). Start from a world of closed circles: a thousand people on a ring, each with ten friends, their nearest neighbours, who also know each other. Clustering is high, but a message from one side of the ring to the other has to walk all the way round. Now rewire a small share of the edges to random strangers.

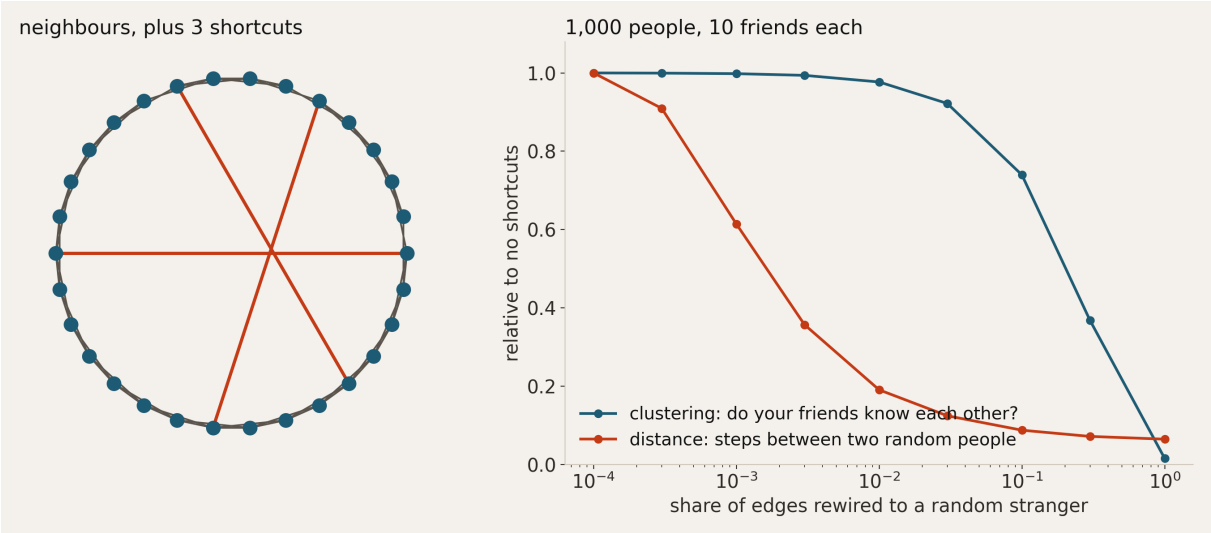


Figure 90: Left: a ring with three shortcuts. Right: for 1,000 people with 10 friends each, clustering and mean distance against the share of edges rewired, relative to the ring without shortcuts.

At one percent rewired, the mean distance between two people has dropped to a fifth, while clustering is still at 98% of what it was. A few shortcuts are enough to make the whole world close, without opening anyone's circle (Watts and Strogatz, 1998). That combination, high clustering and short paths, is called a *small world*, and real social networks are small worlds. It is where "six degrees of separation" comes from: everyone has shaken hands with hundreds of people, so a chain of about six handshakes reaches nearly anyone in the world. Or the Wikipedia game: someone names a random page, the aurochs, and another, the port of Rotterdam, and whoever finds a route of links from one to the other fastest wins. There is always a route within five or six clicks. Facebook measured its own network in 2011: 721 million people, and on average 4.74 steps between two of them. You would not expect it to be that low.

What follows from it: anything that spreads person to person, news, a rumour, a virus, crosses even a huge network in a few steps, through its hubs and brokers. That is also why hubs and brokers have so much control over the flow of information: the network leans on those few to keep its distances short. And a caution for your own analysis: a short average distance in your chat is not a finding; every social network has one.

5.3.4 An open friendship circle, and thoughts of suicide

How far a network measure can reach. Bearman and Moody (2004) used a large American survey in which 13,465 students in grades 7 to 12 named their friends, so the researchers could build the friendship network of each school. For every student they counted the friends of their friends, and asked which share of those are not the student's own friends: the triangles around you that did not close. The paper calls it *intransitivity* (figure 91).

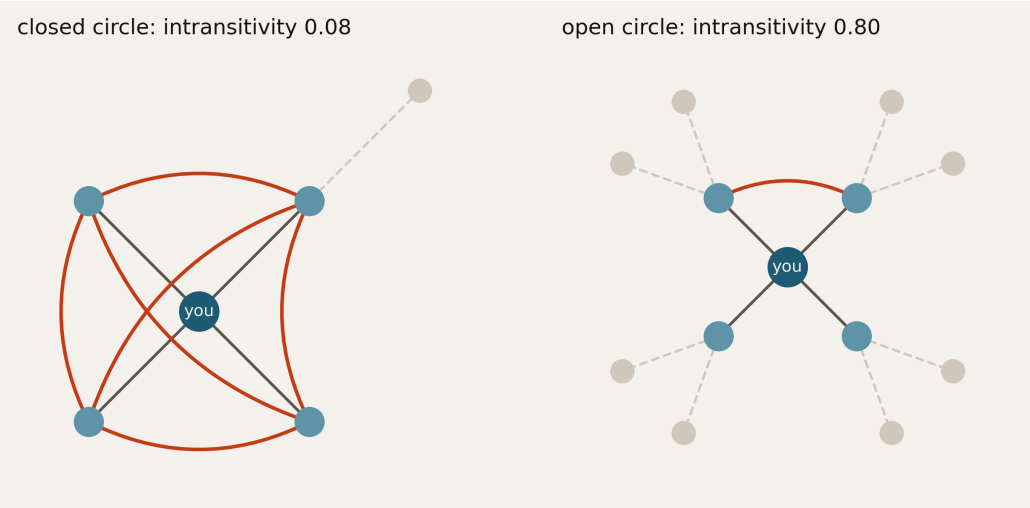


Figure 91: Two made-up friendship circles. Closed: intransitivity 0.08. Open: 0.80.

The outcome was a survey question: in the past twelve months, did you ever seriously think about suicide? For girls, a fully open circle against a fully closed one meant 2.2 times the odds, where 1 means no difference, with isolation, depression and self-esteem held equal. Isolation, the risk you would expect, counts too: 2.0. But the striking part is that the open circle is at least as strong, and it is about girls who *do* have friends.

In class someone asked whether that means you don't really belong to the group. Not quite: you are in a group, but the group is not tight. Take two girls who both know ten people. One knows ten people who do not know each other; the other knows ten people who all know each other. Having friends does not protect you if they do not know each other. For boys neither effect was found, and the authors hardly explain why. For boys as for girls, what did count was a friend's suicide attempt: 2.7 times the odds.

Two cautions. It is a survey at one moment, so it shows an association, not which came first. And the point for us is the method: a number that only exists in the network said something that the individual columns, including a depression score, did not.

You are who you know. This is also why social networks are so pleased with metadata. Suppose I want to know your political preference, and you have decided never to share it on Facebook because it is private. I can still guess it quite well by looking at your social surroundings and what they share. The same goes for much of what we consider private: you do not have to leak anything yourself to be targeted. Another example: the motion sensor of your phone alone, the way you move, turns out to predict depression. If that can be read from movement alone, imagine what can be read from when you are online and whom you talk to: exactly the metadata a chat app has. That is part of why such an app can be free.

5.3.5 How you construct an edge has impact downstream

Now the other side: a measure that adds nothing (figure 92). Under the nearby rule, the *weighted degree* of a person counts how many times someone else wrote within ten minutes of them.

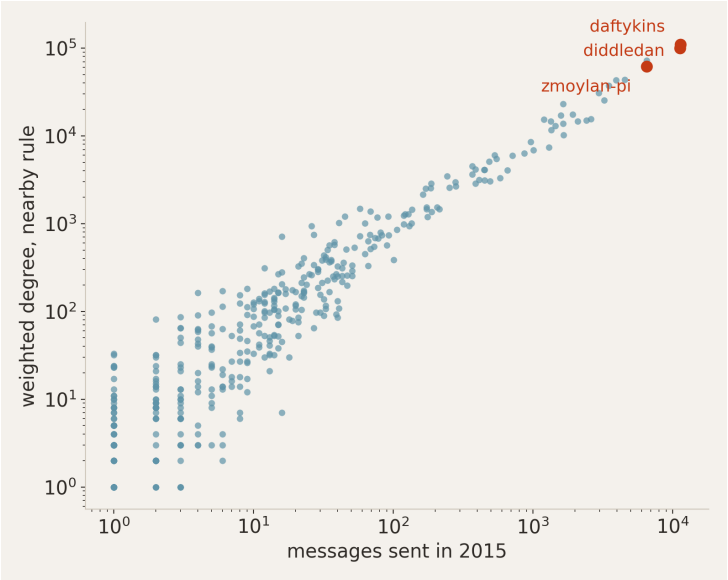


Figure 92: Each dot a person in #ubuntu-uk, on log axes. Across: messages sent in 2015. Up: weighted degree under the nearby rule.

The cloud is a straight band across four orders of magnitude. Of course it is: someone who posts a lot always posts close to other people, whether or not they interact with anyone. The more you write, the more often someone happens to write near you. So "this person is the most central in the channel" says the same as "this person wrote the most messages", which you knew from a `value_counts` in lesson one. You can compute a centrality on such a graph, but then ask what you are measuring. The choice of your network has all kinds of downstream effects on the statistics you compute from it.

That is the test for every network measure you report: put it next to a column you already had, here the message count, and draw the scatter. If it is a line, say so and drop the measure. If people break away from the line, those are the interesting ones.

5.3.6 Is the line carried by the loudest few?

One fair doubt about that scatter: a handful of accounts sent thousands of messages, and an ordinary (Pearson) correlation weighs every point by how far it lies from the mean. So a straight line could be carried by those few, with the crowd doing something else. That is what ranks are for (figure 93). Give the busiest person rank 1, the next rank 2, on both columns, and correlate the ranks:

$$\rho = \mathrm{corr}\Big(\mathrm{rank}(x),\ \mathrm{rank}(y)\Big).$$

That is Spearman's ρ. Eleven thousand messages buy the top rank and nothing more, so every person has one vote, and ρ asks whether the order holds for the whole crowd.

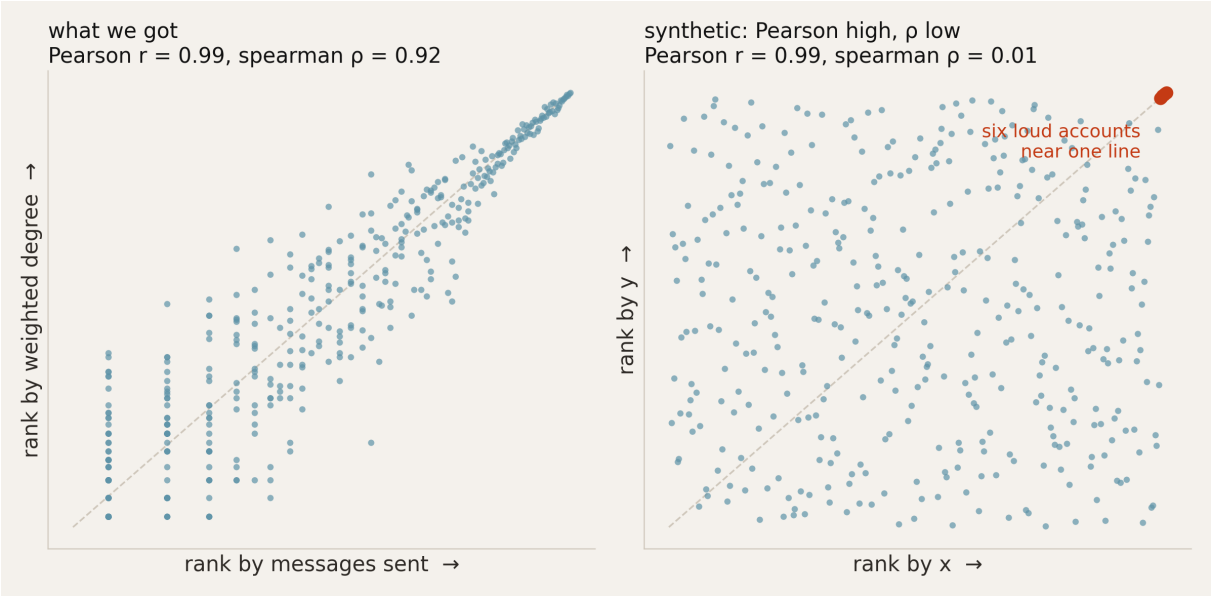


Figure 93: Rank against rank. Left, what we got: weighted degree against messages. Right, synthetic: a random crowd with six loud accounts.

On the left, what we got: Pearson 0.99, Spearman 0.92. The order holds all the way down, so it is not just a few outliers that make the relation. On the right, what the other answer would have looked like, made up and labelled as such: a random crowd and six loud accounts on one line. Pearson says 0.99, because that small group of very active accounts pushes it up; Spearman says 0.01. Then "central means busy" holds for the top six and for nobody else. When you want to express that difference, you need Spearman.

5.3.7 Which degree adds something?

So weighted degree is a message counter. Is there a degree that is not? Plain *degree* counts something else: how many different people you were near, however often (figure 94). The two sound almost the same. One counts every message, the other only people.

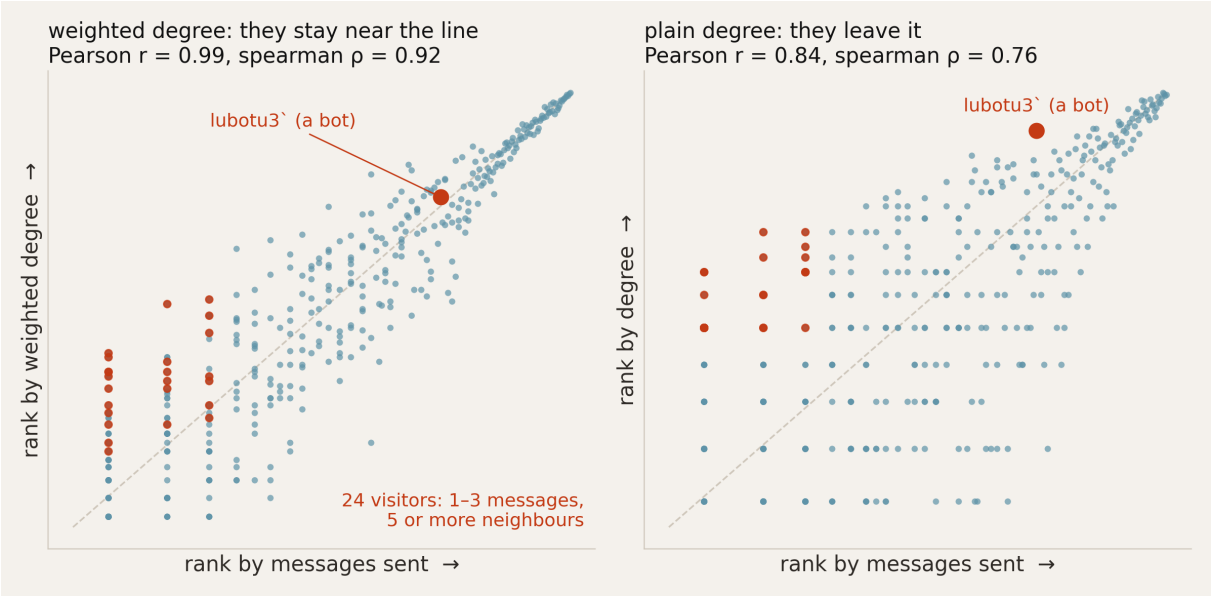


Figure 94: People ranked by messages across. Left: weighted degree up. Right: plain degree up. Marked: the bot and 24 visitors with one to three messages.

Mark the same people in both panels: the bot, and 24 visitors who wrote one to three messages. Under weighted degree they stay in the band; the bot sits right on the diagonal. Under plain degree they leave it: the bot climbs seventy places, and the visitors a median of 148, because their few messages landed in a busy ten minutes and every person writing then became a neighbour. A visitor with one to three messages is hard to find in the left panel, and with a slightly different measure they suddenly lie far off the diagonal.

So plain degree, at $\rho = 0.76$, does add something to the message count, and what it adds is presence: being there when others are talking, not talking with them. A list of people who are everywhere and say little is worth looking at, and the bot is the first name on it. It is a very small tweak in how you measure, and it suddenly shows something the other measure hides. The order of work: put the measure next to a column you already had, and when people leave the line, look at who they are before you give the measure a name.

5.3.8 Does the window decide who is central?

A second use of Spearman, and a different question. Not whether a measure adds something, but whether a result survives a choice you made (figure 95). A longer window adds edges, mostly bystanders. If it also reshuffles who has the most, “most central” is a fact about the window. So rebuild the graph at one, five, ten and thirty minutes, rank everyone by degree each time, and follow the people at the top.

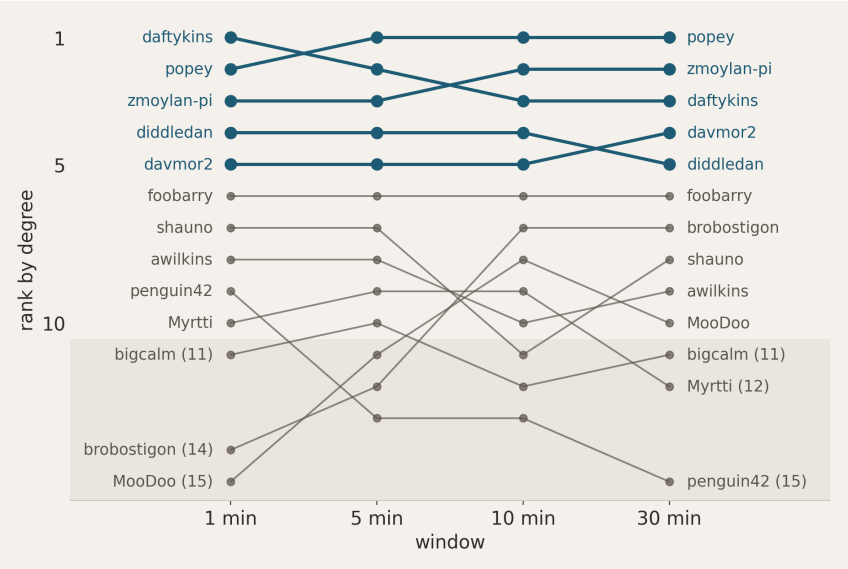


Figure 95: Rank by degree at windows of 1, 5, 10 and 30 minutes, for everyone who reaches the top ten at any window.

The same five people are the top five at every window. They swap places among themselves, between one and five minutes and again later, but the structure stays intact. Spearman against the ten-minute ranking: 0.92 at one minute, 0.98 at five, 0.95 at thirty. For the top five it hardly matters which window you take; you can play with the window freely. Lower down people do move: one climbs from 14th at one minute to 7th at ten. A claim about who moves from place 15 to place 10 needs much more thought: what noise are you pulling in by turning the window up?

Why this matters. With a network you have a lot of knobs: the window, messages or names, counting every message or not, weighted or not. Before you know it you are turning all of them, you see something, and you say: there, I found something, now stop, don't touch it. For a course assignment that may get you a nice plot. But in your work, when you investigate something complex and doubt it yourself (I think I found a network effect, but how solid is it?), people will rightly say: "but you chose a window of ten minutes". If you have done this check, you can answer: I looked at that, and for the top five it does not matter. That is a much more solid way to say something. Had the lines crossed wildly, the analysis would not be wrong: a short window misses conversations, a long one adds bystanders, and neither is the true network. It would mean the ranking belongs to the window, so you report it with its window and show who moves.

Again, concretely. Hubs: some nodes hold a network together. Betweenness found the Medici, who were not the biggest family but the one every route ran through; and a few such shortcuts are why even a huge network is only a few steps across. Connectedness: whether the people around a node know each other, counted in triangles, which can tell you a lot about how close a group is. Outliers: many measures in a chat say nothing new, because activity drives everything. So put each measure next to a column you already had and draw the scatter. The line repeats the column; the people off the line are the finding, if the ranking survives your window.

5.4 Graph or heatmap?

There are two ways to look at a network, and they complement each other. The drawing, dots and lines, is what everyone expects, and it is good at paths, hubs and bridges. But a drawing has a *layout*, and the layout is an algorithm in itself, usually with a random seed in it: position in the picture is not data. The matrix is a heatmap: one row and one column per node and a coloured cell for every edge. It needs no layout, and nothing gets tangled up in it. What it hides is paths, and the order of its rows is a choice too, one we will play with. In the end the question is what story you want to tell, and one is simply better suited than the other.

5.4.1 One graph, two layouts, identical degrees

One network, the mention graph of the people with at least six conversation partners, drawn twice (figure 96). Colour and size are degree, the same in both panels; only the positions differ. In networkx this is choosing `spring_layout` or `circular_layout`. Exactly the same data; nothing in the data changed.

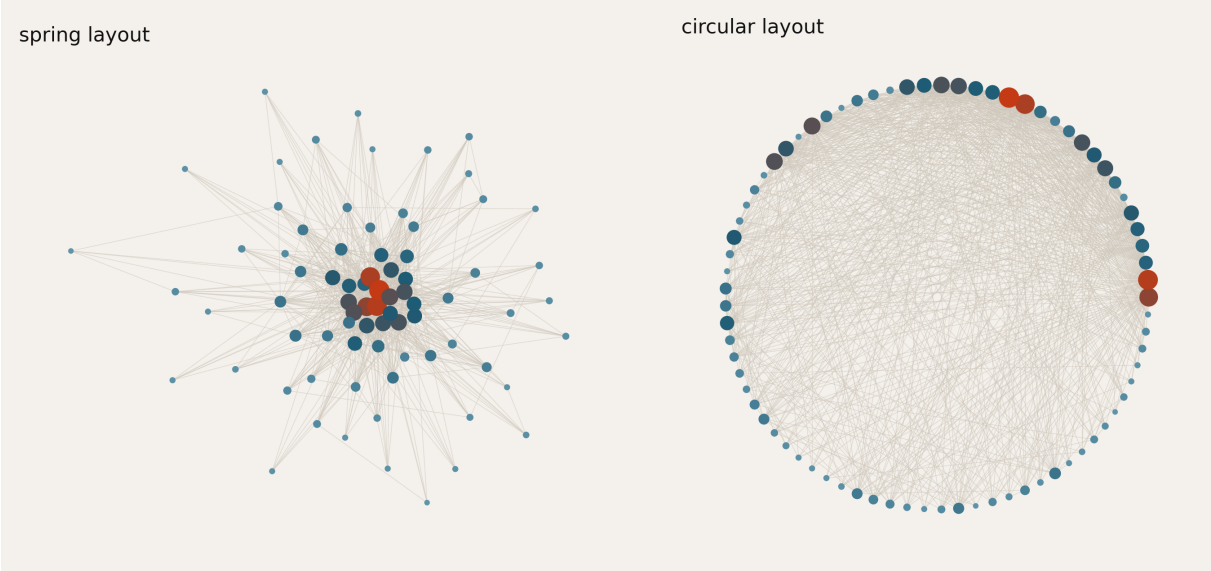


Figure 96: The same network in a spring layout (left) and a circular layout (right).

The spring layout treats every edge as a spring and every node as a charge that pushes the others away, and lets the system settle: the hubs, which have many springs, are pulled into the middle. The circular layout puts all nodes on a circle in the order they come, so the hubs sit on the rim with everyone else. The layout algorithm has parameters of its own as well, which make the picture looser or tighter.

In class someone remarked that the spring layout gives a better feeling of what belongs together. I am not saying one is good and the other bad. The point is that a layout is just some algorithm, and it says less about the data than it seems to. Each time you draw it, it looks different. A sentence like “she is on the edge of the network” is true in one panel and false in the other, so it is a sentence about the layout, sometimes no more than a quirk of the algorithm you chose. Say what the data says: the node’s degree, or how many steps it is from the busiest person.

5.4.2 The same network as a matrix

The same people, the same edges, as an adjacency matrix with the most connected people first (figure 97). A cell is coloured when the two mentioned each other, and darker the more often.

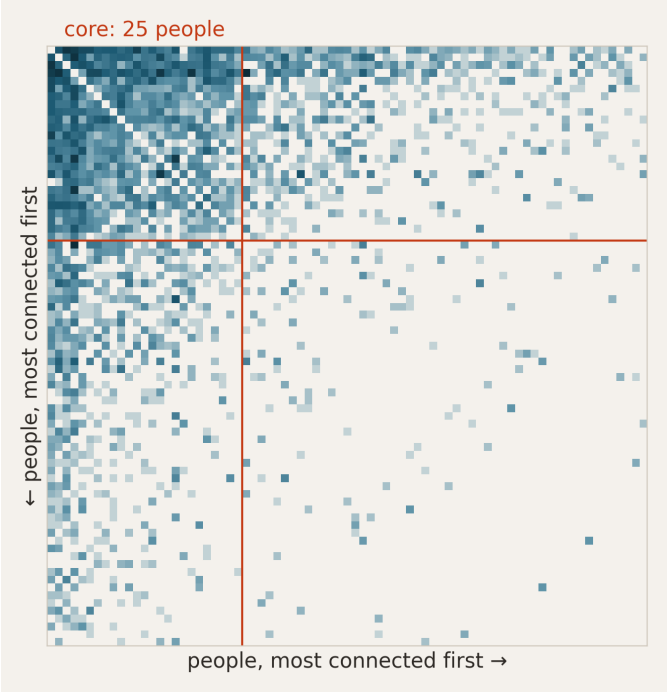


Figure 97: The mention graph as a matrix, people sorted by degree. The vermilion lines mark off the core.

Sorting the rows gives a pattern that is much harder to see in the drawing. The top-left corner is dark: the busiest people all talk to each other. The top rows and the left columns are long bands: the busiest people also talk to almost everyone else. And the bottom-right, the conversations among the less busy people, is nearly empty. You see which groups talk a lot to each other and which hardly at all.

The vermilion lines mark off the *core*, and where to draw them is not a matter of taste. The k-core of a network is the largest group in which everyone is tied to at least k others of that group. You find it by peeling: remove everyone with fewer than k ties, which lowers the count of their neighbours, remove whoever drops below k next, and repeat until nobody drops out (the notebook has a cell that does it). The highest k that leaves anyone here is 19: 25 people, each tied to at least 19 of the others, one big cluster. Inside the lines 91% of the pairs talk; their rows run on to 35% of everyone else; among the rest only 5% of pairs talk. That is the shape of a hub network, a channel where questions go to a few regulars. The spring layout let you guess it with its star; the matrix lets you read it off. All of that pattern comes from nothing more than sorting the columns.

5.4.3 Sometimes the drawing wins

The intercity network is sparse, each station has two or three direct connections, and it has a layout that is not made up (figure 98).

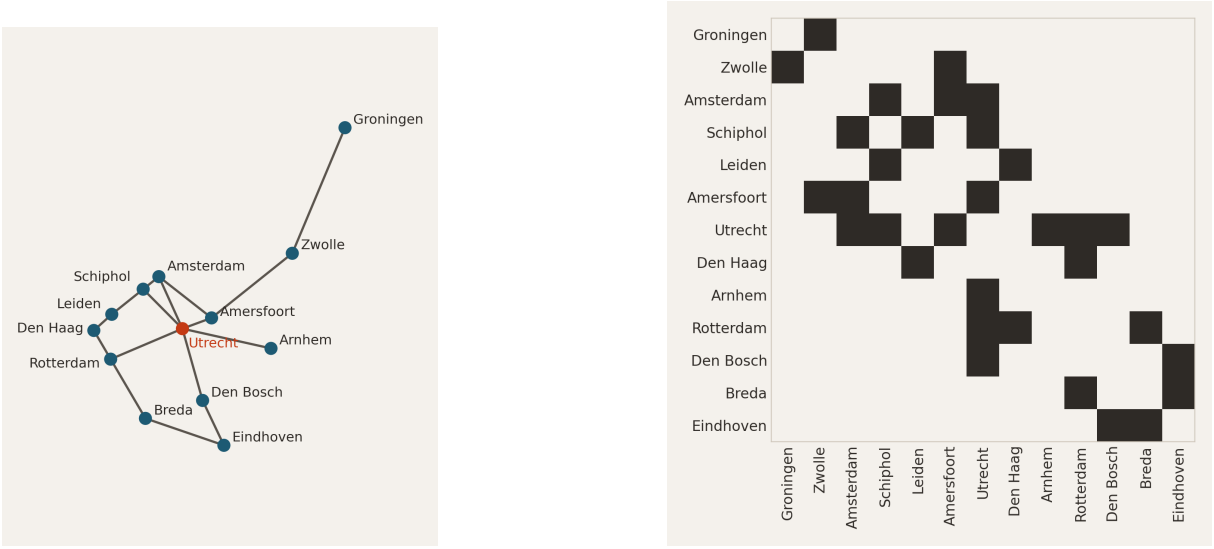


Figure 98: The intercity network on the map, and as a matrix with the stations ordered north to south.

I think most people prefer the map when they want to take the train. Technically the railways could print matrices like this one, but I doubt many travellers would be happy with it. Try it anyway: navigate the matrix from Zwolle to Eindhoven. From Zwolle you can go to Groningen and to Amersfoort; from Amersfoort to Amsterdam and Utrecht; but how do you get to Eindhoven? You would have to work backwards from Eindhoven's row. It is a real puzzle. On the map it takes a glance: Zwolle, Amersfoort, Utrecht, Den Bosch, Eindhoven. The matrix does show some interesting little clusters, but here the drawing shows them better too. For a small, sparse network with real geography, draw it.

5.4.4 Sometimes the matrix wins

The opposite case: the eight busiest people in the channel, and who names whom, with the direction kept (figure 99).

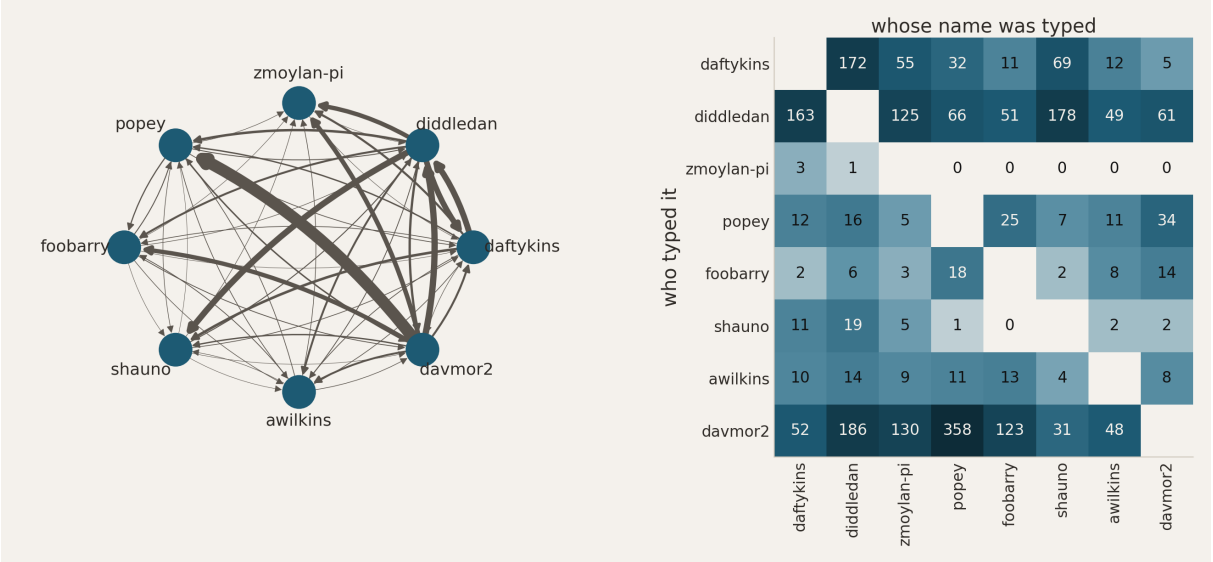


Figure 99: The eight busiest people: a ring of arrows, and a matrix with the person who typed the name as the row and the person named as the column.

As a drawing it is fifty arrows on top of each other, and pre-attentive processing quickly concludes: everyone talks to everyone, and two of them talk a lot. That is about where it stops. In the matrix, things stand out that you would not otherwise notice. One column is full and its row is almost empty: zmoylan-pi is named 332 times by the other seven and names them 4 times. A person people talk to, who answers without typing names. In the drawing you would have to see that in the direction of the arrows, and it does not stand out at all. You also see who names others fanatically, and which pairs talk to each other most, with a small asymmetry between them. A dense, weighted, directed network is a table of numbers, and a table of numbers is what a heatmap is for. Your WhatsApp group of six or eight friends is very likely this case.

5.4.5 A dashboard: ask your agent for one

The rule, the window, the threshold: every decision in this question is a number you would like to drag and watch the network answer (figure 100). In earlier years this lesson taught Streamlit, a Python package for building dashboards. While preparing the lesson it struck me that it is now much easier to ask an agent to build one, so I gave a subagent the prompt from the slide and looked at what came back. It is in the repository as `dashboards/lesson7-dashboard.html`.

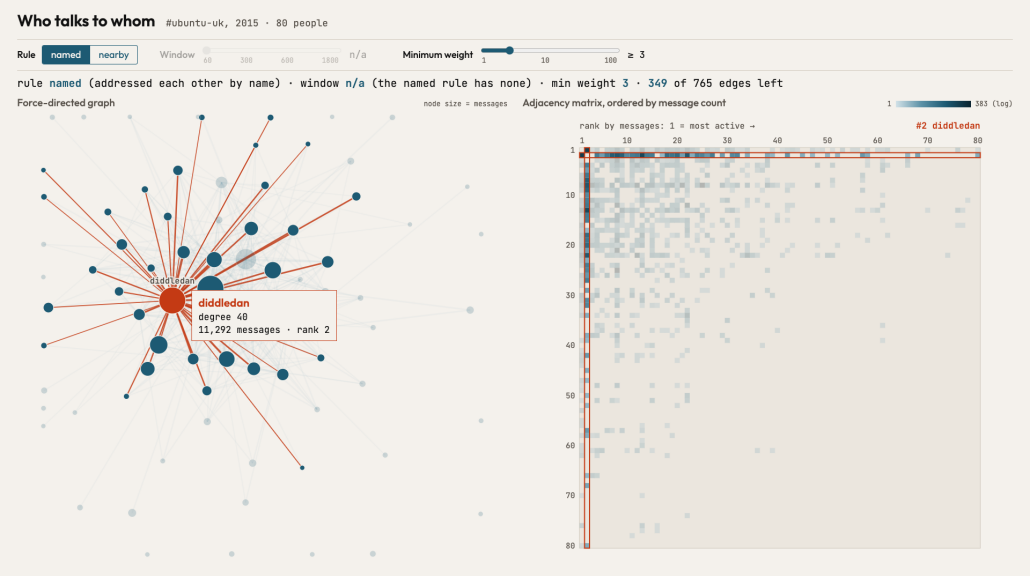


Figure 100: A dashboard built by an agent: a switch for the rule, sliders for the window and the minimum weight, the graph and the matrix side by side.

The prompt asked for a force-directed graph on the left and the adjacency matrix on the right, always the same edges; a switch for the rule, a slider for the window and one for the minimum weight; hovering a person highlights them in both views; and a caption with the rule, the window, the threshold and the number of edges left. Notice that it asks for both views side by side, which is this whole question. The division of labour matters: the network is computed in the notebook, where you can check it, and the page only draws the numbers. And check what comes back, because the agent will not: move the window slider and see whether the edge count changes, and make sure the decisions are in the caption, because a screenshot of a dashboard travels without its sliders.

If you are asking yourself what a good window is, this is how I would find out: interactively, turning the knobs and seeing what happens. It is a good way to explore the structure of your own data, of different topics too.

Again, concretely. Draw every network twice, as dots and lines and as a matrix. They both tell a story, and neither is good or bad in general. The drawing is best at paths and hubs, but its positions are invented by a layout, so never describe a node by where it sits. The matrix has no layout, never tangles and shows weights and direction, and once you sort its rows by degree or by group its shape becomes readable. A sparse network with real geography wants the drawing; a dense, weighted one wants the matrix. It is up to you to pick the one that tells your story best.

5.5 Can an algorithm find the groups?

Networks often fall apart into groups: friend circles, departments, factions. You can see that by eye, in a drawing as clumps with few lines between them, or in a matrix sorted the right way as blocks along the diagonal. With graphs the size of yours that still works: the small graphs of a chat you can simply look at, and say “there is a cluster”. But graphs can be very large.

An example from my own work. An international company had factories all over Europe, and machines in those factories that must not stand still: a machine that stops costs something like 200,000 euros of turnover a day. So every factory kept spare parts in stock. But a part's name and number came from the local supplier. The factory in France had its rings and springs from a French supplier, with a French description and a French article number; the factory in Greece had a Greek supplier who had simply started numbering at 1. Does the company buy the same parts all over Europe? Probably, but nobody had any idea. So the question was whether we could match parts that are the same, which gave a graph of some 80 million edges, full of clusters. Many clusters were neat. Some held 15,000 parts, and we knew those were wrong, because they contained both batteries and coffee mugs. Somewhere an error in the matching connected things that did not belong together. To find it, we computed betweenness: like the Medici, the parts in the middle of such a cluster are the ones that make it so large, and finding them showed which part tied the batteries to the coffee mugs. At that size you cannot look; 80 million nodes on a screen is a screen full of black dots. That is where algorithms come in.

An algorithm that finds groups you could already see is not redundant either: it is a confirmation, it makes the boundary precise, and it tells you who sits on it. The strongest case is when the groups also match something measured separately from the network.

5.5.1 A karate club that split in two

The most famous small network in the field (figure 101). In the early 1970s the anthropologist Wayne Zachary studied a university karate club for two years, and recorded which of its 34 members met each other outside the lessons: who trained with whom, who had lunch with whom. During that time a conflict grew between the instructor and the club's president, and the club split in two: the instructor started a new club, and every member chose a side. So this network comes with an answer that was not part of it.

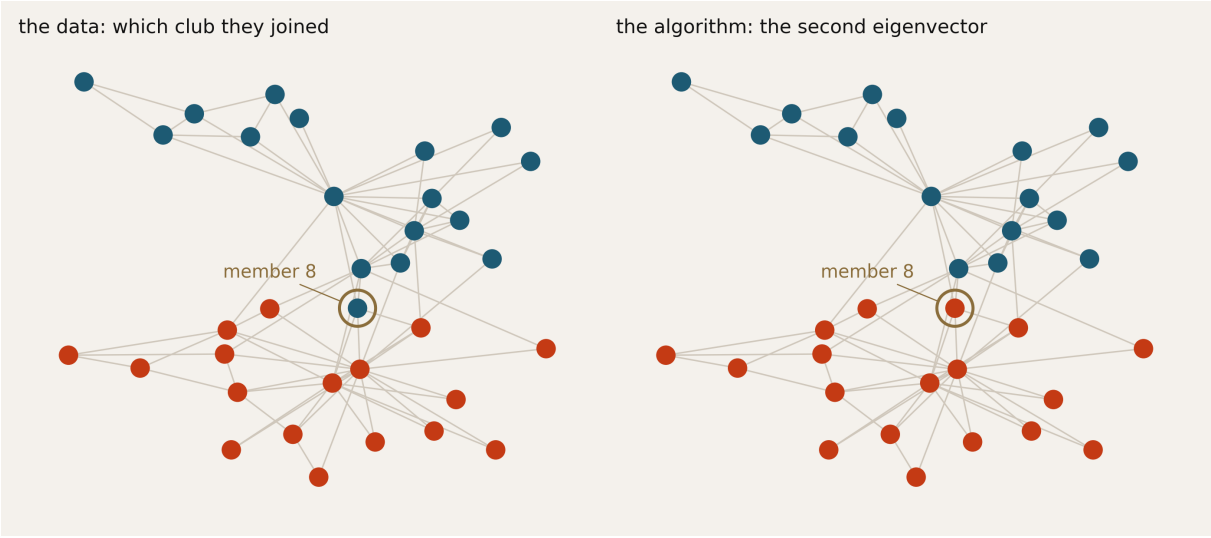


Figure 101: Zachary's karate club, 34 members and 78 friendships. Left: coloured by the club each member joined. Right: coloured by an algorithm that was given only the lines. Member 8 is ringed.

You can already see in the drawing where the dividing line will run. On the left, the data:

the club each member actually joined. On the right, a cut through the same lines, nothing else. They differ in one member: the algorithm puts member 8 on the president's side, and in reality member 8 stayed with the instructor. But member 8 also sits literally in between the two groups; I can vividly imagine the sleepless nights over which club to join. A student pointed out that there is an explanation outside the network, and Zachary's account gives it: member 8 was only weeks away from a black belt under the instructor. No algorithm on these lines could have found that.

5.5.2 One eigenvector predicts the split

How the cut is made (figure 102). You do not have to remember this; what matters is the idea. Write the club as a matrix. Besides the adjacency matrix A, which has a cell for every link, make the degree matrix D, which only has numbers on its diagonal: how many links each member has. Subtract one from the other:

$$L = D - A.$$

Why that works is too long a story for this course, but L, the *Laplacian* of the graph, has eigenvectors that describe the graph's shape, and computing them is a relatively simple piece of linear algebra. The first one tells you nothing for a network that hangs together as one piece, as this club does before the split. The second one, the *Fiedler vector*, gives every member a number such that members connected to each other get similar numbers. Split the members by the sign of that number and you cut the graph in two with few edges between the halves.



Figure 102: The karate club as a matrix. Left: members in random order. Right: sorted by the Fiedler vector.

On the left, member 1, member 2, member 3, in a random order: no structure. On the right, the same matrix sorted by the Fiedler vector: the matrix orders itself into two dense blocks on the diagonal, with few cells between them. That is what a group looks like in a

matrix, the same two clumps the drawing showed. And the cut agrees with history: 33 of 34 members land on the side they actually joined.

What I find interesting about it: there is no machine learning in it. Nothing to train, nothing to learn, no random process. You need the matrix, you compute its statistics, and they tell you how to order it, and then you have your groups. That makes it very cheap. The one drawback is that you need the whole matrix at once. If you are Facebook, with 700 million people, the matrix is far too big, and every time someone joins you have to start again. But for small, manageable graphs it is a very elegant method.

5.5.3 Louvain maximises modularity

The Fiedler vector cuts in two. For more groups the standard tool is *louvain*, named after the Belgian university where it was developed, and it optimises a score you should be able to read: *modularity*, Q:

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i \, k_j}{2m} \right) [\, c_i = c_j \,].$$

In plain words: the edges inside your groups, minus the edges you would expect there if the same nodes, with the same degrees, were wired at random. A_{ij} is the edge that is there, $k_i k_j / 2m$ the edge you would expect from the degrees alone, and the bracket only counts pairs in the same group. So $Q = 0$ means no more inside the groups than chance. The karate club divided into the two clubs it actually split into scores $Q = 0.36$.

Louvain searches for the division with the highest Q. It tries assigning members to groups: what if you are in group A and you in group B, and what if you move to B? Each node in turn joins the neighbouring group that raises Q most, until no single move raises it; then every group becomes one node, and it does the same again. It is a greedy search: it finds a high Q, not guaranteed the highest, and it decides the number of groups itself.

5.5.4 Louvain on the words: five topics

Back to the words of the chat from question two (figure 103). I gave louvain the network of 23 words and asked it to look for clusters.

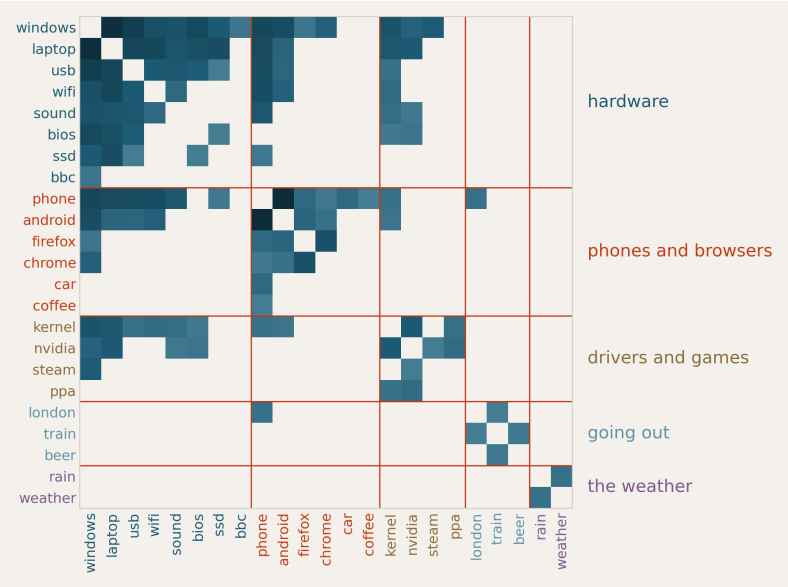


Figure 103: The 23 words as a matrix, sorted by louvain's five groups, each named beside it.

It finds five, with $Q = 0.20$, and the check is to read them: hardware, phones and browsers, drivers and games, going out, the weather. It is not perfect. Coffee and car sit with the phones and browsers, which looks wrong; BBC sits with the hardware. In class someone suggested that this may not be wrong at all: the edges are about what people talk about in the same conversation, and for this group of people it may well make sense. It might. The point is that you get a sensible division for free, one you would otherwise have to dig out yourself, and that you still read it before you believe it.

Again, concretely. Can an algorithm find groups? Yes. Look first: clumps in the drawing, blocks in a matrix sorted by the groups. The Fiedler vector cuts a graph in two, cheaply and without any training, as long as the whole matrix fits; louvain finds as many groups as raise modularity. The karate club is the good case: the picture, the algorithm and the split that actually happened all agree, and they were obtained separately. Most networks do not come with that third thing, and then you need the checks of the next question.

5.6 Are the groups real?

This is something we have seen again and again, in all seven lessons: a method that always finds groups. Last lesson you could let k-means loose on a cloud and ask for three, and you got three groups. Louvain always finds clusters too: every network has a division with an optimal Q, and louvain searches until it has it. So a method that always finds groups is no evidence that there are any. Three checks:

- **Nothing.** Run the same method on the network rewired at random, every degree kept. That is the null model.
- **Triangles.** Friends of friends make clumps that beat that null without any groups behind them.

- **Another period.** Real groups come back when the same rule runs on another part of the data.

5.6.1 Louvain colours every network into tidy clumps

An experiment where we know the answer, because we built the network (figure 104).

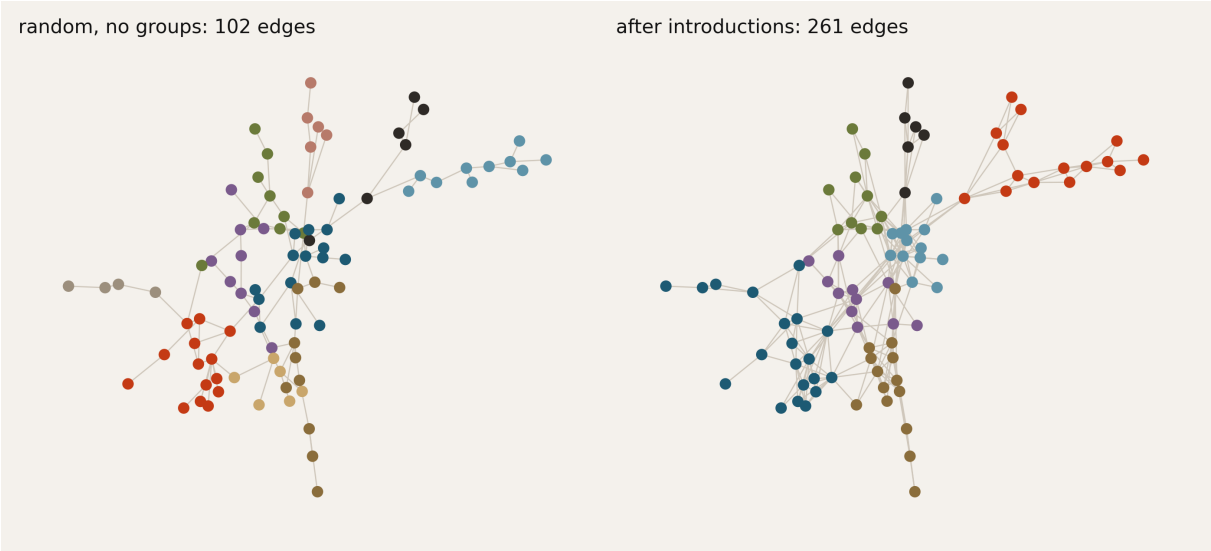


Figure 104: Louvain's communities in two networks without groups. Left: random friendships, 102 edges. Right: the same people after friends introduced their friends, 261 edges.

On the left, a hundred strangers with random friendships: no groups, by construction. Louvain colours it into communities anyway, and some of them, like the clump at the top right, look like a group to us as well. I had a discussion about this with Claude while making the figure: even though the network is random, I still find some of those clumps quite defensible as a group. You can argue about whether one clump is really a different group from the next. But that is the point: louvain and our eye are fooled by the same thing, edges that happen to fall close together. You cannot trust it blindly.

On the right, the same people after one round of introductions: everyone introduces pairs of their friends to each other. That is triadic closure, done on purpose. There are still no groups; nobody chose a side. But louvain's clumps look tighter and more convincing than before.

5.6.2 Is Q more than chance? Rewire and compare

Louvain gives a Q for every network, random ones too, so a Q on its own says nothing. You need to know what Q the same network would get if it had no groups. The test: rewire the network at random, in a way that keeps one thing fixed: everyone keeps their number of links. If you talk to five people in the network, you still talk to five people in the copy, only five different ones; the lines are moved at random. Do that twenty times, run louvain on each copy too, and see where the real network lies (figure 105):

$$z = \frac{Q - \text{mean}(Q_{\text{rewired}})}{\text{sd}(Q_{\text{rewired}})}.$$

Real groups should beat their copies by many standard deviations.



Figure 105: Modularity of five networks (vermilion) against twenty rewired copies of each (grey).

The karate club: $z = 11$, far above its copies, which makes it very unlikely that its groups are a pattern found by chance. The Florentine families: $z = 0.6$, hardly different from their copies, so I would distrust the groups louvain finds there much more than those of the karate club; the Medici network was a broker network, not a set of factions. The random graph: its Q of 0.70 is the highest on the plot, and the test rightly says it is nothing, $z = 1$. So far the test works.

The catch is the fourth row. The same random graph after introductions scores $z = 25.6$, more than twice the karate club, and we know it has no groups. The reason is what the shuffle destroys. It keeps everyone's number of friends, but not the triangles, and triadic closure is a natural process that made a lot of them. Randomising breaks that structure, so the real network stands far above its copies. Any network with many triangles passes this test, and social networks always have many triangles.

5.6.3 Finding hubs typically needs support from additional data

The matrix as a check on communities (figure 106). Sort it by louvain's communities, the most connected person first within each, and draw the borders between communities.

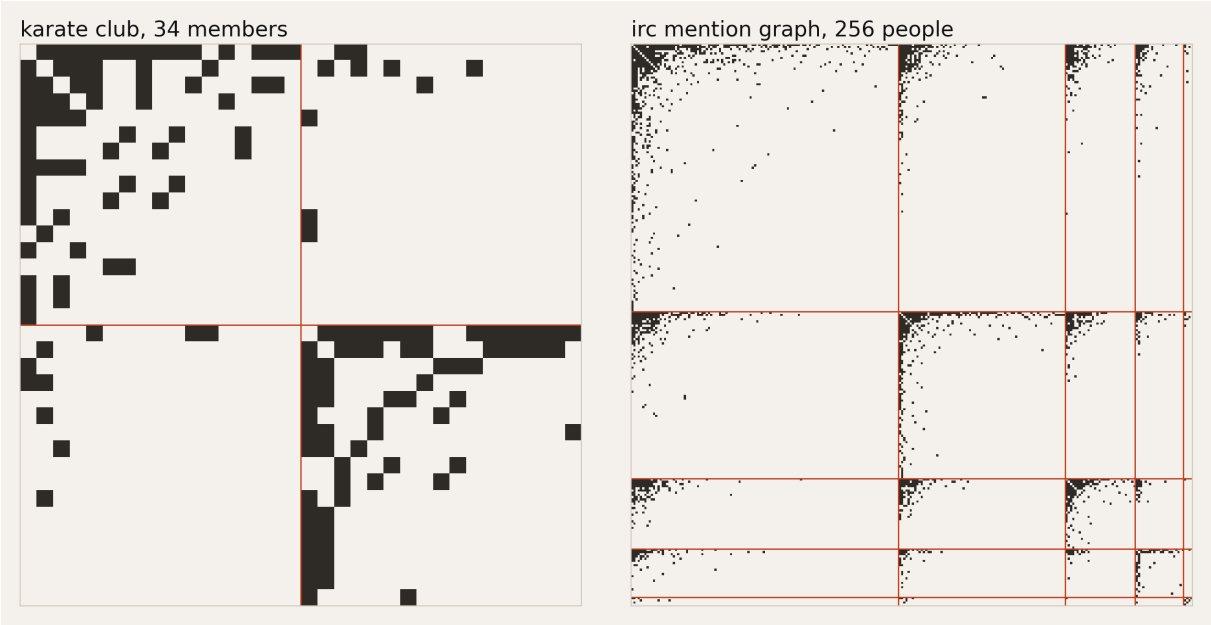


Figure 106: Matrices sorted by louvain's communities. Left: the karate club. Right: the IRC mention graph, 256 people.

In the karate club, two squares that are filled: the hubs' rows stop at the border, and inside a block the other members know each other too. That is what a group is: people who know each other, more than they know the rest. The chat looks different. Each community is a corner with one dark row along its top and one dark column down its left, and very little inside: a hub and the people who address the hub, who do not talk to each other. Louvain has to cut a network of people who all talk to the same few regulars somewhere, and it cuts where the hubs' circles overlap least. Whether such a block is a group needs support from something other than the cut itself.

5.6.4 Do the groups come back?

The oldest check in science: does it come back? Find the communities in the chat for January to June, find them again for July to December, and compare who ends up together (figure 107).

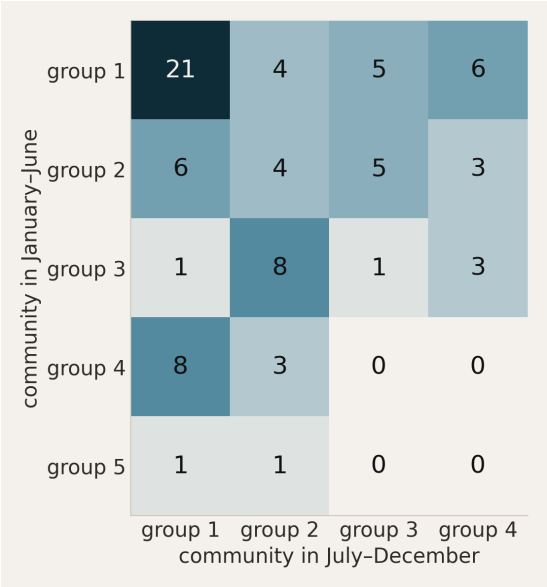


Figure 107: The 80 people active in both halves of 2015: their community in January to June against their community in July to December.

If the groups were stable, each row would have one dark cell. Only the first has anything like it; the other groups spread over the second half of the year, and a large part of the clusters disappears. The adjusted Rand index, a score that is 1 for the same division and 0 for chance, is 0.08. That is an indication that these clusters are not stable.

In class someone made a good point: friend groups change over time too, and that does not mean they were never a group. Exactly. It could be chance that the groups changed, or the groups may simply have changed. This is the situation with all the statistics we do in this course. You see patterns, and you keep asking whether this is chance or not. You can make it more plausible that something is not chance, but you can never exclude it entirely; there is never one hundred percent certainty.

Again, concretely. A coloured drawing and a high Q prove very little: give louvain a random graph and it still finds colours and groups. That it finds something is guaranteed. What you can do is apply tricks, shuffling and comparing, to make it plausible that the structure you found is not random. Compare with a rewired null, and remember that it does not control for triangles, which social networks are full of. Look at the matrix sorted by the communities: a real group is a filled square, not a hub with its audience. And see whether the communities come back in another period or with another rule. The karate club passed all three. The chat passed none convincingly, and saying so is a valid result.

5.7 A network analysis in four steps

The plan for your plot this week, which is also what the goad coach walks you through when your question is about connections:

1. **Brainstorm.** Try several kinds of node and relation, not only people: topics, places, anything. Each is a rule with a window and a threshold; pick the one your question needs.

2. **Draw both.** Put the drawing and the matrix side by side, and let the clearer one carry the point. If you have a drawing and think "it's interesting, but what story is in it?", see whether the matrix adds something.
3. **Measure what it adds.** Who or which topic has the highest betweenness? Where is closure high? Put each measure next to the column you had, as a scatter, not one number.
4. **Test against nothing.** If you find groups, set them against what you would expect if it were random: a null baseline, so you know you are not hunting ghosts. If the structure disappears when you shuffle, it was real. And see whether it comes back in another period.

5.8 The six questions, answered

Question	Answer
What is a graph?	Things and their relations, $G = (V, E)$. You choose both.
What are the nodes and edges in your chat?	People, places, things; every edge a rule with a window.
What does a graph measure add?	Brokers and open circles, or a column you had. Draw the scatter.
Graph or heatmap?	Both, side by side. Sparse wants the drawing, dense the matrix.
Can an algorithm find the groups?	Yes, best as confirmation of what you saw.
Are the groups real?	If they beat a null, are not just triangles, and come back.

Notebooks 07.1 to 07.3 work through the questions: 07.1 the IRC channel, the two rules, the window, a dashboard from your agent, and your own nodes and edges; 07.2 degree, betweenness, the clustering coefficient and triadic closure, a small world, Spearman against a column and across windows, layouts and the matrix; 07.3 the karate club, the Fiedler vector, louvain and modularity, the null model, the introductions experiment and the chat's communities across two halves of the year. For your project: write down at least two networks your chat contains before you build one, draw every network as a drawing and as a matrix, report a measure only with what it adds over a column you had, and run the checks before you call anything a group.

What to hand in for the process step: not your whole project. Your visualisations with a piece of text around them, a short introduction and conclusion, and a link to your GitHub repository are enough. For the description of your code, the atlas picture will do, and it may take two or three pages as an appendix to your report. You can also let a GitHub workflow run atlas on every commit and publish the picture in your README or as a GitHub Pages site; then put a clear link to that walkthrough of your code in your report (section 6.8).

5.9 Feedback on your network plots

In class we discussed several of your plots. The same points came back, so I collect them here.

Let the clusters come from the graph. One plot made a topic map of a household chat: word groups per theme, in which home, food and cooking and the dogs connect everything, and also have the highest betweenness. A nice map of what the conversations are about, and of what relates to what. Pre-attentively it is somewhat messy, though. The word groups were made by hand before the graph was built, while that clustering could also come out of the graph itself. Consider showing the raw data, the original words as nodes, and pointing out the clusters with colour, found by louvain. Or colour the few nodes with the highest betweenness, and show that they all sit in the category home. And ask what the size of the dots and the thickness of the lines add: if the story is that home is the central theme, extra colours and decorations may distract from it.

When the graph is clearer, use it. Another student drew the same network twice: people mentioned in the chat, connected when both were mentioned within thirty minutes, as a graph and as a heatmap. In the graph the two friend circles fall apart naturally into left and right, and the one line between the two mothers survives when you move the threshold slider of the dashboard: a small, clear story. A few people, like one friend mentioned with everyone, overlap both sides. Here both the class and I found the graph the stronger of the two. The heatmap in grey tones makes it hard to see anything; with another colour scale, the contrast between the many names that are rarely mentioned and the few outliers would come out.

Asymmetry belongs in a matrix, and turn the window. One plot of a family group showed three people in the middle. Two of them have the same influence in different ways: one is a connector who often names others, the other sends many more messages for the same connections. That asymmetry does not show in a graph; a matrix would show it. The difference between the two also maps onto the difference between weighted degree, which counts messages, and plain degree, which counts people. The student chose not to look at windows, because the story was about who sits at the top of the family, not how fast people respond. A window may be chosen to fit the question, but then ask what happens to your story when you turn it. Here the story is about betweenness, so rank people by betweenness and turn the window: does the top three stay? According to the student, completely different people come up. That is exactly the interesting question to investigate.

Without a mechanism, "more often than chance" adds little. One plot of a sports team connected two members when one replied within sixty minutes, counting consecutive messages from the same sender as one. The result is an almost perfect star with the team captain in the middle, with surprisingly few other lines. Two orange lines were marked as more frequent than chance. But things sometimes happen more often than chance; if you cannot name a mechanism for why these two members would interact, the orange lines tell you little. The story here is that you can see the team captain in the structure of the graph. Focus on that.

Check what a long window does to your measure. One student asked a lovely question: who introduces a new word in the chat, and who takes it over? Who leads and who follows? The window chosen was a week, and that raises the window question again.

Suppose I introduce a word in week one and you take it over; in week two you are the first to use it on Monday. By this measure you have now introduced it, and I am following you. One way out is to look over the whole period: take the 400 or 1,000 unique words, and for each, who used it first and who used it after. Then you can count, and also see something else that is interesting: if I start a new word five times and you never take it over. Another student pointed out the other side: you cannot make the window too large either, because a word that comes back three months later has hardly been "taken over"; there has to be some closeness in time. That is exactly the knob this lesson is about. Turn it and see what happens; if it has no impact, you are free to choose. Either way, say in words why you chose it ("after three months I no longer call it taking over"). A window that adds noise is not necessarily wrong: you may say that you know it adds noise and find that acceptable.

6 Reading code with atlas

This course asks two things of your code. It has to make the plots in your report, and you have to be able to explain how it does that: in the final assignment you show me a picture of your code and walk me through it. This section explains why I ask for a picture and not for the code itself, and what atlas, the tool that draws the picture, shows you. The argument is the same one this course makes about data: a good visualisation lets you see in a second what a table of numbers hides, and code is no different.

6.1 The problem with reading a lot of code

Until recently, writing code was the slow part. Reading it kept up, because a person can only write so much in a day. With a coding agent that balance is gone. An agent writes a function in seconds, a feature in minutes, and several agents can work on several projects at once. Somebody still has to look at all of it and decide whether it is any good: you, for your own project; a senior developer, for a team (figure 108). That is my own situation: at the province of Utrecht six people write code with Claude in a project I am responsible for, and in this course fifteen of you do. Even if I go through a code base quickly, it takes me half an hour. Writing has become cheap. Reading has not.

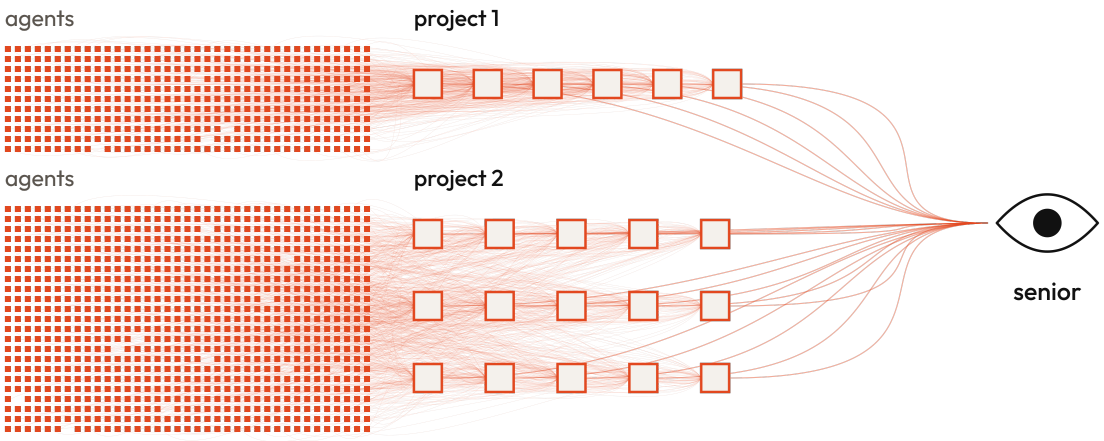


Figure 108: Many agents, a few projects, one person who has to oversee all of it.

Put the times next to each other (figure 109). Reading one function carefully takes about a minute. Reading one pull request, a single proposed change, takes the better part of an hour. Reading the 179 commits an agent can produce in a project takes a day, and by then there are more. And all of that only tells you what each piece does. What you actually want to know is something no single line tells you: how the pieces fit together. To see that from the code, you have to read it all and build the structure in your head, one line at a time.

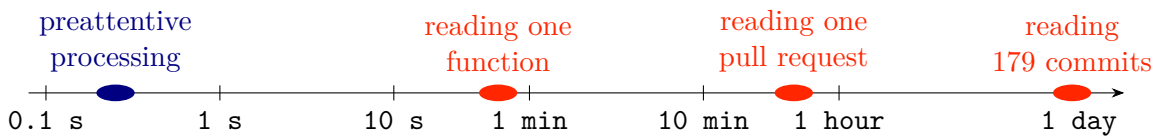


Figure 109: How long it takes to see something, against how long it takes to read it. The axis is logarithmic: every tick is ten times longer.

There is a second problem: the output does not tell you whether the code was good. Take a program that fetches the monthly sunspot counts since 1749 from the web, keeps them as a CSV file, takes the mean over thirteen months and draws both (figure 110). I wrote two programs that make exactly this figure: one a sloppy script, one typed and tidy. The figure cannot tell them apart. Neither can your tests: they check that the figure comes out, not how. And neither can a linter: ruff and ty look at lines and types, and every line of the sloppy script can be fine on its own. What is wrong with it is the way the lines are connected.

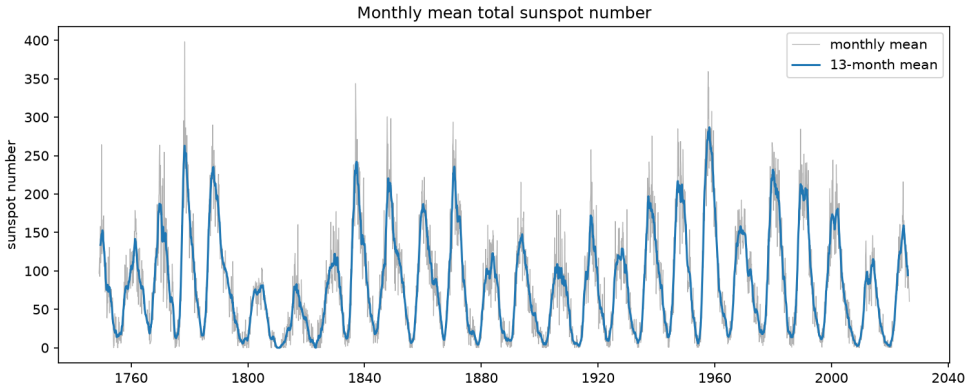


Figure 110: The monthly sunspot number since 1749, with its 13-month mean. Two very different programs make exactly this figure.

What "the way the lines are connected" means is easiest to see. Figures 111 and 112 are the two programs as pictures of their structure, the way atlas draws code. Before you read a label, just look: one falls apart into pieces, the other runs from start to finish. Which one would you rather change next month? Section 6.3 explains how to read them.

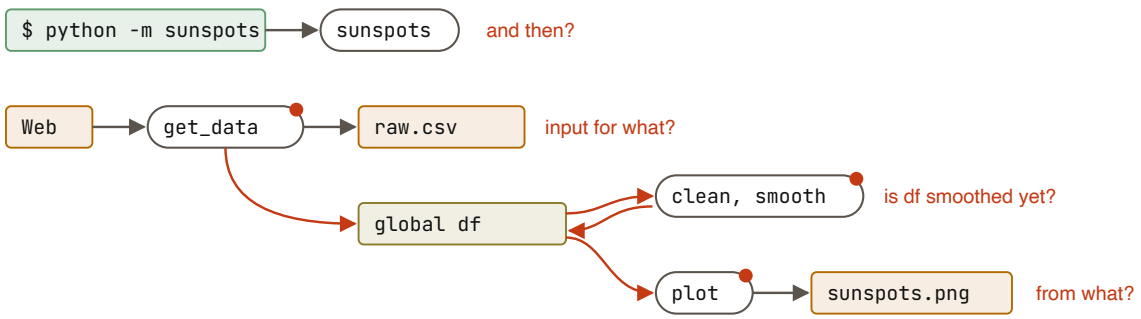


Figure 111: The sloppy script: loose fragments, each leaving a question open. A red dot is a question atlas asks (section 6.3). Simplified from atlas's picture, to show the principle.

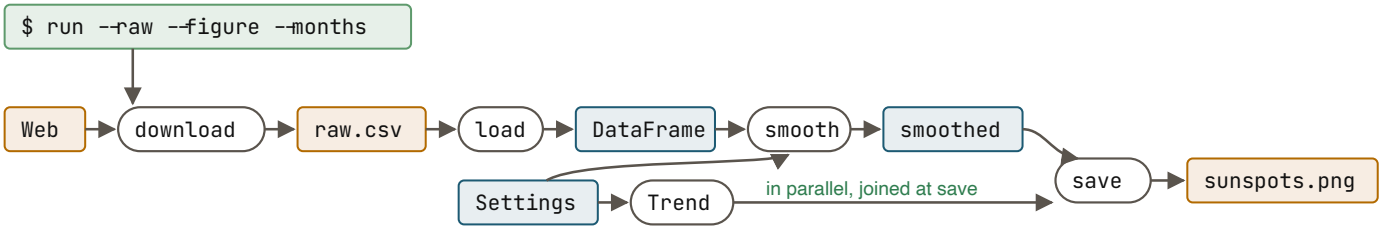
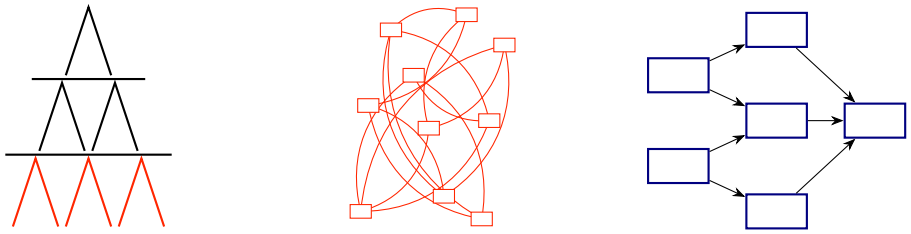


Figure 112: The typed program: one flow, from the command to the figure. Simplified from atlas's picture, to show the principle.

6.1.1 Bad code is a shape

What makes code hard to maintain is rarely one bad line. It is a shape: the way the pieces depend on each other. Figure 113 shows three.



house of cards spaghetti maintainable

Figure 113: Three shapes code can have.

A *house of cards* stands, until you touch it: every layer rests on the one below, and changing a card at the bottom brings everything above it down. *Spaghetti* is code where everything reads and changes everything else, so that nobody can say what a change in one place does elsewhere. *Maintainable* code has a direction: data comes in, goes through named steps that each take something and give something, and comes out. You can replace one step without touching the others. Notice that none of these is a property of a line. They are properties of the connections, and connections are exactly what you lose when you read code line by line.

6.2 Why a picture works better

Look at figure 114. You found the red square before you had decided to look for it. That is *preattentive processing*, which we used in lesson 5 to spot the weekly cycle in the COVID data: in the first fraction of a second, before you read anything, your visual system has already picked out what stands out. It does that for the whole field at once, so it takes the same time with 160 squares as with 10.

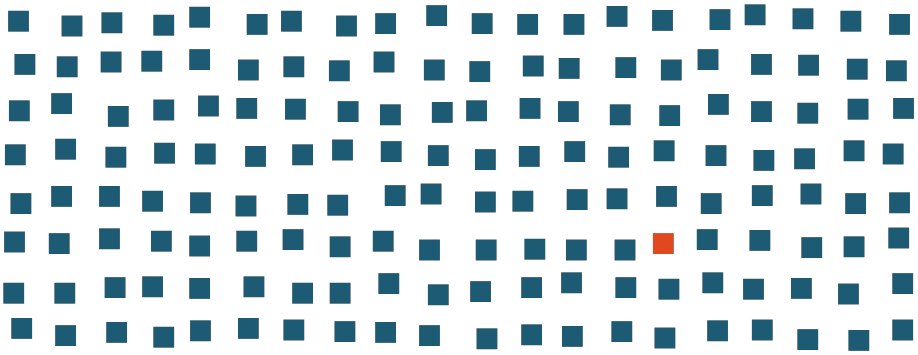


Figure 114: One square is not like the others.

Reading works the other way. You read in series, one word after another, and the time grows with the amount of text: twice the code, twice the reading. Seeing works in parallel. That is the difference between the blue dot and the red dots in figure 109, and it is the whole reason this course exists: a good plot shows in half a second what a table of numbers hides. So if the structure of your code is what you want to judge, turn the structure into a picture, and let your eyes do what they are good at: spotting the box that does not belong, the arrow that goes back, the place everything points to.

You don't need to know the rules behind the picture to use it. Show people a set of faces and ask which ones they like, and many will pick the same faces. Those faces turn out to follow certain mathematical proportions. Nobody who picked them knew that; their eyes knew what a well-formed face looks like. A picture of code works the same way. Atlas checks eleven design principles, built on 31 notions from mathematics (section 6.4), and you don't have to understand them all: a consistent design gives a calm picture, and a messy one looks messy. When something odd pops up (a box you didn't expect, a line that runs back), ask what it is doing there. In my own projects it quite often turns out to be a bug.

6.3 What atlas draws

Atlas[11] reads your code and draws it as two kinds of thing. *Places* are where data rests: a value in memory (a DataFrame, a `Settings` object), a file on disk, a table in a database, a page on the web, a program that runs, the screen, or a global variable that a module keeps. *Functions* are what carries data from one place to the next. A *route* is what one entry point does, a command you run for instance, drawn from left to right: what it finds on the left, what it leaves on the right, and the functions in between, in the order they run.

A picture you can trust. Why not simply ask your agent to draw a diagram of your code? I tried that. Claude made a beautiful HTML page with a flowchart that looked exactly right. Then I read the code, and the diagram turned out to show what I hoped the code did, not what it did: it had made something to make me happy. An LLM is a fuzzy calculator: very powerful, but it doesn't give the same answer twice, and it can invent. A picture you use to check code must not invent anything. Atlas therefore contains no

[11] `https://github.com/raoulg/atlas`. The tour at `https://raoulg.github.io/atlas/` shows every step on a real example.

language model at all. It reads the code, follows the same rules every time, and draws what is there. The same code always gives the same picture.

6.3.1 The same figure from two very different programs

Now read the two sunspot programs of figures 111 and 112. Both reach the same places: the web, the raw CSV, the figure. The difference is what the picture can tell you about the way between them.

The sloppy script (figure 111) falls apart into fragments, and each one leaves a question open. The command starts `sunspots`, and the picture stops there: what happens next is not in any arrow. `get_data` writes `raw.csv`, but nothing reads it, so you can't tell what it is input for. Then every step reads and changes one global DataFrame: `clean, smooth` takes it and puts it back, `plot` takes it too. Is the frame `plot` draws already smoothed? The picture can't say, because there is no order: you only know if you know that the steps happen to be called in the right sequence. That is the spaghetti of figure 113, in a script of a few dozen lines.

The typed program (figure 112) is one process with a head and a tail, and you can read its story off the picture. The command starts it. `download` fetches the data into `raw.csv`, `load` turns that into a DataFrame, `smooth` turns that into a smoothed one, using the number of months from the settings. In parallel, the settings make a `Trend`, the object that knows how to draw. The two lanes join at `save`, which draws the smoothed frame into `sunspots.png`. Every question the first picture left open is answered by an arrow: what the CSV is for, which frame is plotted, in what order things happen. Same output, same libraries. The difference is entirely in the connections, and the picture shows it in a second.

6.3.2 A red dot is a question

Some boxes carry a red dot. A dot is a question atlas asks the author: something in the code does not follow one of the eleven design principles atlas checks (table 9). A step reads and changes a global; a function reads a file whose path it chose itself, so nobody calling it can tell where its data comes from; a function's signature doesn't say what it gives. `atlas-code check` lists the same questions as text, each with the principle, the place in the code, what atlas saw, and a question for you.

A red dot is not a verdict. Atlas coaches; it doesn't fix. Sometimes the answer is to change the code, and the dot disappears. Sometimes the code is right as it is, and then you write that decision down beside the code, with your reason. What I am interested in is that you know which it is, and why. That is the same thing I ask of every number in your report.

Atlas is also not a linter. ruff and ty own the level of lines: style, an unused import, a type that doesn't match. Atlas owns the level above: which steps there are, what travels between them, where data rests. If you make three plots that are really three separate things, the picture should show three separate things; if they come from one preprocessing step, it should show one flow. A linter cannot know which one you meant. Only you can.

6.4 The mathematics behind the picture

Why would a picture of code show whether the code is good? Because good practice in programming is a set of consistent habits, and mathematics is a language for consistency. Atlas reads your code as a small piece of mathematics and draws that. If the code follows the habits, the mathematics is simple, and so is the picture. This doesn't depend on the language: Python, Rust and TypeScript are drawn by the same rules.

The mathematics comes from four fields: category theory (how the parts of a program connect), topology (what stays the same when you zoom in or out), type theory (what goes into a function and what comes out) and graph theory (what leads to what). Atlas's documentation names 31 notions from these fields, each with what it is, what it is in atlas, and where it lives in the code[12]. A test checks each of them on every change to atlas. Atlas is still in beta and there are known exceptions, which are tracked in the open as issues.

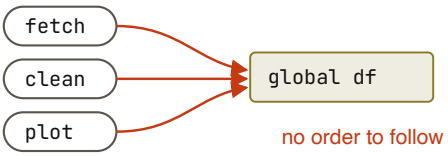
You don't need the theory to use the picture, just as you don't need the proportions to recognise a well-formed face. But six of the notions explain most of what you see. For each, the left picture is how atlas draws it, and the right one what a picture would look like without the principle.

6.4.1 A category: functions are arrows

The places are the objects of a category, the functions are its arrows, and calling one function after another is composing arrows. A route is the composite from the place where a program starts to the places it reaches.



In atlas. Every step hands its result to the next. You can follow the arrows from where the program starts to where the data ends up.



Without the principle. Each step reaches into one shared value. Nothing is handed on, so there is no route: you cannot tell what comes first.

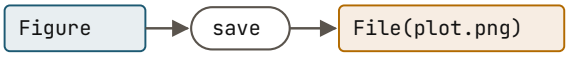
Figure 115: A category.

The right picture is the sloppy sunspot script. Its steps are functions, but they are not arrows from something to something: they all reach into the same global. That is why its order lives only in the head of whoever wrote it.

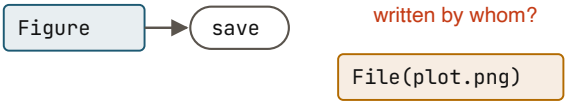
6.4.2 Effects belong to the arrow

What a function does to the world is part of the function. A file it writes is as much a part of what it gives as a value it returns; a file it reads is as much a part of what it takes as a parameter.

[12] `docs/reference/mathematics.md` in the atlas repository; the tour at `https://raoulg.github.io/atlas/#math` has a picture for ten of them.



In atlas. The file a step writes hangs on that step. You see who writes it.

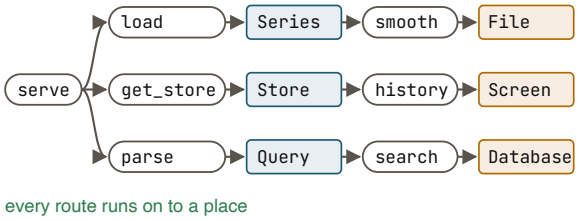


Without the principle. The file is there, but no arrow leads to it. Something writes it, and the picture cannot say what.

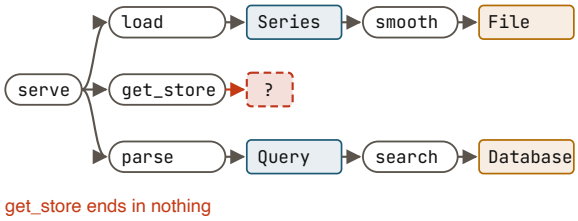
Figure 116: Effects belong to the arrow.

6.4.3 Signatures say the arrow

From type theory: a signature says what goes in and what comes out. A function's type annotations are the arrow's two ends, and they let atlas follow a value from step to step.



In atlas. `get_store` says it gives a `Store`, so the picture follows the value to where it goes, like every other route.



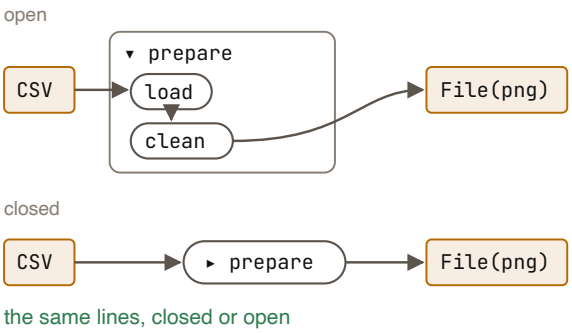
Without the principle. Without a return type, nobody knows what `get_store` gives, and its route stops where the others go on.

Figure 117: Signatures say the arrow.

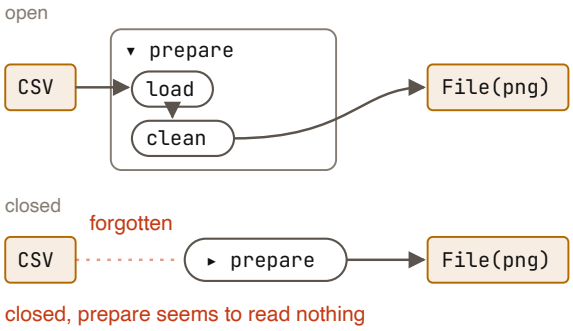
This is why atlas asks for type annotations, and why ty is in your setup from lesson 1: a function without a return type is a route that stops.

6.4.4 A quotient: zooming out is gluing

From topology: closing a box glues everything inside it into one point. A real overview must not lose anything when you zoom out.



In atlas. Closing a box glues its insides into one. Every line in and out stays, so the closed box still says what it reads and writes.



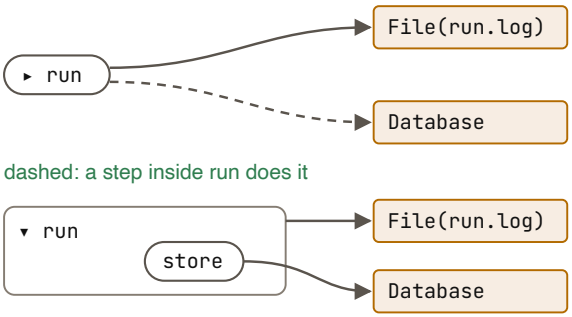
Without the principle. If zooming out could forget a line, a closed box would hide what it reads, and no overview could be trusted.

Figure 118: A quotient.

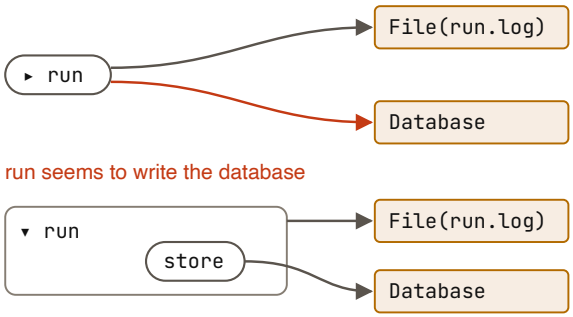
This is what makes the big picture of a large project trustworthy: you can close a whole package into one box and it still shows every place it reads and writes.

6.4.5 Reach: what a step does, and what a step inside it does

From graph theory: a line from a closed box can mean that the function does it itself, or that a function it calls does.



In atlas. Closed, a solid line is what `run` does itself, a dashed one what a step inside it does. Open it, and the dashed line turns out to be `store`'s.

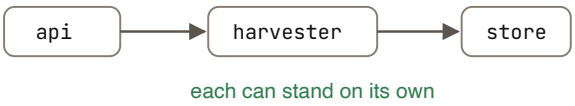


Without the principle. Drawn alike, the closed picture says `run` writes the database itself. You look for the write in `run`'s code, and it is in `store`.

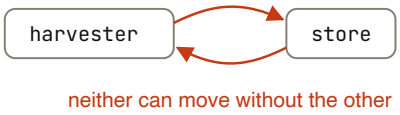
Figure 119: Reach.

6.4.6 One direction

Also from graph theory: packages depend on one another in one direction, without cycles.



In atlas. Packages depend in one direction, so each can be read, tested or replaced on its own.



Without the principle. Two packages that reach for each other have no order: you cannot take either apart.

Figure 120: One direction.

This is the house of cards and the spaghetti of figure 113 again, now as a rule. (It is about packages: inside one route a line may run back, where a step hands its result to the step that called it.)

Four more in the tour. A *pushout*: the same meaning gives the same picture, so an option the caller can change is one box whether Python, Rust or TypeScript spells it. *Yoneda*: a thing is what it connects to, not what it is called, so a function you only renamed shows as unchanged when you compare two commits. A *partial map*: a design that is not built yet is part of the picture, drawn dotted. And *Liskov substitution*: a subclass keeps its base's promise, so it fits wherever the base is expected.

6.4.7 Eleven questions atlas asks

On top of this mathematics, `atlas-code check` asks eleven questions of your code, one per design principle (table 9). Each is stated once, on the graph, for every language; a language decides how much of it can be traced. Rust's compiler already forbids some of the careless forms, so there atlas has less to ask than in Python. Each principle has its own section in `docs/reference/principles.md`, with what it means in Python, Rust and TypeScript, and your agent can fetch that section for you through atlas's MCP tool `explain`.

Id	Principle
`doors`	A package names its doors, and each door says what goes in and what comes out.
`sums`	A sum type is unwrapped where it arises.
`overrides`	An override keeps the promise of its base.
`places`	Library code does not decide where data lives.
`changes`	A change to an input shows in the signature.
`gives`	A function whose value is used says what it gives back.
`choices`	A choice is made once, in a place with a name.
`nothing`	A call leaves something behind.
`layers`	The packages of a part depend on one another in one direction.
`lens`	What a lens puts, it gets back.
`prophecy`	Code that is built does not lean on code that is not.

Table 9: The principles behind the red dots, in atlas's own words.

Three of them you will meet most in your own project. `places`: where data rests is decided where the program starts, in your script, CLI or notebook. A function that reads or writes

a file takes the path as a parameter; the script builds it (`Path("data") / "raw.csv"`) and hands it in. That is why the typed sunspot program takes its CSV and its figure as options of its command (`--raw`, `--figure`), while in the sloppy one `get_data` and `plot` choose their own files. `changes`: a function that changes a DataFrame it was handed has an effect its caller cannot see, and in Python no signature can say so. Either the object changes itself through a method of its own, or the function leaves its input alone and hands back a new value, as `sorted(xs)` does next to `xs.sort()`. And `choices`: when the same choice is tested in several places (`mode == "weekly"` here, `mode in {"weekly", "daily"}` there), every new case is a change in all of them. Make the choice once, in a place with a name, for instance a dictionary from each case to what it means. Each finding ends in a question, and the answer is yours.

6.5 Working with your agent on the same picture

Your coding agent lives in text, and you are much faster with images. Atlas gives the two of you the same picture. Register it once with your agent (`claude mcp add --scope user atlas -- atlas-code mcp` for Claude Code), run `atlas-code serve` in your repository and open the page it prints. Then you can ask "where does the smoothing happen?" and the agent points at the box on your page, instead of describing it. You can write a note on a box ("what happens if the data on the web changes?"), and the agent reads it and answers on the box. And when you ask for a change, the agent can draw it first, beside the code as it is, so you see what it would add before a single file changes. You say yes to a picture, not to a paragraph of text.

This matters for the same reason as everything above. You will write much of your code with an agent, and the danger of that is a fine outside and a mess inside: the plots come out, and nobody, you included, knows how. Looking at the picture together is a way to keep the inside something you understand.

6.6 A new level of abstraction

Programming has moved up a level before. In Fortran, in the 1960s, you allocated memory yourself and built everything from scratch. Python manages memory for you, and nobody misses doing it by hand. Working with an agent and a picture of your code is a shift of the same kind: you spend less time on syntax and more on design, on which steps there are and what travels between them. What has to stay is the determinism. Python's memory management does the same thing every time; an agent does not. That is why the picture has to be deterministic: it is the part that keeps the fuzziness of the agent under control, while you still get its speed, and sometimes its creativity.

Does that mean you no longer need to learn to program? No. Think of the step from a slide rule to a calculator. A calculator lets you do things that were impossible before, and still, for the first years of primary school, children don't get one: they first learn what multiplying is. An LLM is a new calculator, a fuzzy one. What you still need to learn is the way of thinking behind code: how to cut a problem into smaller problems, how a process runs step by step, what an if-else decides. That is why in class I sometimes ask you to close your laptop and work a problem out with pen and paper first. You learn to think with your hands, and only then open the laptop. The picture of your code is where that thinking shows: if you can explain it, I know you understand what you built.

6.7 What the picture of your project should look like

Conceptually, what does the code for your plots do? When we drew it on the board in lesson 6, it came out much the same for almost everyone (figure 121).

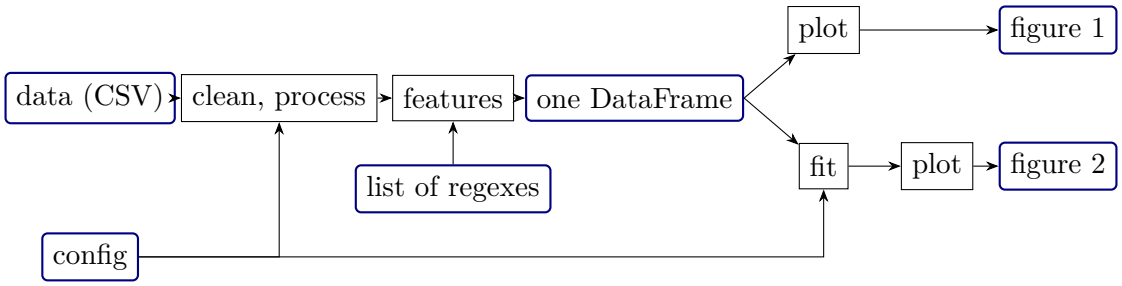


Figure 121: The flow most of your projects share: data and settings in, one table of features, a route per plot.

You start with data, a CSV, and settings. A first step cleans and processes the data: timestamps, perhaps anonymising. Then you make features: time features, and above all regular expressions. Those regexes live apart, as a list of thirty or eighty, and one function applies them all. The result is one DataFrame with all your features, which every plot uses. Some plots come straight from that DataFrame; others first go through a step such as a distribution fit, or set two things against each other. Every route ends in a figure.

Why doesn't atlas draw your code like that? Because you have fifty notebooks, mine and your own, each with six experiments mixed together, and the flow is spread over the whole repository. What I would do is start a new, clean project and move the pieces into it:

- a folder with its own `pyproject.toml` and its own `.venv`, with the dependencies you need (goad for the pipelines and plots, and for instance `sentence-transformers` for a model from Hugging Face);
- `data/raw` and `data/processed`;
- a package under `src/` with a real name, and a `main.py` that runs the flow above in broad strokes: process, add the regex features, make the plots;
- perhaps `scripts/` and `tests/` for pytest;
- lefthook, which you installed in lesson 1, so that every commit runs `ruff format` (tidy lines), `ruff check` (inconsistencies and sloppiness) and `ty` (types: if one function says it gives a DataFrame, the next must take one).

Use type hints and classes. Point atlas at that project, and the picture should show one flow from start to end, like the typed sunspot program of figure 112, instead of loose fragments.

6.8 Atlas in your final assignment

The command is `atlas-code`, because other tools install an `atlas` too; add `alias atlas=atlas-code` to your shell if you want the short name. Install it from `https://github.com/raoulg/atlas`: there is an install script for macOS and Linux, one for Windows, and a Homebrew formula. In your repository run `atlas-code view`, pick the

entry point that makes your plots in the dropdown, and open the steps you want to show. Atlas writes one HTML file and keeps your code on your own machine.

There are two other ways in. *Atlas Studio* is a desktop app for macOS and Windows: the picture, your git history, a preview for Markdown, HTML and PDF, and a terminal in one window, with its own `atlas-code` inside, so you need nothing else installed. Reading a repository the first time takes a while; after that it opens in a moment. Studio is not signed with a paid identity, so the first time macOS asks you to allow it under *Privacy & Security*, and a laptop managed by your employer may not let you install it at all. For that case there is a GitHub workflow that runs atlas on GitHub's machines on every push and keeps the picture in your README current; it needs nothing on your laptop[13].

The atlas picture of your code is required, and it is the baseline: it is what I look at when I assess your code. In the assessment I will sit next to you and ask you to explain that picture: where your data comes from, which steps it goes through, where each plot is made, and what the red dots ask. If you get atlas to draw a picture as calm as the typed sunspot program, you have enough grip on your code, and then you may also describe the picture in your report, or in an appendix beside it. A diagram of your own comes after atlas, never instead of it: atlas draws the code you actually wrote, and a diagram can be beautiful and have nothing to do with the code. Your own diagram is a nice-to-have, to present the atlas picture more clearly or to highlight what matters in it; it does not replace it. If your own picture and atlas's disagree, I would like to see why; there may well be a mistake in atlas, and I would only be glad to find it. A tidy picture is easy to explain. If yours looks messy, that is not a failure; it is the question to work on: what would you change in your code so that the picture tells the story you meant? Just as with a model of your data, the aim is not a perfect picture. It is a picture you can explain.

[13] `docs/reference/ci.md` in the atlas repository; your agent can set it up from those instructions.